POLITECNICO DI BARI

DIPARTIMENTO DI INGEGNERIA ELETTRICA E DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN
INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE

TESI DI LAUREA IN CONTROLLO DIGITALE

STUDIO DELL'INTERFACCIA GRAFICA DI CONTROLLO PER UN SISTEMA SCARA DI SCRITTURA E DISEGNO

RELATORE:
PROF. PAOLO LINO

CORRELATORI:
DOTT. NOME COGNOME
PROF./DOTT. NOME COGNOME

Laureando:
NICOLO'
ANDREA DUMAS

ANNO ACCADEMICO 2024-2025

Politecnico di Bari

LIBERATORIA ALLA CONSULTAZIONE DELLA TESI DI LAUREA DI CUI ALL'ART.4 DEL REGOLAMENTO DI ATENEO PER LA CONSULTAZIONE DELLE TESI DI LAUREA (D.R. n. 479 del 14/11/2016).

Il sottoscritto_____matricola_____

Corso di Laurea

_____

autore della presente tesi di Laurea dal titolo

_____

_____

Parola chiave_____

Abstract_____

_____

_____

_____

□ Autorizza

□ Non autorizza

la consultazione della presente tesi, fatto divieto a chiunque di riprodurre in tutto o in parte quanto in essa contenuto.

Bari, XX/XX/XXXX

Firma_____

La/il sottoscritta/o,

Cognome: _____

Nome: _____

Matricola: _____

Data Seduta Laurea: ____/____/_____.

dichiara

che la tesi cartacea consegnata in data odierna è conforme a quella caricata su esse3, quindi, approvata dal Relatore Prof. _____

(in stampatello)

Bari, ____/____/_____

Firma

_____

Si può resistere ad
un'invasione da parte
di un esercito ma non ad un'idea
il cui momento è giunto.
Alexander Dumas

INDICE

ELENCO DELLE FIGURE Pag 1

ACRONIMI E ABBREVIAZIONI Pag 3

Introduzione e Panoramica del Progetto: Contesto e Obiettivi della Tesi Pag 4

Capitolo 1: SISTEMA SCARA DESKARM Pag 6

1.1 Il Confronto tra i Manipolatori

1.2 Il Manipolatore SCARA

1.3 Come Progettare la Modellazione e Simulazione di un Manipolatore SCARA

1.4 Struttura Principale di DESKARM

1.5 Attuatori

1.6 Riduttori

Capitolo 2: MODELLISTICA, PIANIFICAZIONE E CONTROLLO DEL MANIPOLATORE Pag 19

2.1 Rappresentazione dello Spazio Operativo

2.2 Cinematica Diretta e Inversa

2.3 Cinematica Differenziale e Matrice Jacobiana

2.4 Pianificazione del Moto e della Traiettoria

2.4.1 Motion Planning

2.4.2 Trajectory Planning

2.4.3 Continuous Path e Point to Point

2.5 Modellistica Dinamica e Architettura di Controllo

Capitolo 3: FIRMWARE UPDATE Pag 25

Unità di Controllo: Microcontrollore STM32

3.1 Modellistica e Simulazione Dinamica

3.2 Evoluzione della Strategia di Controllo

3.3 Architettura del Firmware da Monolite a Modulare

3.4 Sicurezza Attiva

3.5 Codici Principali per l'Azionamento e il Controllo del Manipolatore

3.6 Implementazione delle Melodie

Capitolo 4: ARCHITETTURA DELL'INTERFACCIA GRAFICA DI CONTROLLO Pag 34
4.1 Algoritmi Fondamentali per il Funzionamento di DESKARM
4.2 Frontend della HMI
4.3 Architettura di Backend e Protocollo di Comunicazione Seriale
4.3.1 Backend di Basso Livello
4.3.2 Backend di Alto Livello
Capitolo 5: GUIDA ALL'USO DELL'INTERFACCIA GRAFICA Pag 67
5.1 Configurazione e Connessione Seriale
5.2 Definizione del Workspace
5.3 Menu Customization
5.4 Menu Principale
5.4.1 TEXT MODE
5.4.2 DRAWING MODE
5.4.3 IMAGE MODE
5.4 Modalità di Lavoro
5.4.1 TEXT MODE
5.4.2 DRAWING MODE
5.4.3 IMAGE MODE
5.5 Control Motion Menu
5.6 Gestione e Controllo della traiettoria
6.7 Modalità Debug e Simulazione Dinamica
Capitolo 6: IMPLEMENTAZIONI SOFTWARE E HARDWARE Pag 74
6.1 Stato Iniziale e Problemi Iniziali
6.2 Riprogettazione Firmware
6.3 Evoluzione dell'interfaccia
6.4 Modalità Debug e Simulazione Virtuale e Reale Dinamica
6.5 Schema Elettronico
6.6 Sviluppo della PCB
6.7 Design dello Scrittoio
6.8 End Effector e Micro-Servo

Capitolo 7: TEST DI FUNZIONAMENTO Pag 82

7.1 Test sul robot as-received

7.2 Test con robot modificato

CONCLUSIONI E SVILUPPI FUTURI Pag 93

BIBLIOGRAFIA E SITOGRAFIA Pag 96

RINGRAZIAMENTI Pag 98

Elenco delle Figure

Figura 1.1: Manipolatore Cartesiano - Pagina 6
Figura 1.2: Manipolatore Gantry - Pagina 7
Figura 1.3: Manipolatore Cilindrico - Pagina 7
Figura 1.4: Manipolatore Sferico - Pagina 7
Figura 1.5: Manipolatore Antropomorfo - Pagina 8
Figura 1.6: Manipolatore Scara a 4 gradi di libertà - Pagina 8
Figura 1.7: Azionamenti Manipolatore Scara - Pagina 9
Figura 1.8: Formule del Manipolatore a due gradi di libertà - Pagina 10
Figura 1.9: Workspace del manipolatore - Pagina 10
Figura 1.10: Schema di Progettazione Manipolatore Scara - Pagina 12
Figura 1.11: Funzionamento attuatore idraulico - Pagina 13
Figura 1.12: Attuatore pneumatico - Pagina 14
Figura 1.13: Attuatore non convenzionale - Pagina 14
Figura 1.14: Attuatore Nema 17 - Pagina 15
Figura 1.15: Driver TIDRV8825 - Pagina 16
Figura 1.16: Riduttori MG17-G10 e MG17-G5 - Pagina 17

Figura 2.1: Rappresentazione schematica di un manipolatore - Pagina 19
Figura 2.2: Traiettoria a percorso continuo - Pagina 22
Figura 2.3: Traiettoria punto per punto - Pagina 23

Figura 3.1: STM32F411RE Microcontroller - Pagina 26
Figura 3.2: Implementazione controllo feed forward di velocità - Pagina 28
Figura 3.3: Pendenza m della retta - Pagina 29

Figura 5.1: Connessione Seriale - Pagina 67
Figura 5.3: Workspace Digitale Lineare - Pagina 67
Figura 5.2: Workspace Digitale Curvo - Pagina 67
Figura 5.4: Customization Menu - Pagina 68
Figura 5.5: Application Menu - Pagina 69
Figura 5.6: TEXT MODE - Pagina 69
Figura 5.7: DRAWING MODE - Pagina 70
Figura 5.8: IMAGE MODE - Pagina 71
Figura 5.9: Control Motion Menu - Pagina 72
Figura 5.10: Trajectory Control Menu - Pagina 73

Figura 6.1: Schema .ino iniziale - Pagina 74
Figura 6.2: Schema elettrico iniziale - Pagina 75
Figura 6.3: Schema Pin-in/out del controllore STM32 - Pagina 75
Figura 6.4: CNC-Shield - Pagina 76
Figura 6.5: Interfaccia Grafica Iniziale - Pagina 77
Figura 6.6: Schema Elettrico Attuale - Pagina 77
Figura 6.7: PCB DUMARM - Pagina 78
Figura 6.8: PCB montata sul controllore - Pagina 78
Figura 6.9: Vano Tecnico - Pagina 79
Figura 6.10: DESKARM rivestito - Pagina 79
Figura 6.11: Scrittoio Progetto iniziale - Pagina 80
Figura 6.12: End-Effector - Pagina 80
Figura 6.13: Mini Servo MG90S - Pagina 81

Figura 7.1: Test di Scrittura - Pagina 82
Figura 7.2: Grafico senza ottimizzazione del testo - Pagina 83

Figura 7.3: Disegno Linea 1 - Pagina 84
Figura 7.4: Disegno Spirale e Cerchio - Pagina 84
Figura 7.5: Grafico senza ottimizzazione della linea 1 - Pagina 85
Figura 7.6: Grafico senza ottimizzazione della spirale e del cerchio - Pagina 86
Figura 7.7: Fotografia di partenza - Pagina 87
Figura 7.8: Disegno del Manipolatore - Pagina 87
Figura 7.9: Grafico senza ottimizzazioni dell'immagine - Pagina 88
Figura 7.10: Disegno Linea 2 - Pagina 90
Figura 7.11: Grafico Linea con ottimizzazioni - Pagina 90
Figura 7.12: Grafico Linea con Polinomio di quinto grado - Pagina 91

Acronimi e Abbreviazioni

DEI Dipartimento di Ingegneria Elettrica e dell'Informazione

HMI Human Machine Interface

SCARA Selective Compliance Assembly Robot Arm

PID Proporzionale-Integrale-Derivativo

ID Identification

FMS Flexible Manufacturing Systems

SMA Shape Memory Alloy

ARR Auto-Reload Register

CCR Capture Compare Register

TI DRV8825 Texas Instruments Driver(8825)

SE Special Euclidean

FLSC First Linear Second Curved

PWM Pulse -widht Modulation

Introduzione e Scopo della Tesi

Entrare nel mondo della robotica e del controllo digitale significa partecipare a una delle sfide più affascinanti dell'ingegneria moderna: dare "intelligenza" e precisione alla materia inerte. La robotica industriale moderna affonda le sue radici nell'ambizioso traguardo di sollevare l'uomo dalle mansioni più logoranti e alienanti, delegando alle macchine compiti che richiedono una ripetibilità e una rapidità impossibili da eguagliare per la natura umana.

In questo affascinante percorso evolutivo, l'introduzione dell'architettura SCARA (Selective Compliance Assembly Robot Arm) nei primi anni '80 ha segnato uno spartiacque fondamentale per l'automazione dei processi di assemblaggio planare. La genialità di questa tecnologia, nata originariamente in Giappone [4], risiede nella sua "cedevolezza selettiva": un compromesso cinematico magistrale che combina una flessibilità controllata lungo gli assi orizzontali, essenziale per assecondare i delicati movimenti di inserimento dei componenti, con una rigidità assoluta e incrollabile lungo l'asse verticale. Questa peculiare dinamica ha consacrato lo SCARA come il dominatore indiscusso per tutte le lavorazioni industriali che operano dall'alto verso il basso, tipiche delle frenetiche catene di montaggio dedicate al pick-and-place.

Il progetto DESKARM nasce proprio dall'intuizione di decontestualizzare questa eccellenza meccanica, trasponendo la robustezza e la velocità di un manipolatore industriale in un ambito squisitamente creativo, artistico e di altissima precisione. Studiare e perfezionare un sistema simile significa confrontarsi con il "Controllo Digitale" nella sua forma più pura poiché non si tratta solo di far muovere un motore ma di orchestrare segnali PWM, algoritmi PID e calcoli cinematici in tempo reale affinché la punta di una penna possa danzare su un foglio con la stessa naturalezza di una mano esperta.

Il presente lavoro di tesi documenta lo sviluppo software e la progettazione di componentistica hardware per la sperimentazione su un sistema robotico a due gradi di libertà (2-DOF) affidabile e capace di interpretare geometrie complesse o testi con una fluidità dal sapore quasi umano. Questo studio ha mirato a trasformare un robot SCARA elementare in un dispositivo in grado di essere utilizzato professionalmente nei contesti dell'insegnamento, la scrittura e il supporto a processi creativi.

Lo studio ha preso le mosse da una meticolosa analisi diagnostica dell'elettronica preesistente, per poi continuare con la programmazione e il testing dell'interfaccia grafica e delle prestazioni cinematiche del robot. Il firmware del microcontrollore STM32 è stato completamente riscritto, abbandonando la pregressa architettura monolitica in favore di un paradigma modulare e scalabile. È stato così implementato un sofisticato sistema di controllo PID potenziato da un'azione di feedforward di velocità, un accorgimento che ha permesso di abbattere drasticamente il carico computazionale della CPU, garantendo un inseguimento corretto della traiettoria (tracking) e una risposta dinamica fluida anche in regime di alte accelerazioni.

Parallelamente al rinnovamento del nucleo di basso livello, si è resa necessaria la creazione di un ponte comunicativo empatico tra l'operatore e il robot. Questo si è concretizzato in un'interfaccia uomo-macchina (HMI) in Python tanto intuitiva quanto potente, strutturata su tre versatili modalità operative: Image, Text e Drawing. Per garantire la massima stabilità e operare in totale sicurezza, l'intero strato software è stato blindato da una Safety Architecture multilivello, progettata per vigilare attivamente sul

sistema. E’ stata eseguita la validazione cinematica preventiva delle traiettorie prima della loro attuazione, scongiurando a monte il rischio di errori logici, singolarità matematiche o catastrofiche collisioni meccaniche.
Infine, la storia di questa evoluzione e la cura per l'ergonomia trovano il loro naturale compimento nel superamento dei limiti fisici di partenza: l'integrazione, tramite tecniche di prototipazione rapida e un di un modulo pen holder custom. Questo speciale end-effector, azionato da un reattivo microservo MG90S, si è rivelato fondamentale per abilitare il controllo dell'asse Z, consentendo la gestione discreta dei tratti e l'esecuzione dei cosiddetti "Smart Jumps", ovvero i movimenti aerei e silenziosi a penna sollevata.
Il coronamento di questo percorso di sviluppo si materializza nello chassis del sistema ovvero uno scrittoio dedicato, progettato ex novo per garantire in un'unica soluzione robustezza strutturale e massima trasportabilità. Al suo interno, in un vano tecnico nascosto, convivono razionalmente l'alimentazione di potenza e una PCB customizzata.
Lo studio del sistema SCARA DESKARM non è stato solo un esercizio di programmazione, ma ha dimostrato che la robotica non è fatta solo di metallo e circuiti, ma di visioni capaci di trasformare la rigidità della catena di montaggio nella fluidità del segno grafico.
È un invito a non fermarsi alla superficie dei componenti, ma a esplorare l'armonia che si crea quando la matematica del controllo digitale incontra la realtà fisica, rendendo l'automa un’estensione della volontà umana.

Capitolo 1

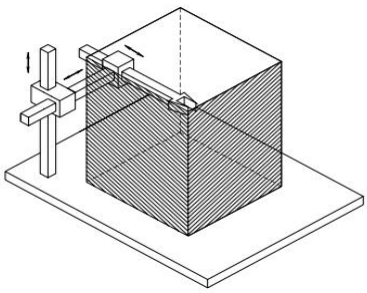
SISTEMA SCARA DESKARM

L'architettura hardware del manipolatore SCARA costituisce le fondamenta fisiche su cui si basa l'intera precisione del sistema di tracciamento. Questo capitolo illustra il tipo di Manipolatore e le specifiche fisiche dei bracci robotici cercando di introdurre il mondo del controllo digitale e della robotica in un aspetto storico e teorico per far comprendere al lettore le esigenze iniziali che hanno dato vita a questo tipo di manipolatore per poi descriverlo nelle sue componenti principali.

1.1 CONFRONTO TRA I MANIPOLATORI

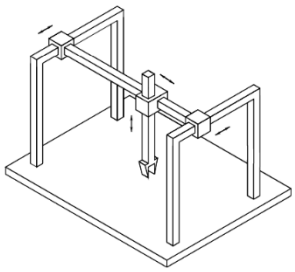
Secondo la norma ISO TR/8373-2.3, lo standard internazionale che definisce il vocabolario [2]-[3], la terminologia e le definizioni fondamentali per i robot e i dispositivi robotici, definisce il robot industriale come: *"Un manipolatore con più gradi di libertà, governato automaticamente, riprogrammabile, multiscopo, che può essere fisso sul posto o mobile per utilizzo in applicazioni di automazioni industriali"*.
La classificazione topologica dei manipolatori robotici industriali si fonda sull'analisi della loro catena cinematica fondamentale, che determina la geometria dello spazio di lavoro e la complessità di controllo.

Il **manipolatore Cartesiano** adotta una configurazione PPP (Prismatico-Prismatico-Prismatico) con assi mutuamente ortogonali; sebbene offra una cinematica totalmente lineare e disaccoppiata su un volume a parallelepipedo, presenta ingombri strutturali che superano nettamente l'area operativa utile.



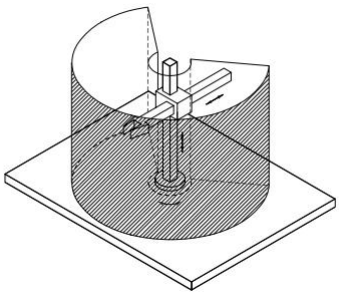
1.1 Manipolatore Cartesiano

La sua naturale evoluzione per la logistica pesante è il **manipolatore Gantry**, che montando gli assi di scorrimento su un ponte sospeso svincola il robot dai limiti a terra, garantendo la movimentazione di carichi estremi su volumi immensi senza flessioni.



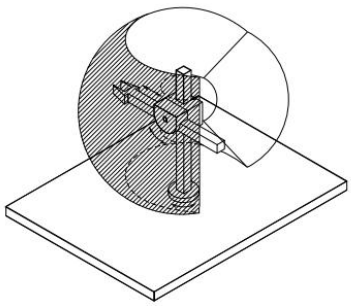
1.2 Manipolatore Gantry

Introducendo un grado di libertà rotazionale alla base si ottiene **il manipolatore Cilindrico (RPP)**, il cui spazio operativo assume la forma di un cilindro cavo: questa architettura minimizza l'inerzia planare rispetto al cartesiano e risulta estremamente rigida ed efficiente nelle operazioni di asservimento macchine.



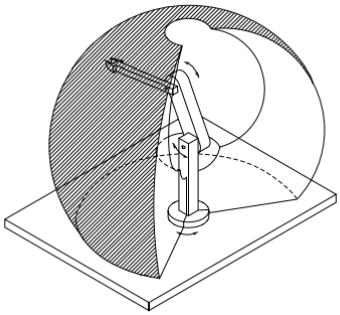
1.3 Manipolatore Cilindrico

Elevando ulteriormente la complessità rotazionale otteniamo il **manipolatore Sferico (RRP)** mappa un settore sferico tramite una rotazione azimutale, un'elevazione della spalla e un'estensione lineare telescopica, offrendo una destrezza intermedia adatta a raggiungere punti distanti mantenendo un baricentro compatto.



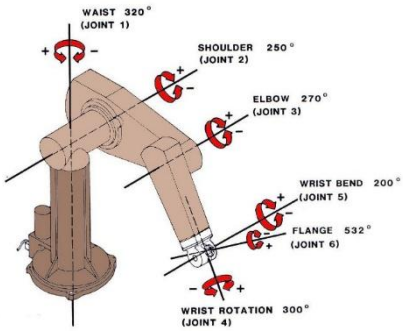
1.4 Manipolatore Sferico

Il culmine assoluto della destrezza spaziale è tuttavia rappresentato dal **manipolatore Antropomorfo (RRR):** emulando biologicamente il braccio umano, garantisce la massima capacità di aggirare ostacoli tridimensionali con un ingombro minimo al suolo. Il prezzo ingegneristico di tale agilità risiede però in una cinematica fortemente non lineare, matematicamente complessa da controllare e fisiologicamente esposta a singolarità.



1.5 Manipolatore Antropomorfo

L'analisi comparativa di queste architetture evidenzia, per profondo contrasto ingegneristico, la validità della topologia SCARA adottata per il DESKARM: rinunciando all'inutile destrezza sferica dell'antropomorfo e superando i severi ingombri del cartesiano, lo SCARA isola la dinamica orizzontale da quella verticale, offrendo la sintesi perfetta per un tracciamento vettoriale 2D ad altissima accelerazione, precisione e ripetibilità.
Solitamente è rappresentato in configurazione a 4 o 5 DOF (tre rotoidali e uno prismatico), lo SCARA garantisce una ripetibilità superiore e velocità di ciclo molto elevate, poiché i primi due giunti possiedono un'inerzia ridotta e il movimento lungo Z è intrinsecamente disaccoppiato dalla dinamica planare.

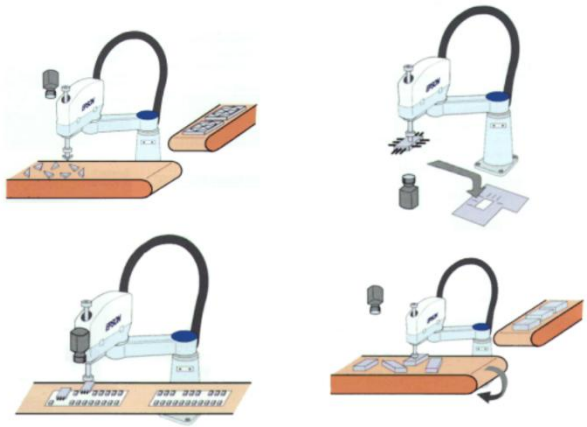


1.6 Manipolatore Scara a 4 gradi di libertà

1.2 Il Manipolatore SCARA

Prima di analizzare le specifiche tecniche di questo progetto, è tuttavia essenziale inquadrare il mondo del controllo digitale e della robotica sotto un profilo storico e teorico. La robotica industriale rappresenta una disciplina ormai giunta a piena maturità tecnologica, incentrata sulla progettazione, il controllo e l'applicazione di manipolatori all'interno di ambienti strutturati. I primi robot industriali videro la luce negli anni '60 [5],

nati dalla convergenza di due tecnologie preesistenti: le **macchine a controllo numerico**, destinate alla manifattura di precisione, e **i teleoperatori**, concepiti per la manipolazione remota di materiali pericolosi. Storicamente, l'integrazione di questi sistemi ha guidato



1.7 Azionamenti Manipolatore Scara

l'evoluzione dei paradigmi produttivi verso tre livelli fondamentali: **l'automazione rigida**, destinata alla produzione di massa con sequenze fisse; **l'automazione programmabile**, che permette di variare le sequenze per gestire lotti variabili; e infine **l'automazione flessibile**, che minimizza i tempi di riprogrammazione passando dinamicamente da un prodotto all'altro. Secondo la definizione pionieristica del Robot Institute of America, nel 1980 [1], il robot si configura pienamente in questi scenari come un manipolatore multifunzionale e riprogrammabile, destinato alla movimentazione dei materiali, alla manipolazione manifatturiera e alla misurazione spaziale.

Nel corso dei decenni, l'automazione si è evoluta dai primi sistemi isolati e pesanti, confinati in rigide celle di sicurezza, verso architetture snelle e integrate. Il passaggio critico è avvenuto con l'adozione di sensori di coppia e feedback cinematici ad alta risoluzione, che hanno trasformato il robot da semplice esecutore di traiettorie cieche a un sistema intelligente e complesso. I moderni "cobot", robot collaborativi, ad esempio, sfruttano la riduzione degli ingombri e sofisticati algoritmi di controllo dell'impedenza per operare in spazi condivisi, superando il concetto di automazione segregata a favore di una collaborazione dinamica e intrinsecamente sicura tra uomo e macchina.
È proprio in questo fervente contesto di automazione avanzata e necessità di manipolazione rapida che si inserisce l'architettura SCARA, acronimo di *Selective Compliance Assembly Robot Arm*. Introdotto per ottimizzare i task di assemblaggio ad alta velocità e le operazioni di *pick and place* su un piano orizzontale, lo SCARA rappresenta una sintesi ingegneristica perfetta per tutte le lavorazioni ripetitive che avvengono dall'alto verso il basso. Sfruttando una struttura seriale tipicamente a quattro assi, in cui i primi due giunti rotoidali si muovono parallelamente al suolo simulando la dinamica di un braccio umano, questo robot sacrifica la capacità di aggirare ostacoli tridimensionali complessi in favore di una ripetibilità e di una velocità di ciclo impareggiabili. Il suo fondamento meccanico e concettuale risiede nella "cedevolezza selettiva" (Selective Compliance): il robot presenta un'estrema rigidezza lungo l'asse

verticale (Z), strettamente necessaria per resistere alle forze di inserzione, e una flessibilità finemente controllata sul piano orizzontale (X-Y). Questa configurazione permette di gestire lievi errori di tolleranza meccanica durante il posizionamento, facilitando l'allineamento dei componenti senza stressare i giunti. L'architettura ad assi paralleli garantisce inoltre una massa mobile ridotta, fondamentale per raggiungere accelerazioni elevatissime, come storicamente dimostrato da modelli industriali di successo serie: AdeptOne, capace di manipolare carichi operativi a regimi estremi mantenendo tolleranze micrometriche.
Il tipo di applicazione esplorato dal prototipo DESKARM, pur ereditando integralmente questa rigorosa impostazione ingegneristica, si discosta dal contesto manifatturiero quotidiano ed è stato progettato come un manipolatore SCARA a **due gradi di libertà (2-DOF)**, la cui architettura seppur in una forma ridotta è specializzata per implementare un azionamento di precisione specificamente calibrato per la scrittura e il disegno.

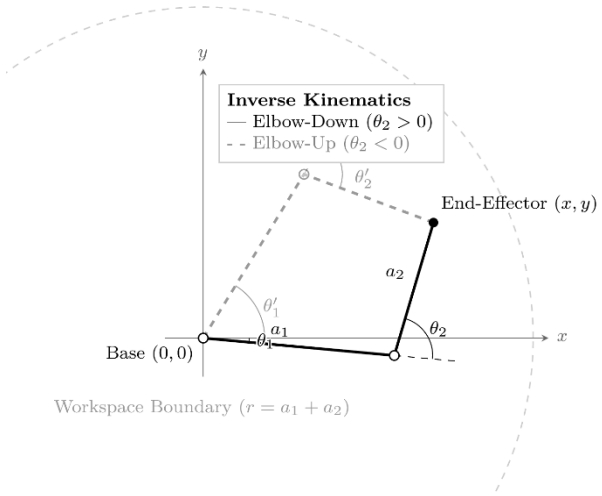
2-DOF Planar Manipulator

$$x = a_1 \cos\theta_1 + a_2 \cos(\theta_1 + \theta_2)$$
$$y = a_1 \sin\theta_1 + a_2 \sin(\theta_1 + \theta_2)$$

1.8 Formule del Manipolatore a due gradi di libertà

Sfruttando la naturale rigidezza verticale dello SCARA per mantenere una pressione costante sul foglio e la sua fluidità planare per tracciare linee continue, il DESKARM trasla i principi dell'industria pesante in uno strumento dove l'assenza di vibrazioni e il calcolo rigoroso della cinematica diventano il fulcro per azzerare l'errore di inseguimento, garantendo un tratto artistico o tecnico impeccabile.

Velocità e precisione nel piano orizzontale per scrittura e disegno
Nel disegno e nella tracciatura, lo SCARA eccelle grazie alla sua capacità di mantenere una forza di contatto costante lungo l'asse verticale pur seguendo traiettorie XY fluide.



1.9 Workspace del manipolatore

La rigidezza dei bracci nel piano orizzontale minimizza l'errore di inseguimento e le vibrazioni, assicurando una tracciatura senza "shuttering". La configurazione meccanica detta i limiti prestazionali: per spingere queste macchine al limite della precisione

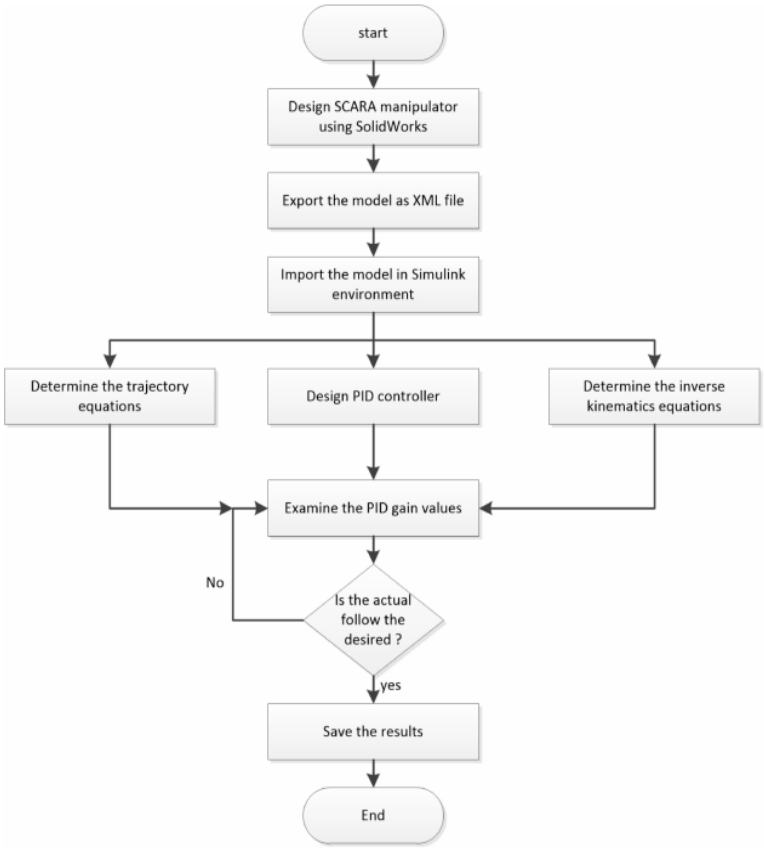
micrometrica, è però indispensabile un framework matematico che gestisca la cinematica e la dinamica in modo rigoroso.

1.3 PROGETTAZIONE DELLA MODELLAZIONE E SIMULAZIONE DI UN MANIPOLATORE SCARA:

Il presente paragrafo illustra il framework metodologico per la modellazione, la simulazione e la sintesi del controllo di un manipolatore robotico SCARA.

Il processo computazionale ha inizio con la progettazione CAD (Computer-Aided Design) della topologia meccanica e dei parametri inerziali del robot avvalendosi dell'ambiente di modellazione SolidWorks, i cui dati strutturali vengono successivamente esportati in formato XML per garantire l'interoperabilità tra le diverse piattaforme software. Tale file costituisce l'input fondamentale per l'ambiente di simulazione Simulink, all'interno del quale il modello virtuale viene importato per condurre l'analisi dinamica. A questo specifico stadio, l'architettura procedurale si dirama in tre task computazionali paralleli: la formulazione analitica delle equazioni di traiettoria per definire la rigorosa legge di moto nello spazio operativo, la sintesi dell'architettura di controllo ad anello chiuso tramite un regolatore Proporzionale-Integrale-Derivativo (PID), e la complessa risoluzione del problema cinematico inverso, strettamente necessaria per mappare le coordinate cartesiane desiderate dell'end-effector nelle corrispondenti variabili dello spazio dei giunti. Le risultanze di questi tre moduli teorici convergono nella fase di tuning analitico, in cui vengono accuratamente esaminati e tarati i valori di guadagno del controllore PID. Il flusso logico procede quindi verso un nodo di validazione condizionale che valuta le prestazioni di tracciamento (tracking performance), verificando analiticamente se la risposta dinamica del sistema simulato minimizza l'errore di inseguimento rispetto alla traiettoria di riferimento imposta. Qualora le specifiche di convergenza non siano soddisfatte, un ciclo di retroazione impone un'ottimizzazione iterativa dei parametri di guadagno del controllore; viceversa, se i criteri di stabilità e precisione cinematica risultano pienamente rispettati, il processo avanza sistematicamente verso l'acquisizione e la storicizzazione dei risultati sperimentali generati, culminando infine nel termine formale della procedura analitica.

Lo schema riporta la progettazione secondo quanto detto precedentemente:



1.10 Schema di Progettazione Manipolatore Scara

1.4 STRUTTURA PRINCIPALE DI DESKARM:

La struttura è composta da due bracci rigidi, denominati link, interconnessi da giunti rotoidali i cui assi di rotazione sono paralleli tra loro e perpendicolari al piano del moto (X-Y).

La geometria del robot può essere sintetizzata nei seguenti parametri:

• Link 1 (L_1): Collega la base fissa al primo giunto mobile.

• Link 2 (L_2): Collega il primo giunto all'estremità finale (End-Effector)

• End-effector

• Variabili di Giunto (θ1, θ2): Rappresentano gli angoli di rotazione dei motori stepper rispetto ai sistemi di riferimento locali, definendo la configurazione del robot nello spazio dei giunti.

-La catena cinematica è costituita da due link rigidi le cui lunghezze operative, misurate tra i centri degli assi di rotazione, sono state fissate a L_1= 0.128 m per il braccio primario (spalla-gomito) e L_2= 0.144 m per il braccio secondario (gomito-polso).

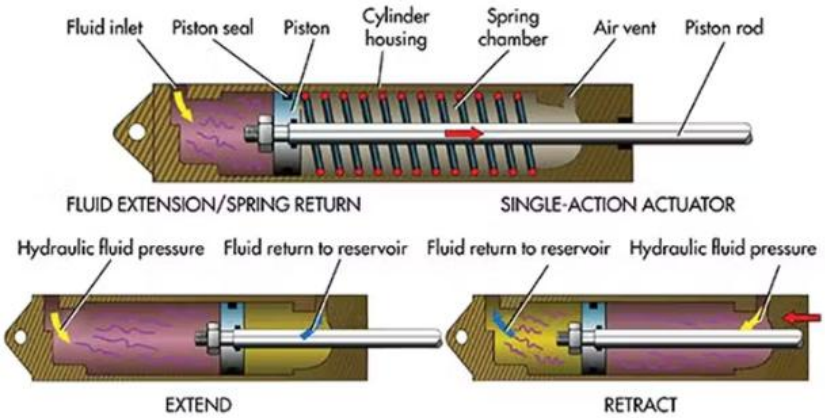
-L'attuazione è affidata a due motori stepper che forniscono la coppia necessaria per vincere l'inerzia dei bracci e gli attriti. Il movimento coordinato tra i due giunti permette

all'End-Effector, configurato come uno strumento di scrittura, di muoversi all'interno di uno spazio di lavoro bidimensionale.

1.5 ATTUATORI:

In questo paragrafo sono spiegati i principali attuatori esistenti e il motivo per cui si è optato per il modello di DESKARM. Nel vasto dominio della robotica industriale e di precisione, gli attuatori costituiscono l'interfaccia fisica vitale che converte l'energia potenziale in energia cinetica, generando il moto articolare. La scelta del paradigma di attuazione definisce in modo stringente le capacità dinamiche del manipolatore sia essa di natura fluida, termica o elettrica:

Attuatori Idraulici e Pneumatici: Gli attuatori idraulici impiegano fluidi incomprimibili ad alta pressione per generare forze immense, garantendo il più alto rapporto potenza-peso e risultando indispensabili per la robotica pesante e i carichi estremi; tuttavia,



1.11 Funzionamento attuatore idraulico

peccano in precisione millimetrica e richiedono infrastrutture ingombranti.

Di contro, gli attuatori pneumatici impiegano fluidi gassosi la cui intrinseca compressibilità termodinamica li rende "cedevoli" (compliant). Sebbene ideali per la soft robotics, tale caratteristica rende la regolazione fine della posizione estremamente complessa e soggetta a forti non-linearità. Sul versante opposto, l'attuazione idraulica sfrutta oli minerali incomprimibili, garantendo un'elevata rigidezza meccanica e un'eccellente tenuta statica della posizione. Ciononostante, l'impiego di tale tecnologia risulta ingegneristicamente incompatibile con un manipolatore desktop come il DESKARM: le servo-valvole proporzionali presentano risposte dinamiche non lineari dovute alla turbolenza dei fluidi, mentre l'ingombrante infrastruttura di pompaggio e il rischio congenito di micro-perdite violano i rigorosi requisiti di compattezza e pulizia operativa dell'ambiente di lavoro.

1.12 Attuatore pneumatico

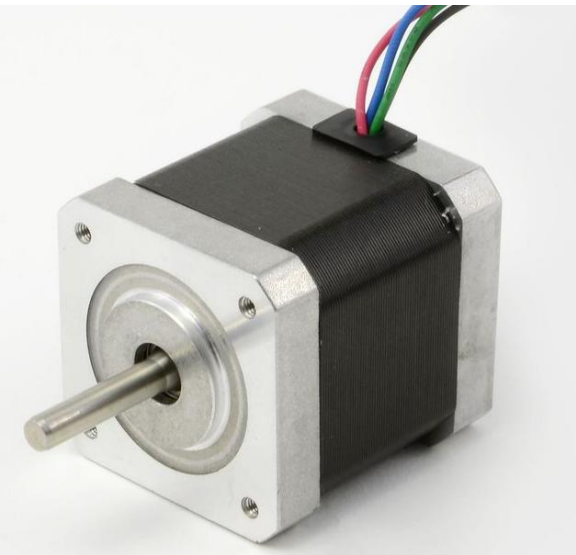
Attuatori Non Convenzionali: Includono tecnologie di frontiera come i materiali piezoelettrici o le leghe a memoria di forma (SMA). Questi sistemi offrono risoluzioni nanometriche o fungono da muscoli artificiali compatti, trovando applicazione in microrobotica e chirurgia, ma erogano escursioni meccaniche limitate.

1.13 Attuatore non convenzionale

Attuatori Elettrici: Rappresentano lo standard de facto per la robotica manifatturiera, i cobot e i plotter di precisione. Sfruttando l'interazione tra campi magnetici, offrono una pulizia operativa assoluta, facilità di miniaturizzazione e una sinergia perfetta con i moderni microcontrollori digitali. All'interno di questa famiglia, i servomotori (motori AC/DC brushless accoppiati a encoder ad alta risoluzione in anello chiuso) dominano per alte velocità e dinamiche complesse, ma richiedono algoritmi di controllo onerosi. Per applicazioni planari ad altissima risoluzione dove il mantenimento statico della posizione è prioritario, la scena è dominata dai motori passo-passo (stepper).

Per l'attuazione dei giunti planari sono stati selezionati due motori passo-passo in standard NEMA 17, scelti per l'ottimo compromesso tra prestazioni dinamiche, coppia erogata e costi contenuti. Le caratteristiche principali della configurazione adottata includono una risoluzione nativa di 200 passi/giro, che è stata incrementata a 3200 micro-

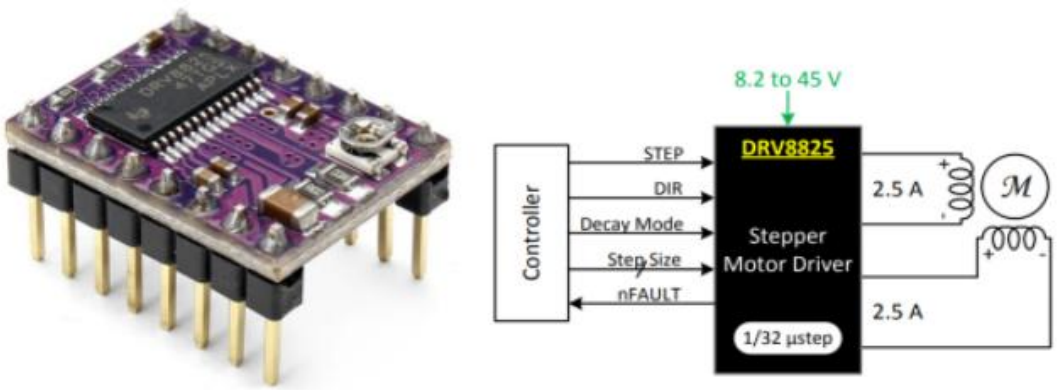
step/giro attraverso una configurazione di microstepping a 1/16 sui driver. Questa elevata risoluzione è fondamentale per garantire la fluidità del movimento necessaria durante le delicate operazioni di scrittura. Inoltre, per il controllo ad anello chiuso, ogni motore è equipaggiato con un encoder magnetico incrementale montato sull'albero posteriore; tale soluzione ha permesso di mantenere un design estremamente compatto per i link del manipolatore.



1.14 Attuatore Nema 17

L'architettura di attuazione del manipolatore DESKARM si fonda sull'impiego di motori passo-passo NEMA 17, scelti per la loro capacità di frazionare una rivoluzione meccanica completa in incrementi angolari discreti. Appartenenti alla categoria degli stepper ibridi, questi attuatori combinano i principi della riluttanza variabile e del magnete permanente tramite un rotore centrale a 50 denti, magnetizzato assialmente, e uno statore ad avvolgimenti multipli. La commutazione sequenziale di queste bobine genera nativamente 200 passi per giro, offrendo una risoluzione in cui ogni step corrisponde a un angolo di 1.8°. Questa struttura conferisce due vantaggi supremi per il disegno robotico: il controllo in anello aperto (open-loop), che permette all'unità STM32 di determinare la posizione teorica del braccio semplicemente contando gli impulsi direzionali, e l'elevata coppia di mantenimento (holding torque), che blocca rigidamente il rotore contrastando le forze di attrito laterale della penna sul foglio senza necessitare di continui micro-aggiustamenti algoritmici. Il controllo della velocità e della posizione si realizza attraverso la rigorosa mappatura dell'algoritmo PID sui registri dei timer hardware dell'unità logica STM32, la quale genera un'onda quadra tramite modulazione PWM. Nello specifico, la gestione dinamica della velocità angolare è affidata all'ARR, il cui valore definisce il periodo della forma d'onda, mentre il CCR mantiene un duty cycle

costantemente fissato al 50% per assicurare fronti di salita netti e inequivocabili. Poiché il fisiologico movimento a scatti di un motore stepper indurrebbe coordinate cartesiane fortemente discretizzate, trasformando la traiettoria di archi morbidi in geometrie a gradino e innescando vibrazioni deleterie, l'interfaccia di potenza è stata realizzata mediante i driver TI DRV8825 [20] riconfigurati a livello hardware per operare in una modalità avanzata.



1.15 Driver TIDRV8825

È stata implementata per questa architettura una metodologia a 1/32 di passo. In questo specifico regime, il driver non eroga una corrente impulsiva discreta, ma la modula utilizzando profili PWM complementari di natura sinusoidale e cosinusoidale; tale strategia permette di bilanciare il rotore in precise posizioni di equilibrio intermedie. L'implementazione del microstepping a 1/32 riduce l'incremento angolare minimo dell'albero motore a un precisissimo valore di 0.05625° per impulso, innalzando la risoluzione elettronica a 6.400 passi per giro.

Sebbene l'anello aperto fornisca una solida base, il DESKARM supera i limiti di tale configurazione integrando encoder magnetici incrementali sull'albero posteriore, permettendo al sistema di verificare l'esecuzione del passo e compensare eventuali perdite di sincronismo. L'accuratezza finale del tracciamento, tuttavia, è profondamente vincolata alla trasmissione meccanica.

1.6 RIDUTTORI:

Per garantire un'adeguata autorità inerziale, la catena cinematica impiega riduttori epicicloidali interfacciati tramite pulegge in rapporto 1:1.

Il primo giunto adotta un riduttore 10:1 per erogare la coppia elevata necessaria a vincere l'inerzia globale, mentre il secondo sfrutta un rapporto 5:1 per privilegiare la velocità operativa nei dettagli grafici. L'impiego di tali stadi introduce la problematica del gioco meccanico (backlash): l'encoder, solidale all'albero motore a monte, risulta cieco rispetto alle tolleranze del riduttore a valle, permettendo micro-spostamenti passivi che causerebbero deviazioni millimetriche del tratto.

1.16 Riduttori MG17-G10 e MG17-G5

Per compensare questa invisibilità algoritmica, è stata integrata una vite di precarico sul primo stepper, che forza l'ingranamento costante dei satelliti, restituendo al sistema un'assoluta rigidità torsionale, Anche i tracciamenti prima e dopo questa accortezza hanno migliorato notevolmente le microoscillazioni che il manipolatore ha avuto anche dopo l'evoluzione del Firmware del microcontrollore che vedremo nel prossimo capitolo.

Questa massiccia frammentazione del moto e la solidità meccanica, propagate attraverso le equazioni della cinematica diretta, permettono di quantificare analiticamente la risoluzione spaziale teorica all'end-effector. Il primo giunto gode di una drastica demoltiplicazione angolare che porta la sua risoluzione a

$$\Delta\theta_1 = 0.05625°/10 \ = \ 0.005625° \, (9.81 \times 10^{-5} radianti)$$

Nello scenario cinematico più critico di massima estensione radiale un singolo micro-passo al primo motore induce uno spostamento d'arco alla punta della penna pari a:

$$s_1 = (L_1 + L_2) \times \Delta\theta_1 \cong \ 0.0267 \, mm$$

Parallelamente, il secondo giunto esibisce una risoluzione angolare di

$\Delta\theta_1 = 0.05625°/5 \ = \ 0.01125°(1.96 \times 10^{-4} radianti)$

che sul braccio di leva $L_2 \ = \ 144\ mm$ genera uno spostamento cartesiano pari a:

$$s_2 = L_2 \times \Delta\theta_2 \cong \ 0.0283\ mm$$

Tali inequivocabili risultanze analitiche dimostrano matematicamente che l'architettura elettromeccanica del DESKARM vanta una risoluzione spaziale sub-millimetrica confinata nell'ordine di grandezza dei 28 micrometri per step logico, una tolleranza dimensionale di gran lunga inferiore allo spessore fisico dell'inchiostro rilasciato da un comune strumento da disegno, garantendo in tal modo una riproduzione vettoriale scorrevole, priva di artefatti e visivamente ineccepibile.

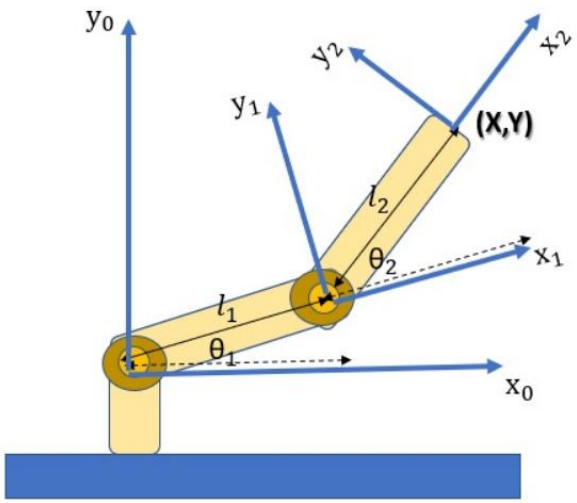
Capitolo 2

MODELLISTICA, PIANIFICAZIONE E CONTROLLO DEL MANIPOLATORE

Il presente capitolo illustra i modelli cinematici, le strategie di pianificazione del moto e le semplificazioni dinamiche adottate per il controllo

Dal punto di vista analitico, lo studio del manipolatore richiede la definizione di:

1. **Cinematica Diretta**: per determinare la posizione dell'End-Effector nello spazio cartesiano note le posizioni angolari dei giunti.
2. **Cinematica Inversa**: per calcolare gli angoli dei link necessari a raggiungere una specifica coordinata (x, y) sulla superficie di scrittura.
3. **Dinamica**: per modellare le forze e le coppie in gioco, considerando le masse dei bracci e le caratteristiche di coppia-velocità dei motori stepper.



2.1 Rappresentazione schematica di un manipolatore

La cinematica studia il moto del manipolatore a prescindere dalle forze che lo generano. Per il DESKARM, operando su un piano 2D, la trattazione matematica risulta semplificata rispetto allo spazio tridimensionale, pur mantenendo il rigore formale delle trasformazioni spaziali. La cinematica studia il moto del manipolatore a prescindere dalle forze che lo generano. Per il DESKARM, operando su un piano 2D, la trattazione matematica risulta semplificata rispetto allo spazio tridimensionale, pur mantenendo il rigore formale delle trasformazioni spaziali.

2.1 RAPPRESENTAZIONE DELLO SPAZIO OPERATIVO:

Nello spazio tridimensionale, la posa di un corpo rigido è descritta da una matrice di trasformazione omogenea $T \in SE(3)$ di dimensioni 4 X 4.
Nel caso di un manipolatore strettamente planare, i gradi di libertà operativi si riducono a tre: due traslazioni lungo gli assi x e y, e una singola rotazione θ attorno all'asse z perpendicolare al piano di lavoro.
La matrice di trasformazione si riduce pertanto al gruppo speciale euclideo $SE(2)$:

$$T = \begin{bmatrix} \cos\theta & -\sin\theta & x \\ \sin\theta & \cos\theta & y \\ 0 & 0 & 1 \end{bmatrix}$$

L'orientamento finale dell'organo terminale θ è dato dalla somma algebrica delle variabili di giunto: $\theta = \theta_1 + \theta_2$ Data la natura del task (scrittura con una penna), l'orientamento θ risulta ridondante ai fini operativi; l'interesse di controllo si concentra esclusivamente sul vettore di posizione $p = [x, y]^T$

2.2 CINEMATICA DIRETTA E INVERSA:

La Cinematica Diretta [6] mappa lo spazio dei giunti nello spazio operativo. Attraverso elementari considerazioni trigonometriche, le equazioni dirette per la posizione dell'end-effector risultano:

$$\begin{cases} x = L_1 * \cos(q_0) + L_2 * \cos(q_0 + q_1) \\ y = L_1 * \sin(q_0) + L_2 * \sin(q_0 + q_1) \end{cases}$$

Il problema della Cinematica Inversa consiste nel determinare i valori angolari $[q_0, q_1]^T$necessari per raggiungere una desiderata coordinata $[x, y]^T$. Essendo un sistema non lineare, esso ammette una soluzione analitica di natura geometrica, basata sul teorema del coseno:

$$\begin{cases} q_1 = \pm\cos^{-1}\left(\dfrac{x^2 + y^2 - L_1^2 - L_2^2}{2 * L_1 L_2}\right) \\ q_0 = atan2(y, x) - atan2(L_1 \sin q_1, L_1 + L_2 \cos q_1) \end{cases}$$

La presenza del doppio segno $\pm$ nel calcolo di $q_1$evidenzia l'esistenza di due molteplicità di soluzione per ogni punto del workspace, comunemente definite configurazioni in alto e gomito in basso. Per garantire la continuità del moto ed evitare singolarità o salti discontinui nello spazio dei giunti, il sistema adotta un approccio basato sulla minima distanza: istante per istante, l'algoritmo seleziona la configurazione che minimizza la norma euclidea $\|q_k - q_{k-1}\|$ rispetto alla posa precedente.

2.3 CINEMATICA DIFFERENZIALE E MATRICE JACOBIANA:

Per controllare le velocità, si introduce la matrice Jacobiana $J(q)$, che esprime la relazione lineare $\dot{x} = J(q) * \dot{q}$. Derivando le equazioni della cinematica diretta rispetto al tempo, si ottiene lo Jacobiano analitico del sistema:

$$J(q) = \begin{bmatrix} -L_1 * \sin(q_0) - L_2 * \sin(q_0 + q_1) & -L_2 * \sin(q_0 + q_1) \\ L_1 * \cos(q_0) + L_2 * \cos(q_0 + q_1) & L_2 * \cos(q_0 + q_1) \end{bmatrix}$$

Nell'implementazione discreta del DESKARM, l'inversione della matrice Jacobiana [6] per il calcolo delle velocità ai giunti non viene calcolata analiticamente. Le velocità $\dot{q}$ e le accelerazioni $\ddot{q}$ vengono approssimate per differenza finita sulle posizioni calcolate dalla cinematica inversa, con un passo di campionamento costante T_c. Questo approccio computazionale mitiga intrinsecamente il problema dell'inversione in prossimità delle singolarità cinematiche.

2.4 PIANIFICAZIONE DEL MOTO E DELLA TRAIETTORIA:

La generazione del riferimento per il robot è rigorosamente divisa in due stadi algoritmici sequenziali, operanti nello spazio cartesiano.

2.4.1 MOTION PLANNING:

Il livello geometrico si occupa di definire il percorso puro nello spazio operativo, ovvero una sequenza di coordinate x e y indipendenti dal tempo. Il sistema scompone le primitive geometriche complesse (testi, immagini o archi) in una sequenza unificata di segmenti lineari contigui (polilinee). Viene inoltre applicata un'ottimizzazione spaziale per eliminare i punti ridondanti che risiedono al di sotto di una tolleranza geometrica prefissata, garantendo un dataset più leggero per il controllo di basso livello.

2.4.2 TRAJECTORY PLANNING

La traiettoria aggiunge la dimensione temporale al percorso geometrico, specificando come il robot debba muoversi lungo il percorso. Il sistema parametrizza lo spostamento tramite una legge oraria normalizzata $s(t) \in [0,1]$, che mappa l'istante t nella posizione lungo la traiettoria, con tempo finale t_f.

l calcolo della durata t_f è vincolato dai limiti fisici dei motori, specificatamente la velocità massima $\dot{q}_{max}$ e l'accelerazione massima. $\ddot{q}_{max}$.

Il sistema permette l'adozione di molteplici profili di moto [9]-[10], ciascuno con proprietà derivabili distinte:

Profilo Trapezoidale della velocità:

Utilizzato come default per l'elevata reattività. Presenta tuttavia un'accelerazione discontinua (jerk teoricamente infinito nei transitori), che può innescare vibrazioni meccaniche.

Profilo Cicloidale (S-Curve):

Definito dalla legge $s(t) = \frac{t}{t_f} - \frac{\sin\left(\frac{2\pi t}{t_f}\right)}{2\pi}$, garantisce continuità sulle accelerazioni (jerk finito), risultando in un moto estremamente fluido e meno stressante per la meccanica.

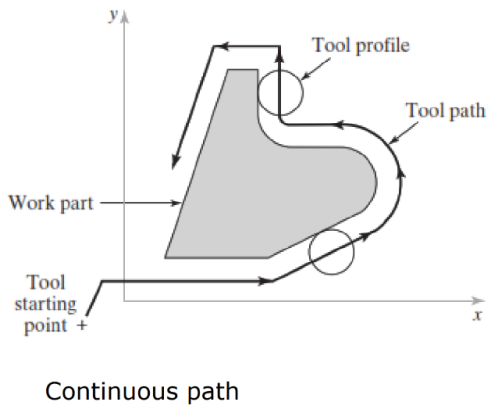
Profilo Polinomiale (Quintico):
Legge oraria del tipo:
$s(\tau) = a_0 + a_1 * \tau + a_2 * \tau^2 + a_3 * \tau^3 + a_4 * \tau^4 + a_5 * \tau^5$.
Il profilo quintico, in particolare, garantisce che velocità e accelerazioni siano nulle agli estremi del movimento, risultando ideale per l'interpolazione di precisione punto-punto.
Analizziamo più approfonditamente l'interpolazione punto per punto:

2.4.3 CONTINOUS PATH E POINT TO POINT:

A valle della generazione delle leggi orarie, il sistema di controllo del DESKARM gestisce l'esecuzione della traiettoria valutando lo stato dell'organo terminale tramite un flag logico dedicato (estratto dal pacchetto dati come *int(data['q'][2][i])* e identificabile come pen_up). Sebbene a livello concettuale la robotica distingua due modalità di spostamento, l'architettura software implementata nel progetto unifica la gestione spaziale adottando un'unica pipeline di calcolo per entrambe le fasi operative.
Movimenti Continuous Path (CP) in fase operativa
Quando l'organo terminale è a contatto con la superficie di lavoro (pen_up = 0), il sistema opera rigorosamente in modalità a percorso continuo (CP). Il manipolatore deve tracciare il percorso geometrico con precisione assoluta. Trattandosi di un plotter diretto, in cui la punta della penna è assimilabile a un punto adimensionale, la traiettoria del centro dell'utensile $P(t)$deve coincidere in ogni istante con la curva parametrica ideale del pezzo $C(t)$:

$$P(t) \equiv C(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}$$

Per garantire questa rigorosa equivalenza spaziale, il sistema processa l'intera pipeline algoritmica (tramite le funzioni di slice_trj() e get_profile_law()), applicando la cinematica inversa punto per punto a frequenza fissa. Questo vincola l'end-effector a percorrere esattamente la polilinea calcolata, rispettando fedelmente il profilo di velocità assegnato.



2.2 Traiettoria a percorso continuo

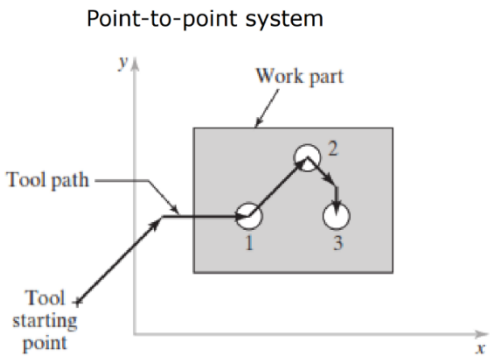
Movimenti Point-to-Point (PTP) a vuoto Quando il manipolatore deve riposizionarsi tra un tratto grafico e il successivo, la penna viene sollevata (pen_up = 1). In sede teorica, per i movimenti non operativi (Point-to-Point), l'obiettivo algoritmico sarebbe la pura traslazione dal punto di partenza $P_1(x_1, y_1)$ al punto di arrivo $P_2(x_2, y_2)$ nel minor tempo possibile. Calcolata la distanza euclidea nello spazio operativo:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Il tempo di percorrenza minimo ideale si otterrebbe muovendo i giunti in modo svincolato alla massima velocità di traslazione rapida v_{max} consentita dalla meccanica:

$$t = \frac{d}{v_{max}}$$

Tuttavia, nell'architettura del DESKARM la distinzione tra CP e PTP è puramente semantica. Non esiste una modalità di posizionamento in "rapido" (analoga al comando G00 dei sistemi industriali) che bypassa la pianificazione temporale della traiettoria. I movimenti a vuoto vengono elaborati dalla medesima pipeline del CP: l'algoritmo interpola il segmento lineare di riposizionamento applicando le stesse costanti dinamiche di limite (accelerazione e velocità massime) utilizzate durante la fase di disegno.
Sebbene questa scelta architetturale rinunci all'ottimizzazione estrema del tempo di ciclo globale, essa garantisce la stabilità deterministica del software, assicura una fluidità di moto superiore e previene l'insorgere di stress meccanici impulsivi sui giunti.



2.3 Traiettoria punto per punto

2.5 MODELLISTICA DINAMICA E ARCHITETTURA DI CONTROLLO:

Il modello dinamico completo in forma chiusa di Eulero-Lagrange [6] è definito dall'equazione $M(q) * \ddot{q} + C(q, \dot{q}) * \dot{q} + g(\dot{q}) = \tau$.
Operando il DESKARM su un piano orizzontale, il vettore di gravità si annulla $g(q) \approx 0$. La matrice d'inerzia M(q) per il manipolatore 2R risulta fortemente non lineare:

$$M(q) = \begin{bmatrix} I_1 + I_2 + m_2 * L_1^2 + 2 * m_2 * L_1 * L_{c2} * \cos(q_1) & I_2 + m_2 * L_1 * L_{c2} * \cos(q_1) \\ I_2 + m_2 * L_1 * L_{c2} * \cos(q_1) & I_2 \end{bmatrix}$$

Nonostante la complessità del modello teorico, la specifica destinazione d'uso del

manipolatore (scrittura a velocità contenute ed elementi strutturali a massa ridotta) rende i termini di accoppiamento dinamico e di forza centrifuga, Coriolis $C(q, \dot{q}) * \dot{q}$,trascurabili rispetto alle forze inerziali prevalenti.
Per questo motivo, l'architettura di controllo adotta uno schema **decentralizzato a giunti indipendenti**. Il controllore hardware assegna a ciascun motore un regolatore PID autonomo, trattando le interazioni dinamiche tra i bracci come meri disturbi esterni da reiettare. Per ovviare alla mancanza di un controllo a coppia calcolata, il controllore impiega un segnale di **feedforward**, iniettando direttamente le velocità $\dot{q}_{ref}$ e le accelerazioni $\ddot{q}_{ref}$ calcolate durante la fase di trajectory planning. Questo ibrido garantisce un tracking dell'errore ad alte prestazioni senza il peso computazionale della linearizzazione esatta in tempo reale.

Capitolo 3

FIRMWARE UPDATE

Il coordinamento delle operazioni in tempo reale e la gestione della logica di controllo sono affidati a un microcontrollore della famiglia STM32 [19]. La configurazione hardware e lo sviluppo del firmware sono stati eseguiti tramite l'ecosistema ufficiale del produttore, utilizzando STM32CubeMX per l'inizializzazione delle periferiche ed STM32CubeIDE per la scrittura del codice.

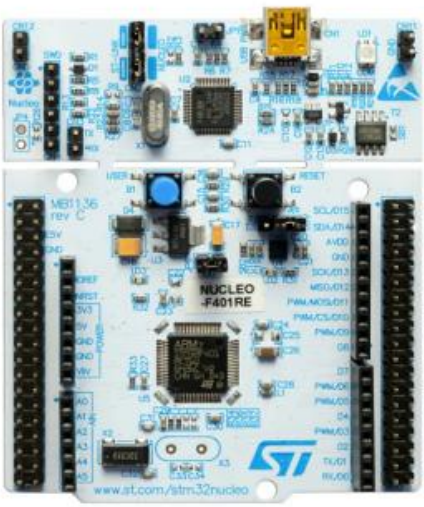
3.1 UNITA' DI CONTROLLO: MICROCONTROLLORE STM32

L'integrazione del sistema sfrutta in modo intensivo le periferiche interne del microcontrollore per ottimizzare le prestazioni:

• Gestione avanzata dei Timer:
Il sistema impiega un set di 5 timer dedicati a funzioni specifiche:
– TIM2 e TIM5: Utilizzati per la generazione dei segnali PWM necessari al pilotaggio dei driver dei motori.
– TIM3 e TIM4: Configurati in Encoder Mode per la lettura diretta dei segnali in quadratura provenienti dagli encoder magnetici.
– TIM10: Utilizzato per scandire il periodo di campionamento costante (Ts) richiesto dall'algoritmo di controllo.
– TIM11: Utilizzato per controllare il micro-servo

• Comunicazione e DMA: Lo scambio di dati tra il microcontrollore e l'interfaccia di alto livello avviene tramite la USART2 con baud rate impostato a 115200 bps. Per garantire la massima efficienza e minimizzare il carico della CPU, la ricezione dei setpoint e la trasmissione dei dati telemetrici sono gestite in modalità DMA (Direct Memory Access).

• Interfacce I/O: Sono stati configurati pin GPIO specifici per il controllo della direzione dei motori (DIR) e per la gestione degli interrupt esterni legati ai limit switch, garantendo l'arresto immediato del moto in caso di uscita dallo spazio di lavoro.



3.1 STM32F411RE Microcontroller

3.1 MODELLISTICA E SIMULAZIONE DINAMICA:

Come primo step per dimensionare e verificare la fattibilità del progetto con l'hardware a disposizione, la dinamica del manipolatore è stata modellata e simulata utilizzando Matlab e Simulink.

Il metodo scelto per il dimensionamento ha previsto l'utilizzo di una traiettoria generata tramite un polinomio del 7° grado per simulare la traiettoria più critica possibile e verificare se i motori scelti reggevano under worst-case. Il 7° grado è usato perché impone continuità su posizione, velocità, accelerazione e jerk ideale, per stimare i carichi massimi. Per definire questo polinomio sono stati imposti specifici vincoli geometrici e cinematici:

- Valori iniziali e finali prefissati per posizione, velocità e accelerazione.
- Accelerazione massima raggiunta al tempo $\frac{T_f}{4}$.
- Passaggio obbligato per il punto ($\frac{T_f}{2}$,0).

Per rendere parametriche le dimensioni fisiche all'interno del simulatore, la massa e le posizioni dei link sono state espresse approssimandoli a dei parallelepipedi con dimensioni generiche e densità costante.

La modellazione matematica ha richiesto diverse iterazioni. Nella prima implementazione, si è tentato di ricavare la dinamica tramite l'equazione Lagrangiana pura, senza l'ausilio di framework esterni. Tuttavia, questo approccio si è rivelato infruttuoso: la matrice di inerzia e, conseguentemente, la matrice di Coriolis non rappresentavano correttamente la dinamica del sistema reale, portandolo alla divergenza. Si è dunque proceduto a comparare queste matrici con quelle ottenute tramite il *Robotics Toolbox* di Peter Corke[7]-[8]. Avendo rilevato un'effettiva discrepanza nel modello dinamico (nonostante le matrici Jacobiane fossero corrette), si è deciso di utilizzare stabilmente il modello fornito dal Toolbox per condurre tutte le simulazioni successive.

3.2 EVOLUZIONE DELLA STRATEGIA DI CONTROLLO:

In fase di simulazione teorica, il sistema è stato validato scegliendo di implementare un controllo a dinamica inversa. L'obiettivo iniziale era calcolare la coppia necessaria τ direttamente a bordo del microcontrollore secondo l'equazione:

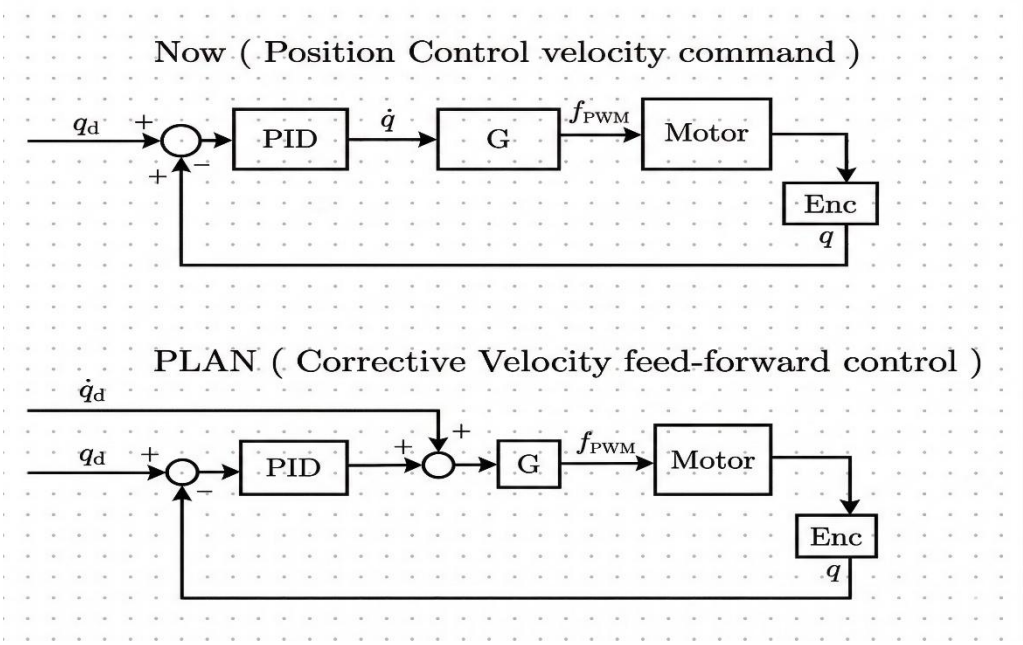
$$\tau = \mathrm{M(q)} * \ddot{q} + C(q, \dot{q}) * \dot{q} + g(q)$$

Tuttavia, durante l'importazione del firmware, il calcolo della dinamica inversa on-board (tecnica del *Computed Torque*) si è scontrato con i limiti hardware del microcontrollore (STM32F4). L'elevato numero di chiamate a funzioni trigonometriche saturava la capacità di calcolo dell'unità in virgola mobile (FPU) al target di 100Hz, causando oscillazioni al movimento.

Per risolvere questa criticità, la strategia è stata radicalmente rivista adottando un approccio di derivazione industriale:

1. **Spostamento del carico computazionale:** La conoscenza della complessa dinamica del sistema è stata delegata al generatore di traiettoria di alto livello (scritto in Python). Quest'ultimo calcola offline i riferimenti esatti di velocità e accelerazione, inviandoli pronti all'uso.

2. **Controllo PID + Feedforward:** Il firmware abbandona la dinamica inversa e implementa invece un loop PID indipendente per i giunti q_0 e q_1, sommandovi un termine di *Feedforward* cinematico. Questa soluzione ha ridotto il carico della CPU a meno dell'1%, garantendo un tracking fluido anche ad alte accelerazioni.



3.2 Implementazione controllo feed forward di velocità

3.3 ARCHITETTURA DEL FIRMWARE DA MONOLITE A MODULARE:

Il passaggio al nuovo schema di controllo ha richiesto una riscrittura completa del codice, superando la vecchia architettura monolitica a "Super-Loop" in cui oltre 1300 righe di codice (con variabili globali e rischio di *race conditions*) risiedevano in un singolo file.

Le principali migliorie algoritmiche e strutturali includono:

- **Design a Strati (Layered):** L'infrastruttura software è ora divisa in moduli applicativi, logici e di protocollo. Lo stato della macchina è incapsulato in strutture dati sicure, passate per riferimento (funzioni pure), incrementando drasticamente l'affidabilità e la testabilità del codice.

- **Gestione DMA e Determinismo:** La ricezione seriale asincrona, precedentemente gestita con interrupt bloccanti che rallentavano il loop di controllo, è stata affidata al controller DMA hardware (Circular DMA). Il *parsing* dei comandi avviene nel Main Loop, ottimizzando i tempi di ciclo. Al suo interno, il comando PEN_UP/DOWN viaggia sincrono ai pacchetti spaziali.
- **Stima della Velocità (Filtro vs. Regressione):** Nel vecchio firmware, la derivazione della velocità dall'encoder era gestita con un filtro passa-basso del primo ordine, che introduceva un inaccettabile ritardo di fase durante le curve veloci. Il nuovo algoritmo applica una regressione lineare (metodo dei minimi quadrati) su un buffer a finestra mobile. Questo restituisce un segnale pulito senza la penalità temporale (lag) dei filtri IIR tradizionali.

3.4 SICUREZZA ATTIVA:

L'aggiornamento architetturale ha coinvolto profondamente anche i protocolli di sicurezza hardware. Mentre in passato i finecorsa modificavano passivamente dei flag di stato per le routine di homing, il nuovo firmware dota il robot di un "Emergency Stop" attivo. L'attivazione di un *limit switch* ora genera un interrupt ad altissima priorità che impone istantaneamente le velocità di target a zero, operando un arresto critico (Hard Stop) che scavalca completamente il loop di controllo per preservare l'integrità strutturale della macchina e del piano di lavoro.

$$m = \frac{N\sum(xy) - \sum x \sum y}{N\sum(x^2) - (\sum x)^2}$$

3.3 Pendenza m della retta

3.5 CODICI PRINCIPALI PER L'AZIONAMENTO E IL CONTROLLO DEL MANIPOLATORE:

I seguenti codici si trovano tutti all'interno della sottocartella Core/Src e le relative definizioni si trovano in Core/Inc).

Ecco i file fondamentali e quale funzione svolgono per azionare il braccio:

1) **Core/Src/manipulator.c**:
È il cuore logico del robot. Contiene le funzioni ad alto livello per gestire le operazioni del braccio, tra cui:
- La procedura di homing e di calibrazione (homing(), calibration_start()).
- La processazione della coda di movimento per l'esecuzione fluida delle traiettorie (manipulator_process_motion_queue()).
- Il controllo della penna tramite PWM (control_pen()).
- Il controllo per verificare se il target è stato raggiunto.

2) **Core/Src/controller.c**:
Si occupa del controllo a basso livello dei motori. Implementa le logiche di controllo PID (Proporzionale-Integrale-Derivativo) per assicurarsi che i motori seguano esattamente la posizione richiesta dai setpoint. Oltre a un normale PID (manipulator_update_position_controller), il file contiene le funzioni manipulator_update_inverse_dynamics_controller, manipulator_calc_B e manipulator_calc_C. Questo significa che il robot non usa un semplice controllore lineare, ma implementa il controllo a Dinamica Inversa: calcola la gravità, l'inerzia e le forze centrifughe di Coriolis in tempo reale per compensare lo sforzo dei motori. Questo rende il movimento precisissimo anche ad alte velocità.

3) **Core/Src/protocol.c**: Gestisce la comunicazione seriale con il computer (la tua GUI in Python/JS). Riceve i comandi (es. coordinate dei punti da raggiungere, comandi di homing, stato della penna) e li impacchetta/spacchetta.

4) **Core/Src/encoder.c**: Si occupa di leggere i sensori encoder sui giunti. È fondamentale per il controllo in anello chiuso (closed-loop), in quanto permette al microcontrollore di sapere esattamente in quale posizione si trovano i bracci in tempo reale.

5) **Core/Src/main.c**: È il punto di ingresso (entry-point) del programma. Si occupa di inizializzare tutte le periferiche hardware del microcontrollore e contiene il ciclo infinito while(1) che richiama le funzioni di aggiornamento del controller, del manipolatore e della comunicazione.

6) **Core/Src/tim.c** e **Core/Src/gpio.c**: Sono i file di configurazione hardware.

tim.c nello specifico è fondamentale perché imposta i Timer del microcontrollore necessari per generare i segnali PWM che fanno muovere fisicamente i motori passo-passo (stepper) e il servomotore della penna (control_pen()).

Oltre ai file "core" della logica, l'architettura si appoggia su componenti di basso livello e file di configurazione essenziali:

1) **Core/Src/stm32f4xx_it.c:**
Questo file gestisce le Interrupt Service Routines (ISR). Quando un timer scatta (ad

esempio ogni millisecondo) o arriva un byte dalla porta seriale, l'esecuzione normale si interrompe e il processore si focalizza in questo file. Qui dentro quasi certamente si trova il richiamo della funzione che calcola il PID o che aggiorna la posizione dei motori (spesso agganciata all'interrupt del SysTick o di un Timer dedicato).

2) Core/Src/dma.c:

Configura il Direct Memory Access. Permette all'hardware di prendere i dati in arrivo dalla porta USB/Seriale e scriverli direttamente in memoria (RAM) senza disturbare il processore principale.

3) Core/Src/ring_buffer.c:

è un array circolare di 50 float usato come coda FIFO tra la ricezione DMA e il loop principale:

1. rbpush: inserisce un dato (se pieno, sovrascrive il più vecchio)
2. rbpop : legge e rimuove il dato più vecchio
3. rbpeek : legge il più vecchio senza rimuoverlo
4. rblast : legge il più recente senza rimuoverlo
5. rbclear : azzera tutto

La DMA riceve dati via UART in modo asincrono e li mette nel buffer; il while(1) nel main li legge con calma, senza perderne nessuno.

4) Core/Inc/manipulator.h: Contiene la definizione della struct manipulator_t, ovvero l'oggetto che contiene tutto lo stato del robot in ogni istante: gli angoli q0 e q1 attuali, le velocità, se la penna è alzata o abbassata, i parametri PID, e lo stato di calibrazione.
5) Core/Inc/protocol.h: Definisce l'elenco dei codici in byte che il PC invia per comandare il robot.

Il file più in rilievo tra tutti è **arm_project.ioc**:

Questo non è un file di codice, ma il file di configurazione visuale di STMicroelectronics. Se in futuro si vorranno aggiungere cambiare un pin del motore, impostare nuovi moduli basta solo aprire file con il software STM32CubeIDE/CubeMX. Una volta modificato graficamente il pin, il programma sovrascriverà, visualizzando Generate Code, generando i settaggi automaticamente nei file come gpio.c, tim.c, usart.c e altri, mantenendo intatta la tua logica in main.c.

3.6 IMPLEMENTAZIONE DELLE MELODIE:

Nel progetto planar-arm-refactoring è presente una piccola componente per la generazione di suoni direttamente dal microcontrollore STM32F411RE.

Le melodie non modificano lo stato di manipulator_t. Vengono invocate subito dopo operazioni critiche per fornire un feedback auditivo all'operatore.

Il protocollo seriale (protocol.c) definisce un comando (CMD_PLAY_MELODY) che include l'ID della melodia; la GUI lo invia quando l'utente preme il pulsante corrispondente.

Le funzioni sonore sono definite in **Core/Src/manipulator.c**

e dichiarate in **Core/Inc/manipulator.h.**

Le funzioni principali sono:

Robot_Sing(uint16_t freq, uint16_t duration_ms):

Genera un singolo tono a frequenza freq (Hz) per qualche millisecondo. Il tono è prodotto configurando il timer in modalità PWM, calcolando i valori di ARR e CCR per ottenere la frequenza desiderata, avviando il PWM, attendendo la durata con HAL_Delay e poi fermando il PWM.

Robot_Sound_Boot(void):

Suono di avvio del firmware. Esegue una breve sequenza di toni (E, G#, B, D) richiamando la funzione:

Robot_Sing, per un feedback sonoro.

Robot_Sound_Success(void):

Segnale di successo al completamento di una traiettoria e procedura di homing o calibrazione. Consiste in una melodia a tre toni ascendenti.

Robot_Sound_Error(void)

Segnale di errore. Produce un tono basso e prolungato o una serie di beep rapidi per indicare una condizione critica.

Robot_Sound_Thinking(void)

Indicatore di elaborazione (ad esempio durante il calcolo della traiettoria). Alterna due toni di frequenza media per dare l'idea di "working".

Robot_Sound_PowerDown(void)

Suono di spegnimento, una sequenza discendente di toni (E, G#, B, D) invertita rispetto al suono di avvio. Viene tipicamente chiamata prima di entrare in modalità low-power.

Robot_Sound_HighFreq(void)

Suono ad alta frequenza, usato per avvisi di limite di velocità o situazioni di emergenza.

manipulator_play_melody(manipulator_t * manipulator, uint8_t melody_id):

Funzione dispatcher che, in base al valore di melody_id, richiama una delle funzioni sopra elencate. Il valore melody_id è inviato dalla GUI tramite il protocollo seriale, permettendo all'interfaccia di far riprodurre al firmware la melodia desiderata.

Attraverso queste definizioni, il manipolatore suona quando si seleziona la modalità desiderata tra TEXT, DRAWING e IMAGE.

Capitolo 4
ARCHITETTURA DELL'INTERFACCIA GRAFICA DI CONTROLLO:

Nel seguente paragrafo sono illustrati i codici che governano l'interfaccia di controllo del manipolatore DESKARM. Il progetto è strutturato secondo il modello Model-View-Controller [21] con una netta separazione tra frontend e backend:

una che gestisce l'interfaccia con l'utente (frontend) e una che controlla il robot e la comunicazione con l'hardware (backend).

Prima di spiegare l'implementazione di questi moduli vengono illustrati gli algoritmi fondamentali che costituiscono DESKARM nelle sue funzionalità di base:

4.1 ALGORITMI FONDAMENTALI PER IL FUNZIONAMENTO DI DESKARM:

Il manipolatore DESKARM è governato da una gerarchia di algoritmi che trasformano l'interazione bidimensionale sul web in movimenti meccanici deterministici.

-Ottimizzazione e Controllo del Flusso Dati

L'efficienza del robot dipende dalla capacità di ridurre la ridondanza dei dati e minimizzare i tempi morti.
FLSC + Nearest Neighbor (trajectory_optimizer.js): Implementano una strategia di ricerca operativa che separa i tratti lineari dai curvi (First Linear, Second Curved) e seleziona il punto di partenza più vicino alla posa attuale per abbattere i tempi di pen-up.
Algoritmo di Douglas-Peucker [12] (utils_drawing.js): Riduce la densità dei vertici eliminando i punti la cui distanza perpendicolare da un segmento ideale è inferiore a una soglia di tolleranza, garantendo una trasmissione seriale fluida senza perdita di dettaglio visivo.
Payload Assembly (main.js): Converte le strutture complesse (oggetti Point, archi) in un array piatto [x, y, true/false] iniettando automaticamente i salti di penna necessari tra

forme non connesse.
Debounce e Resizing (main.js, canvas.js): Ottimizza l'I/O ritardando le chiamate API costose durante l'input e garantisce la coerenza spaziale tramite un rapporto di scala durante il ridimensionamento della finestra.

-Cinematica e Geometria Analitica

$$\boldsymbol{ratio} = \frac{\boldsymbol{W_{new}}}{\boldsymbol{W_{old}}}$$

La traduzione del segno grafico in coordinate fisiche richiede un solido apparato di algebra lineare e trigonometria.

Cinematica Diretta — DK(q) (manipulator.js): Calcola la posizione dell'end-effector partendo dagli angoli dei giunti:

$$p_1 = [L_1 * \cos(\theta_1), L_2 * \sin(\theta_2)]$$

$$p_2 = [p_{1x} + L_2 * \cos(\theta_1 + \theta_2), p_{1y} + L_2 * \sin(\theta_1 + \theta_2)]$$

Matrice di Rotazione 2D (utils_drawing.js): Permette la rotazione arbitraria dei vertici geometrici:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Conversione da relativo ad assoluto: Mantiene la coerenza tra pixel e metri reali tramite il fattore m_p = 0.000777 m/px, gestendo l'inversione dell'asse Y tra canvas e spazio fisico.

Logica XNOR per Archi (trajectory.js): Determina la direzione del rendering (CW/CCW) per prevenire il wrap-around angolare tramite il confronto logico degli stati.

Modellazione Matematica e Segnali Discreti:
Il robot interpreta il disegno non come pixel, ma come equazioni e griglie di quantizzazione.

Curve Parametriche (utils_drawing.js): Generazione di forme pure come il cuore (x = 16 sin^3(t)) o la spirale di Archimede (r(t) = a*t).

Geometria degli Archi (utils.js, canvas.js): La funzione find_circ ricostruisce raggio e centro da 3 punti, mentre la "Eraser Hit Detection" calcola la distanza punto-retta per

identificare le collisioni tra la gomma e i segmenti:

Quantizzazione e Snap (utils_drawing.js, canvas.js): Implementa lo Snap to Grid per forzare le coordinate sui nodi della griglia e la Proximity Search per facilitare la chiusura delle forme agganciando il cursore al punto più vicino.

Architettura Software e Design Patterns:

-**Memento Pattern [21]** (state.js): Gestisce lo stack di Undo/Redo tramite snapshot profondi (JSON.parse(JSON.stringify(obj))), garantendo l'immutabilità dello storico.

Observer/Subscriber (state.js): Permette una comunicazione reattiva tra moduli, aggiornando l'interfaccia ad ogni modifica dello stato senza creare dipendenze circolari.

-**Game Loop** (canvas.js): Utilizza requestAnimationFrame per un rendering a 60Hz efficiente, sincronizzato con il refresh del monitor e capace di sospendersi per risparmiare risorse.

-Science Color: Anche l'interfaccia si adatta dinamicamente per massimizzare l'usabilità: **Luminanza Percettiva HSP** (theme.js): Calcola la luminosità percepita [23] per determinare il contrasto del testo (bianco/nero) su sfondi arbitrari:

$$L = \sqrt{0.299 \times R^2 + 0.587 \times G^2 + 0.114 \times B^2}$$

Rimappatura HEX (theme.js): Manipola aritmeticamente i canali RGB per derivare colori secondari e stati dei pulsanti (hover/active) partendo da un singolo colore base.

4.2 FRONTEND DELLA HMI:

Il frontend, realizzato interamente in JavaScript, è responsabile della presentazione grafica e dell'interazione. Le pagine HTML e il foglio di stile CSS definiscono la struttura e l'aspetto dell'interfaccia, mentre gli script JavaScript mantengono lo stato dell'applicazione, gestiscono gli eventi dell'utente e aggiornano la visualizzazione della traiettoria sul canvas. Il modello dei dati è conservato in un oggetto centrale, e le modifiche a questo modello vengono propagate alla vista, che ridisegna il canvas con le nuove linee o curve. Tutte le operazioni che richiedono l'invio di informazioni al robot, come l'invio della traiettoria o la ricezione di messaggi di stato, avvengono tramite il bridge Eel, che espone le funzioni Python al codice JavaScript.

Modulo api.js:

Per garantire una separazione netta tra l'interfaccia utente (in ambiente Web) e il motore di calcolo (Python), è stato progettato un modulo di astrazione denominato api.js. Questa componente non è un semplice elenco di funzioni, ma agisce come il sistema nervoso centrale del software, orchestrando il passaggio di dati tra l'alto livello del frontend e il controllo hardware del backend tramite la libreria Eel.

Il Wrapper come Garanzia di Robustezza

La scelta di incapsulare tutte le chiamate in un oggetto API nasce dall'esigenza di rendere l'applicazione resiliente. Un problema tipico delle interfacce web-based per la robotica è il crash del software se il server si scollega.

L'infrastruttura implementata risolve il problema alla radice: ogni funzione è protetta da una clausola di guardia che verifica l'esistenza dell'oggetto window.eel. In caso di assenza del backend, il sistema non interrompe l'esecuzione ma restituisce valori di sicurezza (es. [0, 0, false] per la posizione), seguendo il principio della degradazione graduale, principio di progettazione che garantisce a un sistema software, hardware o a un sito web di continuare a funzionare con funzionalità ridotte, evitando il blocco completo.

Flussi di Controllo e Astrazione dei Comandi

Il modulo gestisce anche la complessità delle operazioni asincrone tipiche del protocollo RPC (Remote Procedure Call). Attraverso l'uso delle funzioni async/await, il frontend può interagire con il robot come se stesse operando localmente, ignorando la latenza della comunicazione seriale sottostante.

Gestione Traiettorie e File CAD: Funzioni come importDXF e generateText trasformano l'input grezzo dell'utente in dati pronti per il processing matematico.

Qui l'API funge da ponte per trasportare payload complessi (stringhe Base64) verso le librerie di analisi vettoriale del backend.

La codifica Base64 è un metodo che rappresenta i dati binari utilizzando 64 caratteri stampabili.

Sicurezza Operativa: Il comando stopTrajectory() è ad alta priorità ed è progettato per essere l'istruzione più rapida e diretta, garantendo che il segnale di stop arrivi al firmware nel minor tempo possibile.

L'Ottimizzazione del Trasferimento Dati (Pull-System)

Una particolarità ingegneristica del modulo risiede nella funzione *sendData()*. Invece di inviare array di coordinate, che potrebbe saturare il buffer, l'API utilizza un sistema a chiamata inversa (Side-Channel). Il JavaScript notifica al backend che i dati sono pronti; è poi il processo Python che estrapola i dati delle coordinate tramite la funzione js_get_data(). Questo garantisce che il trasferimento avvenga solo quando il backend è effettivamente pronto a processare i punti, eliminando colli di bottiglia.

Telemetria Real-Time e Feedback Visivo

Per permettere all'utente di monitorare il robot, l'API espone dei callback di sistema che Python può invocare spontaneamente.

Sincronizzazione della Posa (q): Tramite *js_draw_pose*, le variabili di giunto vengono inviate al frontend per aggiornare in tempo reale la rappresentazione grafica del manipolatore.

Tracciamento del Percorso: La funzione *js_draw_traces* permette di visualizzare a schermo l'anteprima delle traiettorie, fornendo un feedback visivo immediato prima che avvenga il movimento fisico.

Logging Centralizzato: Grazie a *js_log*, ogni evento del backend viene riportato nel terminale dell'interfaccia, creando un registro diagnostico unificato essenziale per il debug.

Modulo canvas.js:

Il motore grafico bidimensionale dell'interfaccia risiede in canvas,js.

Progettato per la gestione in tempo reale dell'ambiente di lavoro del robot. La classe principale, *CanvasHandler*, agisce come un sistema di coordinate interattivo che traduce l'input dell'utente in traiettorie matematicamente coerenti con lo spazio fisico del manipolatore SCARA.

Pipeline di Rendering e Ciclo di Vita

L'architettura del motore grafico si basa su un ciclo di aggiornamento continuo (Rendering Loop) implementato tramite il metodo animate(). Sfruttando l'API nativa *requestAnimationFrame*, il sistema garantisce un refresh rate sincronizzato con la frequenza di aggiornamento del monitor (circa 60Hz), minimizzando il jitter visuale e la latenza. Ad ogni iterazione, la pipeline esegue i seguenti step sequenziali:

1) Clear & Reset: Azzeramento del buffer grafico per prevenire artefatti di trascinamento.

2) Context Scaling: Applicazione delle matrici di trasformazione per conservare la coerenza tra pixel e metri.

3) Environment Rendering: Disegno del background, della griglia metrica e dei vincoli cinematici (aree di irraggiungibilità).

4) State Projection: Rendering delle traiettorie salvate nello stato globale e delle anteprime dinamiche.

5) Robot Overlay: Iniezione del contesto grafico nel modulo manipulator.js per il disegno della sagoma del robot in configurazione istantanea.

Trasformazioni di Coordinate e Mapping Spaziale

Una delle sfide ingegneristiche del modulo è la gestione del Viewport. Attraverso i metodi *resize()* e *updateCoordinates()*, il software implementa una trasformazione lineare tra lo spazio nello schermo (espresso in pixel) e lo spazio giunto/mondo (espresso in metri).

Punto di Origine Dinamico: Il sistema proietta l'origine (0,0) del robot in una posizione baricentrica rispetto alla larghezza del canvas e sul margine inferiore, simulando il posizionamento fisico della base sul banco di lavoro.

Costante di Scala $\mathbf{m_p}$: Viene calcolato dinamicamente il fattore di conversione metri-per-pixel, garantendo che le dimensioni del disegno a schermo corrispondano esattamente alle capacità di estensione meccanica dei bracci $L_1 e\ L_2$

Interazione Deterministica e Strumenti di Precisione

La gestione degli eventi di input del mouse è mediata da una macchina a stati che definisce il comportamento degli strumenti di disegno (Tools).

Algoritmo di Snapping: Per ovviare all'imprecisione dell'input manuale, il metodo *findNearbyPoint()* implementa una funzione di prossimità basata su una soglia euclidea. Se il cursore si trova entro un raggio critico da un punto della griglia o di una traiettoria esistente, le coordinate vengono bloccate automaticamente, garantendo la chiusura perfetta dei percorsi geometrici.

Selezione e Cancellazione:

Lo strumento "Gomma" non opera su base pixel, ma su base geometrica. Tramite la funzione *pointToLineDistance*(), il sistema calcola la distanza ortogonale tra il punto del click e i segmenti vettoriali memorizzati, procedendo alla rimozione selettiva dell'oggetto primitivo individuato.

Visualizzazione dei Vincoli e Safety Feedback

Un elemento distintivo del *CanvasHandler* è la rappresentazione visiva dei limiti operativi. Il metodo drawBackground() renderizza in modo semitrasparente l'area di lavoro (Workspace), evidenziando:

-Reachability Limit: Il cerchio massimo di estensione dei due link.

-Dead Zones: Zone di singolarità o limiti meccanici in prossimità della base dove il robot non può operare per evitare collisioni interne.

Questa scelta progettuale fornisce all'operatore un feedback immediato sulla fattibilità della traiettoria prima ancora di inviare i comandi al backend, agendo come primo livello di prevenzione contro errori di programmazione del movimento.

Anteprime Predittive:

Per migliorare la User Experience e la precisione del disegno, il sistema implementa diverse funzioni di Preview (DXF, testo, forme). Queste funzioni proiettano una versione semitrasparente dell'entità geometrica prima della sua scrittura definitiva in memoria. Questo permette di valutare l'ingombro spaziale e l'orientamento rispetto agli assi cartesiani in modo dinamico e interattivo.

Modulo main.js:

Il modulo main.js costituisce il vertice della gerarchia del software lato client, agendo come coordinatore centrale. Il suo compito è supervisionare il ciclo di vita dell'applicazione, gestire il flusso di dati tra i moduli specialistici e garantire che l'interazione uomo-macchina avvenga entro i limiti di sicurezza cinematica stabiliti per il manipolatore SCARA.

Bootstrapping e Iniezione delle Dipendenze
All'avvio del sistema, l'evento *window.load*, il modulo esegue la procedura di bootstrapping. Attraverso la funzione *initUI()*, viene effettuata una scansione del DOM (Document Object Model) per mappare gli elementi critici dell'interfaccia in un registro centralizzato. Contestualmente, avviene l'inizializzazione dei listener di sistema tramite *API.initCallbacks()*. Questo passaggio è fondamentale per il determinismo dell'applicazione: stabilisce infatti come il frontend deve reagire ai segnali asincroni provenienti dal backend Python, come la telemetria della posa o i log di sistema,

garantendo che JavaScript sia sempre sincronizzato con lo stato fisico del robot.

Architettura Event-Driven e Gestione dello Stato
Codesto modulo, inoltre, implementa un'architettura Event-Driven attraverso la funzione *setupEventListeners().*
Questa sezione del codice funge da snodo decisionale:

Gestione della Persistenza Locale: Intercetta i comandi di Undo/Redo, anche via scorciatoie da tastiera, interfacciandosi con il modulo state.js per ripristinare i fotogrammi delle traiettorie.
Reattività ai Parametri Fisici: Ogni modifica ai parametri geometrici innesca nuovamente il calcolo dell'area di lavoro nel *CanvasHandler*, forzando la coerenza visiva tra i limiti impostati e lo spazio di disegno.

Macchina a Stati Finita per il Contesto Operativo
L'applicazione non è un ambiente statico, ma muta dinamicamente attraverso la funzione *setAppMode(mode)*, che definisce il contesto operativo dell'utente: DRAWING MODE, TEXT MODE, IMAGE MODE.
Il passaggio tra queste modalità comporta una riconfigurazione dinamica del layout, tramite manipolazione delle classi CSS, isolando le funzionalità non necessarie e riducendo il carico cognitivo per l'operatore.

Validazione Cinematica Preventiva (Safety-First):
Una delle caratteristiche più avanzate del modulo è il sistema di Validazione Preventiva. Nel contesto della generazione di testo o traiettorie complesse, la funzione *validateGeneratedPatches()* esegue una simulazione virtuale prima dell'invio dei dati.
Se una parte del percorso calcolato eccede lo spazio di lavoro fisico il sistema innesca un protocollo di sicurezza mediante un Feedback Visivo. **Blocco Logico:** L'implementazione dell'inibizione del comando "Invia", impedisce l'esecuzione di traiettorie che causerebbero errori di singolarità o sforzi meccanici imprevisti.

Heartbeat Monitor e Pacchettizzazione:
Per garantire l'affidabilità della comunicazione seriale, main.js implementa un polling ricorrente (updateSerialStatus), il sistema interroga costantemente il backend per verificare lo stato della connessione hardware, traducendo la risposta in indicatori di stato visivi (LED virtuali).
Infine, la funzione *getTrajectoryPayload()* opera come Serializzatore di Dati. Essa aggrega i vettori geometrici, gli stati dell'end-effector (Pen Up/Down) e i parametri di velocità in un array strutturato. Questo pacchetto rappresenta il protocollo di comunicazione finale che, inviato al firmware, permette la traduzione dei pixel in movimenti millimetrici reali.

Modulo manipulator.js:

Il file manipulator.js implementa la logica di controllo e visualizzazione del braccio robotico nel dominio virtuale. Il modulo si occupa della Cinematica Diretta e della telemetria visiva, garantendo all'utente di visualizzare a schermo una proiezione fedele dello stato fisico della macchina.

Modellizzazione della Struttura e Stato Iniziale

All'interno del costruttore della classe Manipulator, il robot viene inizializzato come un sistema cinematico a due gradi di libertà.

Il software incapsula le costanti fisiche del manipolatore, come le lunghezze dei link L_1e L_2, e definisce lo stato iniziale delle variabili di giunto.

Un aspetto fondamentale è l'integrazione dello stato dell'end-effector: la variabile penUp/Down che, agisce come una coordinata Z booleana, impostata all'inizializzazione sempre sollevata. Questo permette al sistema di distinguere immediatamente una traslazione a vuoto e una fase di incisione attiva.

Implementazione della Cinematica Diretta (DK)

Il nucleo matematico del modulo risiede nella funzione dk(q). Partendo dagli angoli di giunto forniti dal firmware, il sistema applica le equazioni trigonometriche fondamentali per determinare la posizione dei nodi strutturali nel piano cartesiano:

Posizione del Gomito (Joint 1),
Posizione dell'End-Effector (Joint 2).

I risultati sono inizialmente espressi in metri e poi convertiti attraverso una funzione di trasformazione spaziale *abs2rel*.

Questo passaggio è cruciale per il **Mapping Spaziale**: converte le unità metriche in coordinate pixel relative al Viewport del browser, garantendo che il robot mantenga le proporzioni corrette indipendentemente dalla risoluzione dello schermo o dal livello di zoom impostato dall'utente.

Flusso Telemetrico e Sincronizzazione Real-Time

La comunicazione tra il backend Python e il modello virtuale avviene attraverso i metodi *Setter* e *Getter* della variabile q.

Quando il pacchetto dati seriale arriva dal microcontrollore, viene iniettato nel sistema tramite set q(q).

Il modulo è progettato per gestire pacchetti di dati tri-dimensionali: oltre ai due angoli motori, il sistema intercetta il bit di stato della penna (Z=0 o Z=1).

Questa architettura permette una **sincronizzazione deterministica**: ad ogni aggiornamento della posizione dei motori, la simulazione digitale ricalcola istantaneamente l'intera catena cinematica, muovendo i bracci virtuali a schermo con una latenza impercettibile per l'utente.

Rendering Grafico e Feedback Cromatico Stato-Dipendente
La funzione *draw_pose* si occupa della visualizzazione fisica sul Canvas. I link sono rappresentati come vettori grafici definiti da uno stile "Industrial Orange" (#ff8800), che conferisce robustezza visiva alla struttura simulata.
Per massimizzare la consapevolezza situazionale dell'operatore, è stato implementato un sistema di Feedback Cromatico Attivo per l'estremità finale: 1) Ciano Luminoso (#00e5ff): Indica che la penna è sollevata e il robot sta effettuando uno spostamento verso il punto desiderato.
2)Rosso Fuoco (#ff4444): Segnala che la penna è in contatto con la superficie e il sistema sta attuando una traiettoria di disegno.

Visual Debugging tramite Analisi delle Tracce
Oltre alla posizione istantanea, manipulator.js integra un **sistema di gestione delle scie,** Trace Management. Attraverso le funzioni *add2trace* e *draw_traces*, il software registra lo storico delle posizioni occupate dai giunti durante il movimento.
A schermo, queste vengono renderizzate come code cometarie semitrasparenti in blu e verde neon. Questa funzionalità non ha solo scopo estetico, ma rappresenta un potente strumento di Diagnostica Visuale: permette all'utente di osservare la fluidità dei percorsi e di individuare immediatamente eventuali anomalie nelle formule di interpolazione o picchi di rumore nei dati degli encoder, osservando se la "scia" lasciata dai giunti segue curve armoniche o presenta discontinuità geometriche.

Modulo state.js:

Il file state.js definisce la classe *StateManager,* istanziata globalmente come appState, che funge da memoria centrale e supervisore dei dati del sistema. L'architettura si basa sul design pattern **Singleton**[21], è un modello generativo che garantisce la creazione di una sola istanza di una classe e fornisce un punto di accesso globale a tale istanza. È utile per gestire risorse condivise come configurazioni, log o connessioni a database. Si basa su un costruttore privato e un metodo statico. Garantendo un ottimo controllo e protezione.

Parametrizzazione Fisica e Costanti di Sistema
All'interno del costruttore del *StateManager*, vengono stabilite le limitazioni reali in cui opera il manipolatore. Il modulo non si limita a conservare dati, ma definisce la coerenza tra l'ambiente virtuale e quello reale:

Geometria del Robot: Vengono memorizzate le lunghezze dei link L_1 e L_2 parametri che il sistema utilizza per calcolare i limiti cinematici.

Fattore di Conversione m_p: In questo modulo risiede la costante di trasformazione

metri-per-pixel. Questo valore è il perno matematico che permette a tutti i sotto moduli (utils.js e canvas.js) di operare con la stessa scala metrica, garantendo che un centimetro disegnato a schermo corrisponda esattamente a un centimetro percorso dall'end-effector.

Il Pattern Observer: Sincronizzazione Reattiva
Dato che l'applicazione opera in un ambiente locale (browser) senza un database persistente in tempo reale, state.js implementa il *Pattern Observer*[21] attraverso i metodi *subscribe* e *notifyListeners*.
Il P*attern Observer* è un modello di progettazione comportamentale che definisce una relazione uno-a-molti tra oggetti. Quando un oggetto (Subject o Observable) cambia stato, tutti i suoi dipendenti (Observer) vengono notificati e aggiornati automaticamente. Questo meccanismo di sottoscrizione permette ai componenti dell'interfaccia di rimanere in ascolto dei cambiamenti dello stato. Quando l'utente compie un'azione, ad esempio disegnare una forma geometrica, *StateManager* invoca una notifica globale. Immediatamente, il motore grafico si ridisegna e i moduli di controllo aggiornano le proprie code di lavoro, eliminando la necessità di controlli manuali ridondanti e ottimizzando le prestazioni della CPU.

Gestione della Memoria Storica (Undo/Redo System)
Una delle funzionalità tecnicamente più avanzate del modulo è il sistema di **gestione della cronologia,** progettato per permettere la reversibilità delle operazioni Annulla e Ripeti.

L'algoritmo non memorizza le singole azioni, ma opera tramite fotografie di stato (Deep Snapshots):
Salvataggio (saveState): Ogni volta che una traiettoria viene confermata, il sistema esegue una clonazione profonda (in JSON) dell'intero oggetto trajectory, archiviata in uno stack di memoria.
Ripristino (undo/redo): Attraverso una gestione a doppia pila il sistema è in grado di sostituire brutalmente lo stato attuale con una versione precedente salvata. Questo approccio garantisce l'integrità dei dati, eliminando il rischio di corruzione dei puntatori.
Ciclo di Vita dei Dati e Sicurezza Operativa
Il modulo gestisce anche la transizione dei dati verso la fase esecutiva. La funzione *moveToSent* opera in questa maniera: quando l'utente invia una traiettoria al manipolatore, i dati vengono spostati in un'area di sola lettura.
Questa procedura blocca la possibilità di modifiche accidentali mentre il robot è in movimento, prevenendo potenziali collisioni o errori di calcolo dovuti alla sovrapposizione di nuovi comandi su traiettorie già in fase di esecuzione.

Infine, i **metodi di Epurazione** permettono un ripristino deterministico del sistema, azzerando i contatori hardware e i buffer grafici per riportare il manipolatore in una condizione sicura.

Modulo theme.js:

Il modulo theme.js non svolge un ruolo puramente estetico, ma agisce come un motore di rendering stilistico dinamico che garantisce la leggibilità dei dati tecnici e l'accessibilità dell'interfaccia in diverse condizioni di illuminazione ambientale.
Gestione Dinamica dell'Interfaccia:
Il file in questione implementa la classe *ThemeManager*, un modulo incaricato di gestire la palette cromatica dell'intera applicazione. Attraverso la **manipolazione programmatica** delle CSS Custom Properties, variabili CSS, il sistema è in grado di ricalcolare l'intero schema visivo in tempo reale, assicurando che il contrasto tra gli elementi grafici del Canvas e i controlli dell'interfaccia rimanga ottimale.
Il sistema si basa su un dizionario di configurazioni predefinite: bianco, nero, rosso, blu, viola, verde e giallo. L'approccio per la creazione dell'interfaccia di personalizzazione è Data-Driven, in cui la logica decisionale e il comportamento del programma dipendono dai dati di input o dalla configurazione esterna, piuttosto che da regole hard-coded.
La funzione *createButtons()* scansiona il dizionario dei colori e genera dinamicamente gli elementi interattivi nella barra laterale "Customization". Questo approccio modulare permette di aggiungere nuovi schemi cromatici semplicemente aggiornando l'oggetto dei preset, senza dover modificare manualmente il codice HTML.

Logica Adattiva e Calcolo della Luminanza Percettiva
Il cuore funzionale del modulo risiede nella funzione *updateTheme()*, che implementa un algoritmo di adattamento cromatico intelligente basato su tre pilastri:

1) Gerarchia Visiva e Profondità: Tramite la funzione *adjustColor()*, il sistema calcola i colori per i componenti secondari e terziari. Utilizzando trasformazioni matematiche sulle componenti RGB, il software schiarisce o scurisce il colore base per creare un distacco visivo per facilitare l'identificazione dei controlli.

2) Contrasto Intelligente (Modello HSP): Per garantire l'accessibilità, viene utilizzata la funzione *isLightColor()*. L'equazione applicata si fonda sulla luminanza percettiva (modello HSP), che pesa i canali RGB in base a come l'occhio umano percepisce la luminosità. Se il colore di sfondo supera una determinata soglia critica, il sistema forza il testo al nero; in caso contrario, lo imposta al bianco.

3) Coerenza del Workspace: Il tema influenza direttamente gli elementi tecnici del Canvas. Le variabili per gli assi cartesiani e la griglia metrica vengono invertite automaticamente per garantire che la scala di riferimento rimanga sempre visibile sopra il colore scelto dall'utente.

Isolamento dei Componenti Critici
Un aspetto fondamentale del design industriale del software è la protezione dei campi di input numerici. Indipendentemente dal tema estetico scelto, i campi dedicati alle

coordinate e ai parametri del robot mantengono uno stile fisso, sfondo grigio e testo bianco, per facilitare l'esperienza utente.

Visualizzazione dei Limiti Meccanici

Il *ThemeManager* coordina anche la visualizzazione dell'area non raggiungibile. Attraverso la variabile *reach-circle-color*, il software si assicura che l'alone semicircolare che delimita lo spazio di lavoro sia sempre percepibile, agendo come un avviso visivo costante per l'operatore riguardo ai limiti fisici del manipolatore.

Modulo trajectory.js:

Il modulo **trajectory.js** definisce la classe *Trajectory*, un'infrastruttura di memorizzazione strutturata che funge da database vettoriale dell'applicazione. Il suo **scopo principale** è la *gestione del buffer data* e *un array di oggetti geometrici* che cataloga sequenzialmente ogni movimento del manipolatore, distinguendo tra spostamenti operativi e spostamenti logistici, a vuoto.

Struttura del Dato e Astrazione del Movimento

Gli elementi del buffer della traiettoria sono interpretati come una primitiva geometrica dotata di attributi spaziali e di stato. La classe astrae la complessità del movimento robotico in due tipologie principali di interpolazione:

Interpolazione Lineare *add_line:* Definisce un vettore spostamento tra due punti p_0 e p_1. Oltre alle coordinate cartesiane, viene memorizzato il parametro booleano della Z. Questa variabile è fondamentale per la cinematica del robot: determina se il comando debba essere interpretato, in linguaggio G-Code, come un movimento di "traslazione rapida" (G0) o come un movimento di "lavoro" (G1) [22].

Interpolazione Circolare *add_circle:* Gestisce la generazione di archi e perimetri curvi. La classe memorizza l'arco in formato polare: centro (c), raggio (r) e gli angoli di fase iniziale e finale (theta_0, theta_1), espressi in radianti. Questo formato è ottimale per il backend Python perché permette di calcolare le traiettorie circolari con precisione analitica.

Gestione dei Metadati e Raggruppamento (Grouping)

L'integrazione del parametro *groupId* permette di etichettare i diversi segmenti come appartenenti a un'unica entità logica. Questa scelta architettonica è il pilastro su cui il sistema di Undo/Redo si appoggia, garantendo di manipolare o rimuovere interi blocchi di traiettoria senza dover gestire singolarmente ogni minuscolo segmento vettoriale.

Pipeline di Rendering Visuale

Il metodo *draw(ctx)* opera come un interprete grafico del buffer dati. Integrandosi nel loop a 60 FPS del sistema, esegue una scansione lineare dell'array data applicando regole di filtraggio e rendering condizionale:

Filtraggio dei Movimenti a Vuoto: Per una questione puramente visiva, il sistema identifica i segmenti con raised impostata a valore true. Questi movimenti, pur essendo presenti nella struttura della traiettoria per istruire i motori, vengono saltati durante la fase di disegno a schermo. L'utente vede così solo l'output finale sul foglio virtuale, eliminando il disturbo visivo dei percorsi di riposizionamento.

Ricostruzione Geometrica: Il renderizzatore ricostruisce, per ogni primitiva di tipo *circle,* l'arco utilizzando il comando *ctx.arc().* Qui viene applicata una logica di correzione tramite operatori booleani (XOR) per assicurare che la direzione dell'arco visualizzato coincida con la direzione di rotazione prevista per i giunti del robot, garantendo la sincronizzazione tra simulazione e realtà fisica.

Determinismo e Reset dello Stato
Attraverso il metodo *reset(),* la classe garantisce un ritorno deterministico allo Stato Zero. L'eliminazione non reversibile dell'array-data assicura che non vi siano residui in memoria tra una sessione di lavoro e la successiva, mantenendo l'integrità del flusso di dati.

Modulo trajectory-optimizer.js:

Questa componente agisce come un ottimizzatore logistico e un pianificatore degli obiettivi. Il suo scopo è trasformare un insieme disordinato di tratti grafici in una sequenza di movimenti efficienti, riducendo i tempi di esecuzione e l'usura meccanica del manipolatore.
Ottimizzazione delle Traiettorie: Il Sistema FLSC e l'Heuristic Greedy
Il modulo TrajectoryOptimizer introduce l'algoritmo FLSC, una *logica di pre-processing dei dati, q*uesto approccio non si limita a ordinare i punti nello spazio, ma organizza la strategia di movimento del robot in base alla tipologia di interpolazione richiesta, minimizzando i "salti" a vuoto e ottimizzando il lavoro del controllore di basso livello.

Il metodo consiste nella classificazione preventiva delle primitive. Attraverso il metodo *classifyPatches(),* l'algoritmo scinde l'intero disegno in due sotto-insiemi distinti:

Segmenti Lineari: Tratti che richiedono una velocità di traslazione costante e un'interpolazione rettilinea.

Segmenti Curvi: Archi e cerchi che comportano una variazione continua delle velocità angolari dei giunti q_1e q_2.
La scelta di eseguire sistematicamente prima tutte le linee e successivamente tutte le curve nasce da un'esigenza di efficienza computazionale e di primi test di tracciamento.
Raggruppando le primitive per tipologia, si riduce il "context switching" logico nel firmware, permettendo al robot di mantenere regimi di accelerazione più fluidi e coerenti

per blocchi omogenei di movimento.

Algoritmo Greedy Nearest Neighbor per il Problema del Commesso Viaggiatore
Una volta smistati i tratti, il modulo deve risolvere un problema di ottimizzazione combinatoria: trovare il percorso più breve per toccare tutti i punti d'inizio dei segmenti. L'ottimizzatore implementa un algoritmo Greedy basato sulla logica del Nearest Neighbor, il vicino più prossimo.
Il metodo *optimizePath()* opera secondo i seguenti step matematici:

Punto di Partenza: Viene identificata la posizione attuale dell'end-effector.

Minimizzazione della Distanza Euclidea: Per ogni tratto rimanente nella lista, l'algoritmo calcola la distanza d utilizzando il teorema di Pitagora.

Selezione Ottima Locale: Viene scelto il segmento il cui punto d'inizio minimizza d.

Iterazione: Dopo aver "disegnato" il segmento, la nuova posizione finale diventa il nuovo punto d'inizio e il ciclo si ripete fino all'esaurimento della lista.
Questo processo trasforma una sequenza casuale di tratti in una catena continua, riducendo drasticamente il tempo totale in cui la penna è alzata.

Gestione Strategica delle Transizioni:
Il metodo *optimize()* agisce da assemblatore finale. Dopo aver ottimizzato separatamente il gruppo lineare e quello curvo, il software ricompone i due blocchi.
Invece di lasciare che il robot si muova in modo imprevedibile tra l'ultima linea e la prima curva, l'ottimizzatore genera deliberatamente un Ponte Aereo, un'istruzione di movimento con flag raised: true, che comanda uno spostamento rapido e sicuro tra i due domini geometrici. Questa transizione impercettibile garantisce che il flusso di coordinate inviato al backend Python sia una lista contigua e priva di salti logici, proteggendo l'integrità fisica del braccio durante il cambio di algoritmo.

Normalizzazione Universale e Data Sanitization
Poiché l'applicazione riceve dati da fonti eterogenee, coordinate pixel dal mouse, array vettoriali dal backend Python, il modulo implementa una fase di Normalizzazione *normalizePatches().*
Questa funzione agisce da **filtro** convertendo ogni input in un formato strutturato standard, traduce le coordinate in metri dall'origine del robot, rimuove i metadati superflui del frontend.
Il risultato è un payload chiaro e matematicamente rigoroso, pronto per essere pacchettizzato dalla funzione *getTrajectoryPayload()* del modulo main.js e inviando le informazioni ai motori.

Modulo utils.js:

Questo modulo è una libreria di utilità puramente matematica che garantisce la coerenza tra il dominio discreto, interfaccia, e il dominio continuo, in metri; è stato progettato principalmente per il mappaggio delle Coordinate.
Senza questo strato di astrazione, il sistema rischierebbe di confondere le coordinate dello schermo con le lunghezze fisiche dei bracci, portando a errori di posizionamento catastrofici.

La **classe Point** rappresenta la componente informatica più piccola dell'intero software, la quale adotta una struttura a doppia verifica per mantenere costantemente sincronizzati i due stati:

Stato Relativo: Coordinate espresse in pixel, utilizzate dal motore grafico per il rendering sul monitor.

Stato Assoluto: Coordinate espresse in metri, utilizzate dal modello cinematico e dal backend Python.

Grazie all'uso di *getter* e *setter*, ogni volta che un valore viene modificato in uno dei due domini, l'altro viene ricalcolato istantaneamente applicando il fattore di scala.
Questa classe è un modulo di calcolo vettoriale che espone metodi per la manipolazione geometrica delle traiettorie:

Operazioni Lineari: Implementazione di somme e sottrazioni per definire vettori direzione e distanze.

Magnitudo Euclidea: Calcolo della distanza tra due punti basato sul teorema di Pitagora.

Rotazione e Proiezione: Il metodo *rot(delta)* applica una matrice di rotazione per traslare un punto nello spazio attorno all'origine.

Analisi Angolare (angle): Restituisce l'orientamento del vettore utilizzando la funzione *Math.atan2(y, x),* fondamentale per determinare la fase iniziale e finale degli archi.

Le funzioni **rel2abs** e **abs2rel** agiscono come traduttori universali del sistema, gestendo le divergenze tra le convenzioni dei sistemi informatici e quelle fisiche:

Inversione dell'Asse Y: Nel sistema grafico, l'origine (0,0) è in alto a sinistra e la coordinata Y aumenta scendendo. In fisica e robotica, l'origine è alla base del manipolatore e la Y, aumenta verso l'alto. La gestione di questa inversione di segno è gestita in modo trasparente.

Offset dell'Origine: Il software trasla costantemente i punti rispetto al centro della base del robot, garantendo che ogni click dell'utente venga interpretato correttamente rispetto al perno del primo motore.

Geometria degli Archi: L' algoritmo *find-circ()* risiede la logica per la generazione di circonferenze partendo da tre punti campionati (inizio, corda, fine). Questa funzione è essenziale per gli strumenti di disegno avanzati.

Analisi Matriciale: Riceve tre coordinate ed esegue il calcolo dei mediatori dei segmenti per individuare il punto di intersezione comune.

Parametrizzazione: Restituisce il raggio **r**, le coordinate del centro **c** e gli angoli $\boldsymbol{\theta_0}$ e $\boldsymbol{\theta_1}$.

Integrazione: Questi dati vengono passati direttamente al modulo trajectory.js, permettendo di trasformare un input grafico "a tre click" in un comando di interpolazione circolare perfetto per il manipolatore.

Modulo utils_drawing.js

Il file utils_drawing.js esporta una libreria di funzioni specializzate nella generazione di traiettorie vettoriali. Il suo ruolo è fondamentale per la discretizzazione delle forme: questo modulo scompone questa identità in una sequenza finita di segmenti lineari o archi interpolati calcolati tramite trigonometria e geometria analitica.

Generazione Poligonale e Simmetria Radiale

Il modulo gestisce la creazione di poligoni regolari e irregolari attraverso un approccio parametrico. La funzione *calculatePolygon()*, ad esempio, utilizza un incremento angolare costante per definire i vertici su una circonferenza virtuale: $\Delta\theta = 2\pi/n$, dove n rappresenta il numero di lati. Questo approccio permette al sistema di generare dinamicamente qualsiasi forma, dall'esagono all'ottagono, garantendo che il robot mantenga una velocità tangenziale costante durante l'esecuzione del perimetro.

Geometrie Parametriche Complesse

Una delle sezioni più sofisticate del modulo riguarda le forme non lineari e le curve parametriche mediante un algoritmo definito **Star-Zig-Zag.** Per evitare che il robot incroci i tratti al centro la funzione *calculateStar* alterna dinamicamente tra un raggio

esterno e uno interno utilizzando l'operatore modulo. Il risultato è un perimetro continuo e ottimizzato per il trascinamento della penna.

Equazioni del Cuore: Per generare la forma a cuore, il sistema risolve in tempo reale un set di equazioni paramet0riche di alto ordine:

$$\boldsymbol{x = 16 * \sin^3(t)}$$

$$\boldsymbol{y = 13 * \cos(t) - 5 * \cos(2t) - 2 * \cos(3t) - \cos(4t)}$$

Questa funzione produce una densità di punti variabile che assicura una curvatura fluida.

Varianti Triangolari e Matrici di Rotazione

Il modulo distingue quattro tipologie di triangoli, applicando matrici di rotazione per renderli orientabili nello spazio. Per ogni vertice (x, y), il sistema calcola la nuova posizione (x', y') per un angolo θ:

$$\boldsymbol{x' = x * \cos(\delta) - y * \sin(\delta)}$$

$$\boldsymbol{y' = x * \sin(\delta) + y * \cos(\delta)}$$

Questo permette di generare triangoli equilateri, isosceli e rettangoli che l'utente può ruotare interattivamente sul canvas prima dell'invio al robot.

Ottimizzazione del Percorso: Algoritmo di Douglas-Peucker

Il contenuto più rilevante di questo file è la funzione *simplifyPath()*. Durante il disegno a mano libera", il mouse genera una sovrabbondanza di punti rumorosi (jitter) che saturerebbero il buffer del microcontrollore. Il sistema implementa l'Algoritmo di Douglas-Peucker, per operare una semplificazione topologica: il suo compito è quello di ridurre il numero di punti appartenenti a una linea spezzata, una polilinea, mantenendo quanto più intatto l'andamento originario.

-Analisi della Distanza Perpendicolare: L'algoritmo individua i punti che deviano meno di una determinata soglia (ε) dalla retta che unisce l'inizio e la fine del tratto.

-Sfoltimento Selettivo: I punti "inutili" vengono eliminati, riducendo drasticamente il peso del file, tra 80-90%, senza perdere la forma essenziale del disegno.

Questo processo garantisce che il braccio robotico riceva solo i vertici necessari, evitando micro-vibrazioni dovute a segmenti troppo brevi.

-Discretizzazione della Griglia (Snapping)

Infine, la funziona di *snapToGrid()* implementa la quantizzazione dello spazio. Arrotondando le coordinate dell'utente a multipli interi della griglia, il modulo assicura che le forme geometriche siano perfettamente allineate agli assi cartesiani, eliminando l'errore umano nel posizionamento dei punti di controllo.

4.3 ARCHITETTURA DI BACKEND E PROTOCOLLO DI COMUNICAZIONE SERIALE

Il coordinamento globale del manipolatore è strutturato su un'architettura software a due livelli che distingue nettamente i compiti tra l'interfaccia utente e il nucleo di calcolo, garantendo una separazione che riflette i moderni standard dell'ingegneria del software. Il livello basso agisce come una sorta di front-office dell'intero sistema, essendo lo strato che si occupa di inizializzare l'applicazione web e di gestire l'asincronia dell'ambiente browser.

Questo strato è incaricato di restare costantemente in ascolto delle chiamate Remote Procedure Call (RPC) provenienti dai moduli JavaScript che abbiamo analizzato in precedenza, comportandosi come un ponte di comunicazione che riceve gli input grafici o i comandi di controllo e li instrada verso il cuore pulsante del software.

In questo contesto, il livello basso non esegue operazioni matematiche onerose ma si limita a una funzione di delegazione strategica, passando letteralmente la palla ai componenti più profondi del sistema situati nella directory /lib.

Il secondo livello, ad alto profilo computazionale, rappresenta il vero motore decisionale del robot dove risiedono i moduli specializzati per la fisica, la matematica avanzata e la codifica dei protocolli di comunicazione. All'interno della cartella /lib il software è compartimentato in unità logiche indipendenti. Alcune si occupano esclusivamente di risolvere i complessi sistemi di equazioni legati alla cinematica del braccio, altre analizzano i file vettoriali come i DXF per estrapolare coordinate cartesiane pure, mentre altre ancora formattano queste informazioni in pacchetti binarizzati sicuri, pronti per essere trasmessi al firmware. Questa rigida gerarchia tra l'interfaccia pubblica che risponde agli input dell'utente e il nucleo di calcolo che calcola i dati, garantisce che a seguito di una modifica alla logica di movimento o al protocollo seriale questo non destabilizzi la GUI, permettendo al sistema di evolvere e scalare in modo modulare senza compromettere la fluidità dell'esperienza utente o la precisione millimetrica dei movimenti del manipolatore SCARA.

4.3.1 BACKEND DI BASSO LIVELLO

Modulo *check_ports:*

Adempie alla funzione di diagnostica, operando come un'unità di scansione indipendente, progettata per la validazione dell'integrità hardware prima dell'inizializzazione del core computazionale.
Lo script interroga il sistema operativo attraverso la libreria *serial.tools*, eseguendo una procedura di discovery passiva volta a censire ogni interfaccia di comunicazione seriale attiva e a estrapolare i metadati identificativi delle periferiche, quali il nome della porta e l'Hardware ID (HWID). Questo processo di filtraggio è essenziale per garantire che il software interagisca esclusivamente con il chip seriale del manipolatore, identificato tramite i codici univoci VID e PID, evitando collisioni con altre periferiche generiche.
Una volta isolato il target potenziale, il sistema avvia una fase di handshake attivo caratterizzata da un approccio deterministico: viene imposto un timeout di 0,5 secondi per massimizzare la velocità di scansione e viene attuata una manipolazione del segnale DTR (Data Terminal Ready) per forzare un reset hardware del microcontrollore, garantendo così che il firmware si trovi in uno stato di ascolto pulito. Il test consiste nella trasmissione di un pacchetto di comando binario per la richiesta di posizione, codificato secondo le specifiche del protocollo proprietario, seguito da una finestra di monitoraggio asincrono in cui il software analizza i buffer in ingresso alla ricerca dei valori: 0xA5 e 0x5A.
L'intercettazione di tali header conferma non solo la connessione fisica, ma anche la compatibilità logica del firmware. Se l'esito risulta positivo, lo script attiva una funzione di feedback sensoriale, inviando un comando di esecuzione per la melodia ID 1; questa operazione sfrutta le frequenze di commutazione dei driver dei motori passo-passo per generare un segnale acustico di conferma, permettendo all'operatore di verificare l'avvenuto accoppiamento senza la necessità di supervisione visiva sul terminale, per poi concludere la procedura con la chiusura sicura del canale di comunicazione per prevenire stati di occupazione della porta durante l'avvio della sessione principale.

Modulo config:

Il modulo config.py rappresenta l'entità di configurazione centralizzata e il registro dichiarativo dell'intero ecosistema software del manipolatore DESKARM, fungendo da nucleo di coordinamento che isola i parametri critici dalla logica algoritmica dei moduli esecutivi. A differenza dei componenti deputati al processing delle immagini o alla pianificazione delle traiettorie, questo file adotta una struttura puramente statica basata su costanti e dizionari, garantendo l'integrità del sistema e facilitando la calibrazione hardware senza la necessità di intervenire direttamente sul codice sorgente della

cinematica o dei driver di comunicazione.
All'interno del dizionario delle impostazioni globali, il parametro fondamentale è il tempo di campionamento T_c impostato a 0.01 s, valore che determina una frequenza di aggiornamento di 100 Hz e assicura il sincronismo deterministico tra il calcolo delle traiettorie lato PC e i timer hardware del microcontrollore. Tale sincronizzazione è supportata dalla definizione di vincoli cinematici quali *max_acc* e *max_speed*, i quali operano come limitatori di sicurezza per il profilo di velocità trapezoidale, riducendo drasticamente il fenomeno del Jerk e preservando l'integrità strutturale dei giunti durante le fasi di accelerazione e decelerazione.
Il modulo gestisce inoltre lo stato della comunicazione seriale tramite il flag *ser_started*, che permette di commutare dinamicamente il flusso dei dati tra l'invio fisico all'hardware situato sulla porta seriale e l'ambiente di simulazione virtuale integrato nella GUI. Dal punto di vista elettromeccanico, la sezione SIZES definisce le costanti geometriche del manipolatore planare a due gradi di libertà (RR), specificando le lunghezze del primo link (0.128 m) e del secondo (0.144 m); questi dati costituiscono la base dei calcoli per la cinematica diretta e inversa, permettendo al sistema di mappare correttamente lo spazio di lavoro operativo e le zone di irraggiungibilità.
La gestione dell'interfaccia utente web, orchestrata tramite il dizionario WEB_OPTIONS, introduce una logica di allocazione dinamica delle porte di rete impostando il valore su 0, una tecnica che delega al sistema operativo l'individuazione di un indirizzo libero per prevenire collisioni di socket durante il riavvio dei servizi.
La sicurezza attiva è ulteriormente garantita dal MAX_ACC_TOLERANCE_FACTOR, deputato a intercettare e bloccare la trasmissione di traiettorie anomale o potenzialmente dannose generate da input errati del frontend, mentre il flag DEBUG_MODE permette di modulare la verbosità dei log di sistema, ottimizzando il carico di I/O sul terminale durante le fasi di produzione per mantenere una visualizzazione pulita delle coordinate vettoriali in tempo reale senza sovraccaricare il processore con stampe di dati ciclici ridondanti.

Modulo gui-interface:

Il file gui_interface.py è l'API Gateway centrale dell'intero ecosistema DESKARM, esso agisce come l'unico punto di ingresso, controllo e gestione per tutte le comunicazioni tra gli utenti e i vari microservizi backend, Tramite la libreria Eel, permette una comunicazione bidirezionale e deterministica tra il backend Python e l'interfaccia JavaScript del browser. All'interno di questa architettura, ogni funzione decorata con @eel.expose viene elevata al rango di risorsa pubblica, consentendo al frontend di invocare procedure remote come se fossero componenti locali, garantendo al contempo la sicurezza e l'integrità dei dati.
La funzione più importante è *py_get_data()*, un motore di elaborazione critica che riceve gli array di primitive geometriche dal browser e avvia una complessa pipeline di trasformazione: il sistema concatena la posizione attuale del manipolatore al punto di

ingresso del disegno inserendo automaticamente un vettore di traslazione aerea per evitare collisioni o segni indesiderati sul foglio, procedendo poi alla Chiusura dei Micro-Gap mediante un algoritmo di coesione magnetica che rileva distanze inferiori a 0.1 mm per stabilizzare il tratto tra una lettera e l'altra. Durante questa fase, il modulo invoca *validate_trajectory* per operare un controllo dinamico preventivo; se il profilo di movimento richiesto eccede i limiti fisici dei motori, il sistema scala temporalmente la traiettoria, ricalcolando le tre matrici fondamentali delle variabili di giunto q, delle velocità dq e delle accelerazioni ddq prima di inoltrarle al gestore della comunicazione seriale.
Al tempo stesso, il file controlla l'hardware, attraverso le funzioni:
-*py_stop_trajectory*, agisce come un'interruzione di emergenza che solleva una alert nello stato globale e trasmette il comando firmware 0x03 (STOP) per paralizzare istantaneamente i motori.
-*py_homing_cmd* gestisce il riposizionamento del braccio, calcolando traiettorie cicloidali virtuali in modalità emulazione per garantire un feedback fluido sulla GUI anche in assenza di hardware connesso.
La versatilità del backend si estende alla gestione del testo tramite
-py_generate_text, limitando gli input a 500 caratteri per evitare crash di sistema e delegando a *char_gen.py* la scomposizione dei glifi in vettori; qui, le routine *apply_linear_transform* e *apply_curved_transform* proiettano le stringhe sul piano di lavoro, permettendo deformazioni geometriche complesse come la disposizione circolare o diagonale dei caratteri.
Per concludere, il modulo coordina l'esperienza sensoriale attraverso
-*py_trigger_sound* e *play_pc_melody*, utilizzando i driver dei motori come sintetizzatori per generare melodie di feedback o attivando i segnali acustici dell'OS per monitorare lo stato del sistema.

Modulo main:

Il file main.py agisce come l'Entry Point definitivo e attiva l'intero ecosistema DESKARM, orchestrando la cooperazione tra i diversi strati del software attraverso una gestione avanzata della concorrenza. Al vertice dello script, è presente il modulo *gevent* con la procedura di *monkey.patch_all()*, che rappresenta una scelta architettonica cruciale alterando il comportamento standard delle librerie di rete e di I/O di Python, trasformando il runtime in un ambiente asincrono e cooperativo, permettendo al server web locale di coesistere con i processi di calcolo delle spline cinetiche e con il monitoraggio della porta USB senza generare latenze o freeze dell'interfaccia.
Per garantire la robustezza del sistema durante la fase di terminazione, la routine *handle_closure* implementa una strategia di Graceful Exit (chiusura con grazia) per prevenire la formazione di processi zombie; intercettando la chiusura del browser o il segnale di interrupt, la funzione arresta preventivamente il *serial_manager* e svincola le risorse hardware della porta COM, evitando che il canale di comunicazione rimanga

occupato in background e costringa l'utente a interventi manuali sull'hardware.
La sezione principale, racchiusa nel costrutto
"*if __name__ == "__main__":*", esprime la resilienza del progetto tentando un'inizializzazione anticipata della seriale tramite *scm.ser_init*; la logica è progettata per gestire il fallimento critico del collegamento hardware non interrompendo l'esecuzione, ma commutando automaticamente in modalità Simulator/Demo GUI, garantendo così la continuità dello sviluppo software anche in assenza fisica del manipolatore.
Contemporaneamente, viene lanciato il thread sotterraneo *serial_manager.start_monitor()*, un'entità multithreading dedicata esclusivamente all'ascolto e alla decodifica dei pacchetti binari in tempo reale, isolata dalla logica di presentazione dell'interfaccia.
Dopo aver calcolato dinamicamente i percorsi assoluti, il programma esegue il comando *eel.start()*, avviando la sequenza dell'interfaccia su index.html e bloccando l'esecuzione in uno stato di ascolto perpetuo, mantenendo in vita l'intero apparato finché l'operatore non decide di concludere la sessione di lavoro.

Modulo plotting:

Il modulo **plotting.py** costituisce l'unità di diagnostica e post-processing visuale del progetto DESKARM, attraverso la libreria matematica **Matplotlib [17]**. Quando il sistema opera in *DEBUG_MODE* o al termine di una sessione di simulazione, questo modulo genera autonomamente una directory /images per l'archiviazione di "reperti" grafici che permettono di validare la precisione dei profili di velocità e l'integrità delle traiettorie prima dell'invio al firmware. All'interno della suite di debug standard, la funzione debug_plot(q, name) analizza l'evoluzione temporale degli angoli di giunto, permettendo di verificare visivamente la corretta generazione dei profili **S-Curve**, **Trapezoidale, Quintic** fondamentali per la fluidità del moto, mentre *debug_plotXY(x,y)* esegue il rendering sul piano cartesiano delle coordinate spaziali derivate dalla cinematica diretta per intercettare eventuali distorsioni geometriche pre-attuazione.
Il contributo ingegneristicamente più raffinato è rappresentato dalla funzione *plot_recorded_data*, un metodo di analisi comparativa avanzata che mette a confronto la realtà virtuale con la realtà fisica dei dati registrati. Per ovviare ai fenomeni di **Time-Drift** e **jitter temporale** indotti dai micro-lag della comunicazione asincrona, particolarmente evidenti nelle sessioni emulate via Eel, il codice implementa un algoritmo di **Allineamento Temporale (Time-Warping)**: attraverso l'analisi del gradiente matematico della velocità, il software individua i trigger istantanei di inizio e fine movimento allo scopo di effettuare un ritaglio millimetrico e una dilatazione elastica delle basi dei tempi, ricombinando i segnali in una sovrapposizione perfettamente fasata che scongiuri l'insorgenza di falsi positivi negli errori di inseguimento del PID.
Questo processo culmina nella generazione di due output fondamentali: **recorded_trajectory.png**, dove il profilo desiderato viene confrontato con il segnale solido catturato dai sensori, e **recorded_xy.png**, che funge da foglio di prova finale per

la visualizzazione dell'errore di posizionamento spaziale. In quest'ultima visualizzazione, l'impiego della direttiva *plt.axis('equal')* è cruciale per imporre un rapporto di scala 1:1 rigoroso, garantendo che l'utente possa valutare la precisione millimetrica della mano meccanica senza distorsioni ottiche provocate dall'auto-scaling del viewport grafico.

Modulo serial_manager:

Il modulo **serial_manager.py** costituisce il fulcro dell'infrastruttura di comunicazione del progetto DESKARM operando come coordinatore asincrono che interfaccia il driver di basso livello (serial_com.py) con l'ambiente ad alto livello della simulazione e dell'interfaccia web. In termini architettonici, questo componente agisce da Centralinista che risiede in un thread parallelo dedicato, garantendo che le operazioni di I/O sulla porta seriale non interferiscano con la reattività della GUI.
La necessità di questa separazione deriva dalla natura bloccante delle chiamate di lettura seriale in Python; per ovviare a tale criticità, il sistema istanzia un *monitor_thread* configurato come demone impostato a True, si arresta esclusivamente alla chiusura dell'applicazione. Il cuore operativo di questo thread è il metodo *monitor_loop*, un ciclo infinito che esegue il polling dei buffer hardware con una frequenza di aggiornamento di circa 200 Hz, ad intervalli di 5 ms. Durante ogni iterazione, il software esegue una scansione deterministica alla ricerca dei valori di allineamento del frame, identificati dai byte 0xA5 e 0x5A; una volta validato l'header, il sistema procede alla decodifica dei pacchetti binari.
Nel caso specifico del tipo RESP_POS, il feedback di posizione, il manager estrae i 12 byte del payload per un totale di 15 byte complessivi, decodificando istantaneamente le coordinate di giunto q_0 e q_1 provenienti dai sensori reali. Questi dati non solo aggiornano lo stato interno del backend, bypassando le previsioni teoriche dell'executor, ma vengono simultaneamente archiviati come telemetria grezza per le analisi delle prestazioni effettuate dal modulo plotting.py.
L'innovazione principale del sistema consiste nel passaggio da una logica di aggiornamento "Pull" (Polling) a un'architettura "Push". Il SerialManager, utilizzando l'istruzione eel.js_draw_pose, trasmette i dati al frontend a 20 fps, garantendo un'animazione fluida e minimizzando la latenza tra il manipolatore fisico e la sua rappresentazione digitale. In uscita, il metodo send_data agisce da dispacciatore intelligente tramite una logica match-case. Se riceve una traiettoria (trj), la affida al TrajectoryExecutor, che ne fraziona l'invio in pacchetti da 50 ms per non sovraccaricare la memoria del microcontrollore. Al contrario, i comandi diretti (cmd) come procedure di homing, SOS o segnali acustici vengono inoltrati istantaneamente sulla connessione hardware, scavalcando qualsiasi coda per garantire una risposta in tempo reale.

Modulo state:

Il modulo state.py costituisce l'infrastruttura di memoria condivisa dell'ecosistema DESKARM, rivelandosi il pilastro dell'architettura multithreading del progetto. Poiché il sistema esegue in parallelo il server web, l'esecutore delle traiettorie e il monitor seriale, questa gestione centralizzata è l'unico baluardo contro le Race Condition e la corruzione dei dati. Il modulo garantisce l'integrità delle informazioni attraverso due classi nevralgiche:

-**FirmwareState**: Replica fedelmente l'hardware memorizzando in tempo reale le variabili di giunto q_0 e q_1, lo stato dell'end-effector e la saturazione del buffer del microcontrollore. L'integrità è assicurata da un Mutex (threading.Lock): ogni metodo di accesso ai dati è incapsulato in un contesto protetto, scongiurando letture inconsistenti durante gli accessi concorrenti tra frontend.

-**seriale.RobotState** (Coordinatore Logico): Supervisiona lo stato di alto livello del backend. Gestisce il logging telemetrico (recording_active), implementa un protocollo di emergenza E-STOP asincrono (stop_requested, che blocca i motori se attivato dal web) e ottimizza il traffico di rete salvando la posa in last_known_q, permettendo di inizializzare nuove traiettorie senza interrogazioni hardware ridondanti.L'intera struttura è implementata secondo il pattern Singleton, rendendo l'oggetto di stato un'entità unica, globalmente accessibile e operativamente sicura per tutto il sistema.

4.3.2 BACKEND DI ALTO LIVELLO

Modulo binary_protocol:

Il presente modulo si fonda sul consolidato paradigma architetturale Master/Slave, un modello di comunicazione asimmetrico e deterministico. In tale costrutto, il Master opera come unità di elaborazione centrale, demandata al decision-making, alla pianificazione delle traiettorie e all'orchestrazione del traffico dati. Le unità Slave, di contro, agiscono come nodi subordinati e reattivi: eseguono strettamente le istruzioni impartite dal livello superiore e trasmettono un feedback esclusivamente a seguito di un'interrogazione diretta (polling).
Il Modulo binary_protocol.py. costituisce l'infrastruttura fondante del data-link tra l'ambiente di controllo (Master), sviluppato in Python, e il microcontrollore (Slave). La sua implementazione decreta il definitivo superamento della trasmissione testuale ASCII a favore di un protocollo binario di basso livello, ingegnerizzato per massimizzare il throughput, ottimizzare l'efficienza spettrale della banda e garantire un'elevata immunità alle interferenze elettromagnetiche (EMI).

L'allineamento spaziale dei pacchetti dati è garantito dall'impiego di costanti di sincronizzazione, specificatamente i byte 0xA5 e 0x5A. Questa sequenza di intestazione (header) obbliga l'hardware ricevente a scartare il rumore di fondo, elaborando il payload solo in seguito all'identificazione esatta del pattern di validazione. Il protocollo definisce inoltre un dizionario operativo rigoroso, composto da identificatori di comando (es. CMD_TRAJECTORY 0x01, CMD_STOP 0x03) e codici di stato (es. RESP_POS 0x02), assicurando un dialogo deterministico tra i nodi.
La robustezza della trasmissione è affidata all'algoritmo *calculate_crc32*. Questa funzione genera un codice di controllo a 32 bit basato su ridondanza ciclica, incapsulando ogni frame. Il ricevitore ricalcola localmente la firma per validare il pacchetto, azzerando il rischio di accettare dati affetti da corruzione bit-level indotta dal mezzo trasmissivo.
La codifica in trasmissione sfrutta la libreria Python struct per mappare le variabili di alto livello in blocchi binari nativamente compatibili con l'architettura C dello Slave.
La funzione critica *encode_trajectory_point* comprime un payload cinematico complesso (sei variabili float e un unsigned char per lo stato dell'end-effector) nel formato standardizzato <fffffffB. Questa elevata densità informativa riduce l'occupazione spaziale a soli 32 byte, minimizzando il tempo di latenza sul canale seriale.
Per i comandi atomici, il protocollo alloca pacchetti dummy a lunghezza fissa, mentre varianti funzionali come *encode_melody_command* applicano un polimorfismo sui byte di stato per incapsulare parametri secondari, quali l'ID della traccia audio o il periodo di campionamento T_c espresso in millisecondi. In fase di ricezione, il processo di parsing inverte la serializzazione. La funzione *decode_position_feedback* agisce come un filtro logico di sicurezza: estrae i 15 byte relativi alla posa articolare e subordina l'aggiornamento dell'interfaccia grafica alla validazione del CRC. I pacchetti invalidati vengono attivamente rigettati, generando log di errore in console. Tale meccanismo di reiezione previene la proiezione a schermo di configurazioni anomale, garantendo una coerenza assoluta tra l'esecuzione fisica dei motori e la sua rappresentazione nel Digital Twin (GUI).

Modulo char-gen:

Il modulo char_gen.py funge da tipografo vettoriale del manipolatore DESKARM, costituendo lo strato logico deputato alla trasformazione di stringhe alfanumeriche in sequenze deterministiche di traiettorie per l'end-effector. A differenza dei font convenzionali di tipo raster o outline, questo modulo implementa un sistema di Stick Font [15], in cui ogni carattere, simbolo o icona è descritto esclusivamente attraverso **primitive geometriche vettoriali**. All'interno del dizionario FONT_DEFS, la morfologia di ogni glifo è definita mediante una combinazione di segmenti lineari e archi ellittici, le cui coordinate sono rigorosamente normalizzate all'interno di un dominio unitario 1 X 1; tale astrazione geometrica permette al sistema di scalare linearmente ogni lettera in base al parametro *font_size* scelto dall'operatore senza incorrere in artefatti di quantizzazione. La gestione delle geometrie curve rappresenta uno dei punti di forza del modulo: poiché il

firmware del microcontrollore esegue esclusivamente interpolazioni lineari tra punti discreti, la funzione *sample_ellipse* interviene per operare una discretizzazione ad alta risoluzione. Seguendo la logica "Ultra Smooth", l'algoritmo campiona l'equazione parametrica dell'arco con una densità di un punto per ogni grado sessagesimale, spezzettando la curva matematica in una serie di micro-segmenti contigui che, pur essendo intrinsecamente poligonali, risultano fluidi e perfettamente circolari alla percezione umana. Questo processo di discretizzazione trasforma l'eleganza analitica del font in un carico di lavoro gestibile per i driver dei motori stepper, garantendo che il robot possa riprodurre il suo vocabolario di caratteri con una precisione millimetrica e una continuità di tratto professionale.

Generazione della Traiettoria

La funzione *text_to_traj* rappresenta l'istanza di coordinamento superiore all'interno del modulo di generazione tipografica, agendo come il motore di sintesi che trasforma una sequenza di glifi astratti in una catena cinematica di segmenti di traiettoria pronti per l'attuazione. Il processo inizia con una fase di scalatura e mappatura spaziale, in cui le coordinate normalizzate del vocabolario vengono moltiplicate per il parametro *font_size* e traslate lungo l'asse di scrittura in base alla posizione del cursore virtuale e alla spaziatura intercarattere definita. Un elemento non di poco conto risiede nella logica operativa che gestisce autonomamente gli stati dell'end-effector: il sistema analizza costantemente la distanza euclidea tra il termine di una primitiva e l'inizio della successiva e qualora rilevi una discontinuità spaziale, inietta automaticamente un comando di sollevamento della penna. Questa procedura evita che il robot lasci segni indesiderati sul supporto fisico durante le traslazioni aeree tra lettere o segmenti non contigui dello stesso carattere. Parallelamente, l'algoritmo implementa una strategia di **profilazione dinamica della velocità**: identificando i caratteri appartenenti alla classe CURVED_CHARS (come la 'O' o la 'C'), il sistema commuta il profilo di interpolazione verso modalità specifiche, modulando la velocità tangenziale per garantire che i raccordi circolari siano eseguiti con maggiore precisione cinetica rispetto ai tratti rettilinei. Tuttavia, l'aspetto tecnicamente più sofisticato risiede nel **sistema di rilevamento degli spigoli vivi**, progettato per contrastare l'inerzia meccanica che tenderebbe a smussare i vertici acuti, come per le lettere 'Z' o 'W', a causa delle accelerazioni residue. Attraverso l'analisi vettoriale di tre punti sequenziali, l'algoritmo calcola il prodotto scalare normalizzato tra i vettori di ingresso e di uscita; se il valore di similarità risulta inferiore a 0.707, soglia che identifica una variazione angolare superiore a 45, il sistema intercetta la discontinuità geometrica e interrompe deliberatamente la continuità della traiettoria nel vertice. Questo blocco forzato impone al pianificatore cinematico a valle (Trapezoidal Planner) di generare una rampa di decelerazione fino a velocità nulla esattamente nel punto di giunzione, permettendo al manipolatore di cambiare direzione da fermo prima di ripartire. Questo approccio garantisce la riproduzione di spigoli perfettamente nitidi e privi di artefatti dovuti al trascinamento inerziale, elevando la qualità del tratto grafico a uno standard di tipo industriale.

Modulo executor:

Il modulo executor.py, incardinato sulla classe centrale TrajectoryExecutor, costituisce l'infrastruttura di gestione e attuazione del sistema, demandata alla trasmissione deterministica dei tensori cinematici — posizione, velocità e accelerazione — verso l'hardware fisico o l'ambiente virtuale. Per ovviare alla marcata asimmetria di banda e memoria tra il calcolatore general-purpose e il microcontrollore STM32, il sistema previene il critico fenomeno del buffer underrun (causa primaria di vibrazioni e discontinuità nel moto) implementando una rigorosa strategia di controllo di flusso a batch : la routine avvia una fase di pre-roll per saturare preventivamente la memoria del firmware con 50 setpoint, transitando poi in un ciclo di mantenimento che rabbocca il buffer ogni 50 ms (assumendo un periodo di campionamento T_c di 10 ms) e garantendo al contempo l'intercettazione asincrona del flag di emergenza (E-STOP). Qualora il sistema operi disaccoppiato dall'hardware, l'architettura commuta fluidamente in modalità simulazione, operando come un Digital Twin governato da un rigoroso clock software assoluto; in questo scenario, il modulo aggiorna sincronicamente la classe FirmwareState e inietta le coordinate nel frontend tramite il bridge asincrono Eel, offrendo all'operatore un'anteprima visiva perfettamente fasata e archiviando i setpoint per la generazione di report diagnostici atti a validare la correttezza dei profili di velocità trapezoidali prima di procedere all'esecuzione fisica sul campo.

Modulo image-processor:

Il modulo image_processor.py costituisce il sottosistema di traduzione visiva dell'architettura DESKARM, fungendo da strato computazionale deputato alla conversione di input grafici eterogenei — dalle acquisizioni fotografiche ai file vettoriali SVG — in traiettorie spaziali continue sul piano cartesiano x-y, strettamente compatibili con la cinematica del manipolatore. Sfruttando le primitive della libreria di computer vision OpenCV [16], l'intero processo è governato dalla funzione gateway process_image, che identifica la natura del file (raster o vettoriale) e avvia una rigorosa normalizzazione geometrica: tramite l'estrazione della bounding box, i pixel vengono mappati in coordinate metriche reali in funzione della larghezza target (target_w) specificata dall'operatore. Il modulo integra, inoltre, un algoritmo di rotazione intelligente (Smart Rotation) che, analizzando il rapporto d'aspetto, orienta automaticamente i formati landscape di -90 gradi per massimizzare lo sfruttamento dell'area di lavoro lungo l'asse Y (la maggiore escursione del braccio) senza alcun intervento manuale. A valle della normalizzazione, il sistema seleziona dinamicamente la strategia di estrazione ottimale. Per i loghi e le sagome piene, la modalità Outline applica una sogliatura binaria e il rilevamento dei contorni, mitigando la frammentazione tramite compressione poligonale (*cv2.approxPolyDP*) e addolcendo le scalettature digitali mediante l'algoritmo di smoothing ricorsivo di Chaikin [13]. Di contro, per immagini ad alta densità o tratti sottili, la modalità Skeleton impiega l'algoritmo di assottigliamento di Zhang-Suen [14] per far

collassare i tratti a uno spessore unitario (pixel-level), ripulendo le ramificazioni spurie e gli artefatti di compressione tramite una funzione di filtraggio spaziale dedicata, prune_skeleton. La fase terminale della pipeline è votata all'ottimizzazione dell'efficienza meccanica: poiché l'output nativo di OpenCV presenta tipicamente segmenti spazialmente disordinati, la funzione _sort_paths implementa l'euristica del Nearest Neighbor per riordinare vettorialmente i tratti, minimizzando la distanza euclidea nelle transizioni a vuoto e abbattendo i tempi di riposizionamento del manipolatore. Infine, la funzione optimize_patches applica l'algoritmo di decimazione Ramer-Douglas-Peucker [11]-[12] ai segmenti rettilinei, restituendo un payload seriale estremamente compatto e privo di coordinate ridondanti, a garanzia di un'esecuzione fisica fluida, rapida e tecnicamente ineccepibile.

Modulo serial-com:

Il modulo serial_com.py costituisce lo strato di astrazione hardware di più basso livello dell'ecosistema DESKARM, fungendo da interfaccia fisica per il data-link bidirezionale tra l'host di calcolo e il microcontrollore STM32 via bus USB. Basato sull'impiego della libreria pyserial, il modulo instaura una connessione deterministica a un baudrate fisso di 115200 bps, precondizione stringente per il controllo cinematico in tempo reale. La routine di inizializzazione, incapsulata nella funzione ser_init, implementa un algoritmo di auto-discovery euristico: in assenza di una designazione esplicita della porta da parte della GUI, il sistema esegue una scansione ricorsiva delle periferiche enumerando i nodi attivi, avvalendosi di una lista di fallback predefinita (es. /dev/ttyACM0 o COM4) per forzare l'aggancio su indirizzi hardware canonici. Un connotato ingegneristico di spiccata rilevanza risiede nella gestione dell'inizializzazione del microcontrollore, ottenuta mediante la manipolazione elettrica del segnale DTR (Data Terminal Ready). L'inversione logica programmata di tale pin induce un Hard Reset fisico sulla scheda madre del robot; questa tecnica assicura l'avvio del sistema embedded da uno stato noto e incontaminato, provvedendo contestualmente allo svuotamento dei buffer hardware (reset_input_buffer()) per neutralizzare eventuali pacchetti orfani o residui di sessioni pregresse che comprometterebbero l'allineamento del protocollo binario. Sotto il profilo operativo, il modulo espone un'interfaccia duale: una modalità legacy per la transazione di stringhe ASCII terminate da newline (riservata a finalità di debug) e una modalità binaria nativa, asse portante dell'architettura attuale. In quest'ultimo dominio, i metodi write_data e read_data processano esclusivamente stream di byte puri generati dal protocollo di livello superiore; la ricezione è rigorosamente vincolata all'intercettazione di frame a dimensione fissa (come il payload da 15 byte deputato al feedback di posizione), azzerando la latenza intrinseca del parsing testuale e massimizzando il throughput del canale. Infine, l'intera infrastruttura di I/O è blindata da una topologia di gestione delle eccezioni "silenziosa", ingegnerizzata per scongiurare crash critici dell'applicativo a fronte di disconnessioni fisiche accidentali del cablaggio, preservando la stabilità del demone Python e garantendo la prontezza operativa per una riconnessione

a caldo senza necessità di riavviare il software.

Modulo trajpy:

Il modulo trajpy.py costituisce il nucleo algoritmico e computazionale dell'architettura DESKARM, fungendo da libreria matematica dedicata alla cinematica e alla pianificazione avanzata del moto. La sua funzione primaria consiste nel mappare le astrazioni geometriche del piano cartesiano nei domini articolari (q_1, q_2) dei servomotori, vincolando la dinamica per prevenire sollecitazioni meccaniche critiche. Il sistema implementa un robusto framework di risolutori basati su spline e interpolazioni polinomiali di grado elevato (cubici, quartici e quintici) per la generazione di traiettorie fluide; tramite l'inversione delle matrici di Vandermonde (np.linalg.inv) [18], le funzioni spline3 e spline5 ricavano analiticamente i coefficienti necessari a imporre la continuità su posizione, velocità e accelerazione (S-V-A). Congiuntamente, il factory method get_profile_law governa le leggi di moto, consentendo la transizione dal classico profilo trapezoidale — standard industriale ottimizzato per l'efficienza a velocità di crociera — a quello cicloidale, ingegnerizzato per annullare il jerk (la derivata dell'accelerazione) e abbattere le risonanze strutturali nei transienti. Il substrato spaziale si fonda sulle funzioni di cinematica: la Cinematica Diretta (DK) ricava la posa dell'end-effector mediante trigonometria elementare sui link, mentre la Cinematica Inversa (IK) risolve il problema non lineare ricorrendo al teorema di Carnot, implementando routine di clamping per neutralizzare errori di dominio (Math Domain Error) in caso di target extra-spazio di lavoro. L'intero flusso converge nel metodo slice_trj, che agisce da supervisore terminale: elaborando le primitive geometriche, la funzione applica la legge oraria selezionata, stima il tempo di esecuzione t_f e discretizza il moto con un periodo di campionamento Tc = 10 ms. Per ogni istante temporale, l'algoritmo risolve la cinematica inversa e produce un tensore discreto di setpoint (angoli, velocità, accelerazioni), successivamente iniettato nel modulo esecutivo per garantire un tracking fisico di precisione millimetrica.

Modulo transform:

Il modulo transform.py opera come l'architetto morfologico dell'ecosistema, demandato a una complessa alterazione topologica definita Mappatura Cilindrica. La sua funzione è trasporre primitive geometriche rettilinee, tipicamente i percorsi tipografici generati da char_gen.py, avvolgendole matematicamente lungo un arco circonferenziale per generare output grafici ad anello. La logica della funzione *apply_curved_transform* trascende la mera rotazione bidimensionale, attuando una rigorosa conversione dal dominio cartesiano a quello polare: l'asse X originario viene reinterpretato come lunghezza d'arco per determinare la fase angolare, mentre l'asse Y (l'elevazione del carattere) definisce lo scostamento radiale rispetto al raggio di curvatura. Tale trasformazione vincola le rette

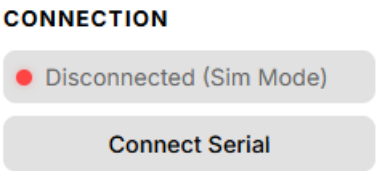
verticali dei glifi a convergere verso il fuoco del sistema, preservando al contempo l'invarianza dell'asse Z per non corrompere le transizioni aeree dell'end-effector. Il calcolo viene eseguito riparametrizzando ogni segmento interpolato tramite la computazione dell'offset angolare istantaneo e della fase theta. L'uso della sottrazione nel calcolo di theta impone al robot di scrivere avvolgendo le lettere in senso orario (Clockwise): partendo dallo zenit (90° o "ore 12"), il manipolatore si muove verso destra mentre la stringa prosegue. Una volta completata la deformazione polare, l'algoritmo esegue la riconversione finale in formato Cartesiano per alleggerire il carico dei motori. Questo processo permette di generare traiettorie dalla complessità visiva estrema senza che il generatore di traiettoria principale debba minimamente preoccuparsi della curvatura, mantenendo il sistema modulare e matematicamente elegante.

CAPITOLO 5: GUIDA ALL'USO DELL'INTERFACCIA GRAFICA

Il seguente capitolo illustra come utilizzare le implementazioni spiegate nel capitolo precedente.

5.1 CONFIGURAZIONE E CONNESSIONE SERIALE:

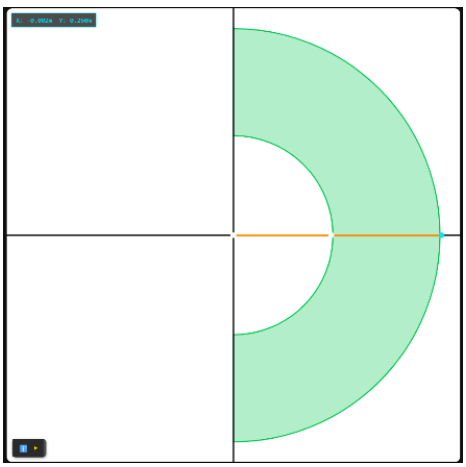
Partendo con ordine sono stati implementati per valutare l'effettivo collegamento della seriale due elementi chiave: lo **status check** della connessione e un pulsante virtuale per permettere la connessione, di seguito la foto di questa prima implementazione:



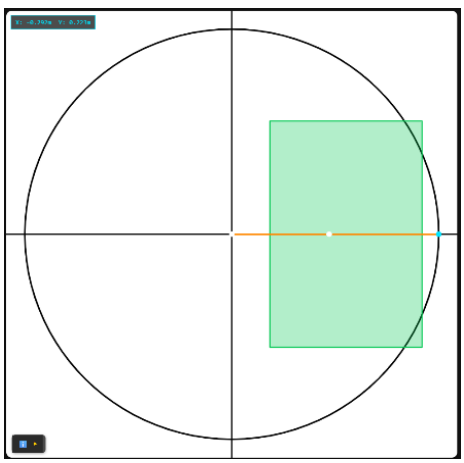
5.1 Connessione Seriale

5.2 DEFINIZIONE DEL WORKSPACE:

Il sistema è stato ampliato definendo il piano di lavoro del manipolatore, ora è possibile scegliere tra un piano curvilineo e uno di forma rettangolare.



5.2 Workspace Digitale Curvo



5.3 Workspace Digitale Lineare

5.3 MENU CUSTOMIZATION

Per rendere l'esperienza d'uso adattabile alle diverse condizioni di illuminazione ambientale e alle preferenze dell'operatore, è stato implementato un menu a scomparsa denominato CUSTOMIZATION, posizionato strategicamente nell'angolo in alto a destra dell'interfaccia. Questo pannello centralizza tutte le impostazioni estetiche e geometriche della simulazione, permettendo una configurazione dinamica senza interrompere il flusso di lavoro sul canvas.

Personalizzazione Estetica e Visiva

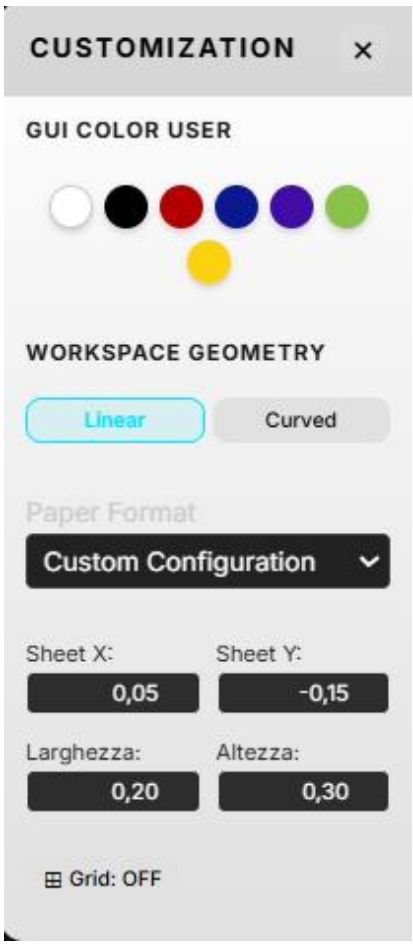
Il menu offre una selezione di 7 tipologie di temi cromatici per l'intera interfaccia. Questa scelta non ha solo una valenza estetica, ma funzionale: permette di aumentare il contrasto tra il cursore e il piano di lavoro, riducendo l'affaticamento visivo durante sessioni prolungate di progettazione o monitoraggio.

Gestione Dinamica del Piano di Lavoro

La personalizzazione si estende alla geometria stessa dell'area operativa, con opzioni specifiche per la tipologia di cinematica scelta:

Workspace Curvilineo: In questa modalità, l'utente può intervenire direttamente sulle dimensioni e sulla posizione del piano. È possibile scalare l'area di lavoro o traslarla nel piano cartesiano dell'interfaccia per allinearla perfettamente al raggio d'azione fisico dei link del manipolatore.

Workspace Rettangolare: Per simulare l'utilizzo di supporti cartacei standard, il menu permette di selezionare diversi formati di foglio. Sebbene la libreria includa già i formati più comuni, sono stati inseriti dei preset per implementazioni future, garantendo la scalabilità del software verso nuovi standard di stampa o aree di lavoro estese.



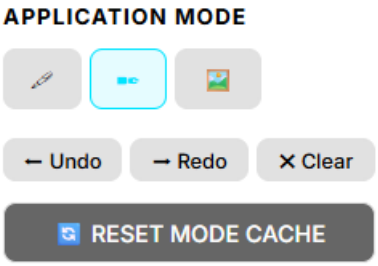
5.4 Customization Menu

Supporto alla Precisione: La Modalità Griglia

Un elemento fondamentale per il posizionamento accurato delle forme è la modalità griglia. Una volta attivata dal menu Customization, l'area di lavoro viene sovrapposta da un reticolo millimetrato dinamico. Questo strumento fornisce un riferimento spaziale costante, essenziale quando si opera in Drawing Mode per allineare vettori o per verificare le proporzioni delle traiettorie generate in Image Mode rispetto alle dimensioni reali del foglio.

5.4 MENU PRINCIPALE

La parte più importante dell'interfaccia è il menu a sinistra in cui è possibile scegliere fra 3 modalità d'uso, tornare indietro o andare avanti, pulire il canvas e reimpostare la cache della modalità. Quando si passa da una modalità ad un'altra il manipolatore emetterà dei suoni specifici per avvertire l'utente del passaggio alla modalità selezionata.
Sono state inserite altre 2 melodie per la conferma del riconoscimento della seriale e alla

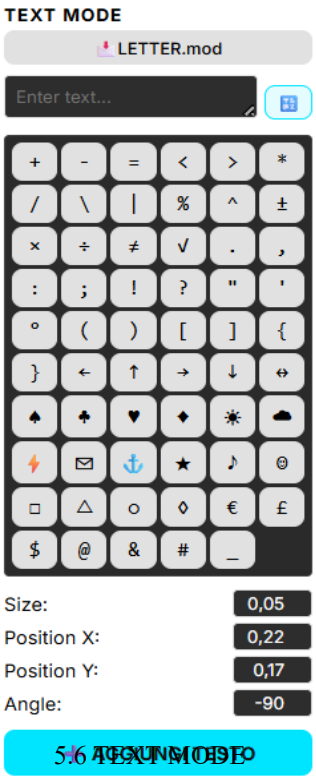


5.5 Application Menu

fine del tracciamento di una sequenza. Le modalità non sono concorrenti ed è possibile usarle contemporaneamente.

5.4.1 TEXT MODE

La modalità TEXT MODE è stata progettata per trasformare input testuali digitali in traiettorie cinematiche continue. A differenza della semplice grafica vettoriale, questa modalità integra un motore di generazione di caratteri capace di gestire l'intero set alfanumerico, inclusi simboli matematici, punteggiatura e caratteri speciali.
Configurazione dei Parametri di Scrittura:
Prima di inviare il comando al manipolatore, l'interfaccia permette una regolazione fine di diversi parametri che influenzano direttamente la qualità del tratto fisico: Contenuto Testuale, Inserimento dinamico di stringhe di testo semplici o complesse. Dimensione e Scalabilità, Regolazione della grandezza del carattere per adattarsi alle dimensioni del workspace selezionato, Parametri di Spaziatura, Gestione dell'interlinea e della spaziatura tra i caratteri per garantire la leggibilità del testo una volta tracciato sul supporto fisico. Una volta definiti questi

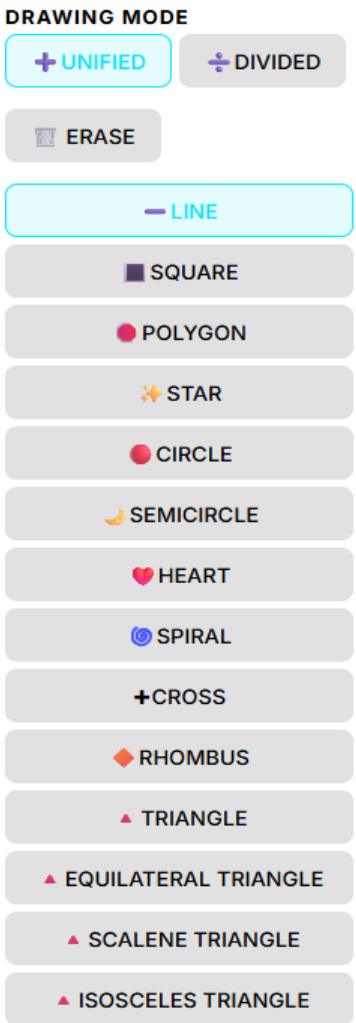


5.6 TEXT MODE

parametri, il comando “AGGIUNGI TESTO” converte i glifi digitali in una serie di coordinate (x, y), pronti per essere processati dal pianificatore di traiettoria.
Funzionalità Avanzata: Modalità LETTERA
Un aspetto distintivo di questa sezione è la modalità LETTERA. Questa funzione trasforma il canvas in un layout preformattato, ottimizzato per la redazione di documenti o messaggi personali.
Formattazione Automatica: Il sistema imposta margini di sicurezza e una giustificazione del testo che impedisce al manipolatore di uscire dai confini fisici del foglio. Ottimizzazione del Tratto: Nella modalità lettera, l'algoritmo di pianificazione privilegia la fluidità del movimento per simulare una scrittura manuale naturale, riducendo i tempi morti tra la fine di una parola e l'inizio della successiva.

5.4.2 DRAWING MODE

La modalità DRAWING MODE rappresenta l'estensione più flessibile dell'interfaccia, configurandosi come un vero e proprio ambiente CAD (Computer-Aided Design) semplificato. A differenza di un comune software di disegno raster, questa modalità ragiona per entità geometriche vettoriali, permettendo al manipolatore di interpretare ogni forma non come un insieme di pixel, ma come una successione matematica di coordinate precise.



L'utente ha la possibilità di operare secondo due stili esecutivi:
Stile Unito: Le forme vengono generate in un'unica sequenza logica, ottimizzando gli spostamenti del braccio robotico.
Stile Singolare: Permette il posizionamento e la manipolazione di singole primitive geometriche all'interno del canvas.
Libreria delle Forme e Geometrie Complesse:
Il sistema mette a disposizione una vasta libreria di primitive, progettate per testare la fluidità di movimento del manipolatore SCARA su diversi profili di curvatura:
Lineari e Poligonali: Linee, quadrati, rombi e poligoni regolari con un range variabile da 5 a 12 lati (permettendo di passare da un pentagono a un dodecagono).
Curvilinee: Cerchi, semicerchi e spirali logaritmiche, essenziali per valutare la precisione dell'interpolazione dei

giunti.
Forme Simboliche: Forme complesse come stelle, cuori e la croce greca.
Forme triangolare: Sono state implementate 4 tipologie di triangolo (equilatero, rettangolo, isoscele e scaleno) per coprire ogni esigenza di layout geometrico.

Strumenti di Editing: La Gomma Vettoriale
Per elevare l'interfaccia a uno standard CAD professionale, è stata implementata una funzione di cancellazione selettiva. A differenza di una gomma tradizionale, questo strumento agisce sui nodi dei vettori: l'utente può individuare e rimuovere intere entità geometriche dal canvas con un click. Questo garantisce che la "cache" delle traiettorie venga aggiornata istantaneamente, assicurando che il robot non esegua movimenti residui di forme precedentemente eliminate.

5.4.3 IMAGE MODE:

La modalità IMAGE MODE rappresenta la funzione più avanzata dell'interfaccia, trasformando l'HMI da un semplice controller a un potente elaboratore di visione artificiale. Questa sezione permette di convertire file raster (JPG, PNG) in coordinate vettoriali eseguibili dal manipolatore SCARA, attraverso una pipeline di processamento dinamico.

Algoritmi di Segmentazione e Filtraggio

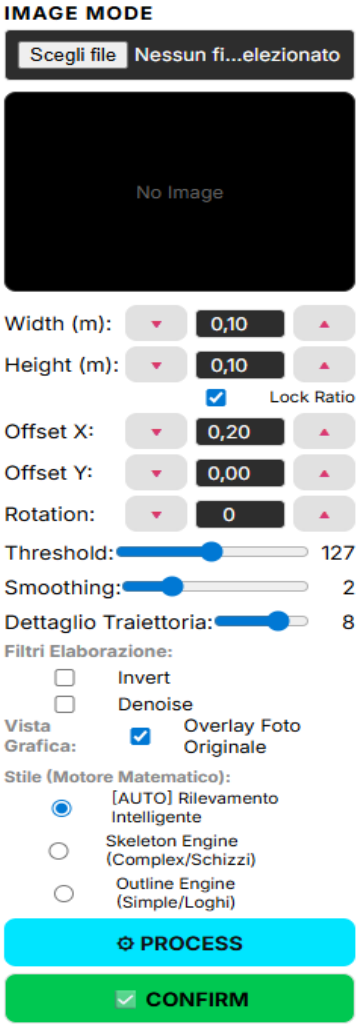
Il cuore di questa modalità risiede nella capacità di interpretare i gradienti di colore e luminosità per estrarre i contorni significativi. L'utente ha il controllo diretto su due parametri fondamentali:

Segmentazione (Soglia di Binarizzazione): Determina il livello di intensità oltre il quale un pixel viene considerato "tratto" o "sfondo". Regolando questo valore, si definisce quanto dettaglio l'algoritmo deve catturare dalla sorgente originale.

Smussatura (Smoothing): Un filtro di post-processing che agisce sui vettori generati per eliminare il rumore digitale e le "seghettature" tipiche delle immagini a bassa risoluzione. Questo parametro è cruciale per garantire movimenti fluidi dei servomotori, evitando micro-vibrazioni meccaniche durante il tracciamento.

Strategie di Elaborazione: Simple vs Complex

Per ottimizzare i tempi di calcolo e la fedeltà del risultato, sono stati implementati due stili di elaborazione distinti:



5.8 IMAGE MODE

Simple Style (Ottimizzato per Bianco e Nero):

Ideale per loghi, icone e disegni a tratto netto. In questa modalità, l'algoritmo lavora su una segmentazione standard (valori consigliati: 100-170). Il sistema identifica rapidamente i bordi chiusi e genera traiettorie lineari pulite, minimizzando i sollevamenti dell'end-effector.

Complex Style (Ottimizzato per Fotografie):

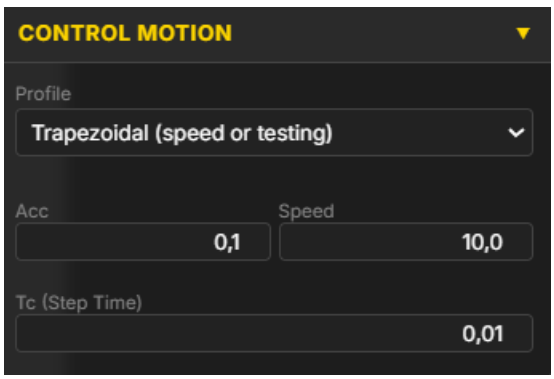
Progettato per gestire immagini ricche di sfumature e dettagli complessi. In questo caso, si consiglia di impostare valori bassi di Segmentazione e Smussatura per non perdere la definizione dei lineamenti o delle texture. L'algoritmo genera una densità di punti molto più elevata, traducendo le ombre in tratteggi o aree a riempimento variabile che il

manipolatore riproduce con estrema precisione.

Validazione e Risultati Sperimentali
L'efficacia del sistema di visione è stata validata attraverso test comparativi tra l'immagine digitale sorgente e il tratto fisico prodotto dal robot. I risultati hanno confermato la solidità degli algoritmi: il sistema è in grado di mantenere le proporzioni geometriche e la continuità del tratto sia su grafiche minimaliste che su ritratti fotografici complessi, dimostrando un'eccellente versatilità nella gestione dei dati in ingresso.

5.5 CONTROL MOTION MENU

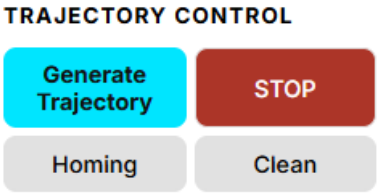
L'interfaccia presenta anche un altro menu a scomparsa:
che permette la selezione di 3 tecniche di pianificazione del movimento: Trapezoidale, S-curve, Polinomio di 5° grado e dei parametri di velocità, di accelerazione e del tempo di campionamento:



5.9 Control Motion Menu

5.6 MENU TRAIETTORIA

Una volta che si è sicuri di ciò che si vuole far tracciare dal robot basta andare nella sezione di controllo della traiettoria e selezionare sul pulsante “Generate Trajectory”;



5.10 Trajectory Control Menu

è possibile anche interrompere l’operazione di scrittura/disegno tramite il comando “STOP” ed è possibile eseguire il comando Homing e la pulizia del canvas.

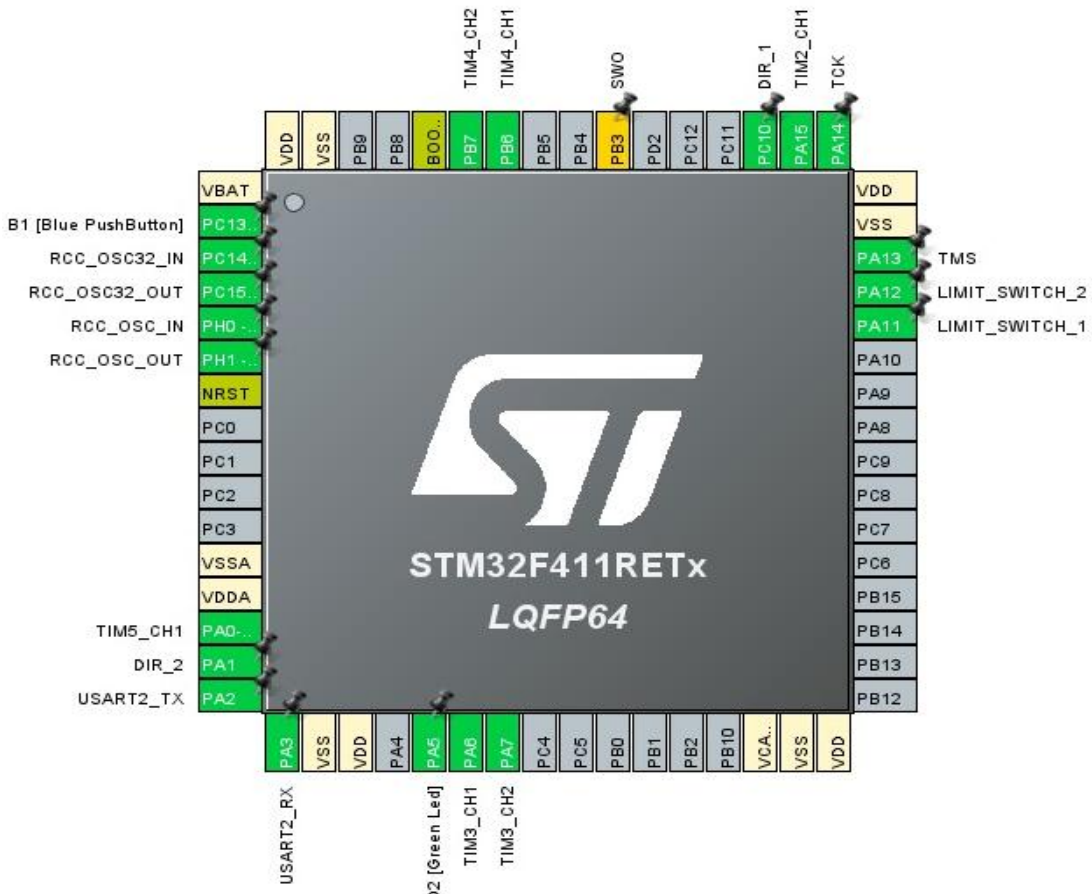
5.4 MODALITA’ DEBUG E SIMULAZIONE VIRTUALE E REALE DINAMICA:

Un nuovo aspetto è stato introdotto successivamente al sistema interfaccia. L’impiego della HMI è possibile in due configurazioni differenti, tramite il riconoscimento della connessione seriale. DEBUG MODE è il funzionamento del codice manipulator.js che permette di visualizzare la comunicazione tra hardware e manipolatore anche se non è presente. L’unico scopo è quello di calcolare la geometria del robot e di rappresentarlo correttamente all'interno del riquadro del browser, simulando esattamente come si muoverà o come è posizionato in quel momento nella realtà.
Altrimenti se la connessione seriale risulta positiva, dal file api.js gli encoder trasmettono i dati reali e manipulator.js permette la visualizzazione virtuale del braccio in tempo reale.

CAPITOLO 6: IMPLEMENTAZIONI SOFTWARE E HARDWARE

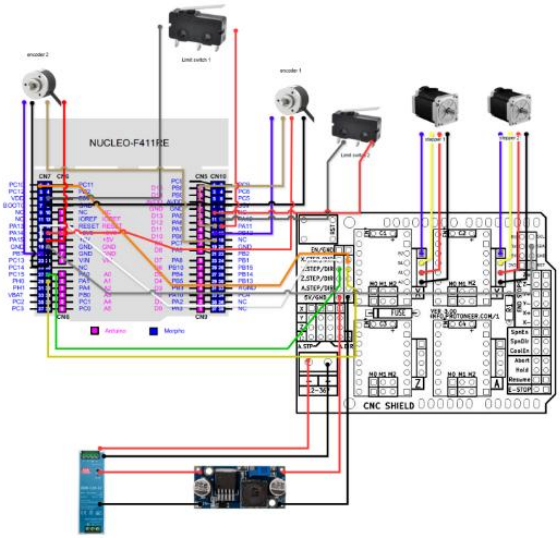
6.1 STATO INIZIALE E PROBLEMI INIZIALI:

L'esperienza di tesi è nata in laboratorio e si è occupata in primo luogo dello stato del sistema. La verifica attuata ha portato risultati tutti corretti tranne che per un errore lampante dovuto ad una impostazione imprecisa riguardante la modalità del pin per funzionamento come resistenza di polarizzazione. Dato che il limit switch al contatto ha una natura a circuito aperto per evitare rumore magnetico il limit switch in STM32CubeMX operava per valori di soglia e impostazioni hardware errate, impedendo al microcontrollore di rilevare correttamente i confini fisici del workspace. Questa configurazione è passata da Pull-down a Pull-up che imposta il pin a stato logico alto di default.



6.1 Schema .ino iniziale

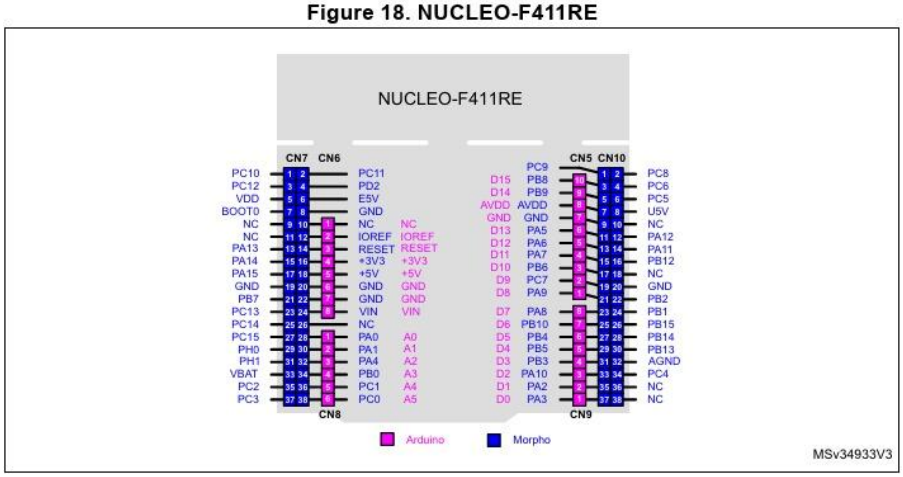
Il sistema era composto da 3 principali macro-componenti, viene illustrato prima il sistema elettrico completo e poi le componenti nel dettaglio:



6.2 Schema elettrico iniziale

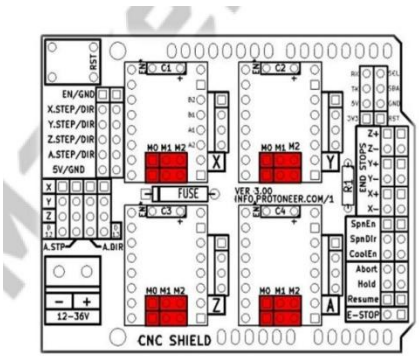
1) Il manipolatore SCARA
2) STM32 – Nucleo-F411RE:

Figure 18. NUCLEO-F411RE



6.3 Schema Pin-in/out del controllore STM32

3) la CNC-Shield:



6.3 CNC-Shield

basata su cablaggi provvisori. L'analisi ha rilevato gravi limiti in termini di affidabilità del segnale. La CNC-Shield [24], la nostra prima forma di controllo del manipolatore, per una serie di criticità ha evidenziato la necessità di abbandonare l'elettronica commerciale di basso livello in favore di un'architettura su misura più robusta.

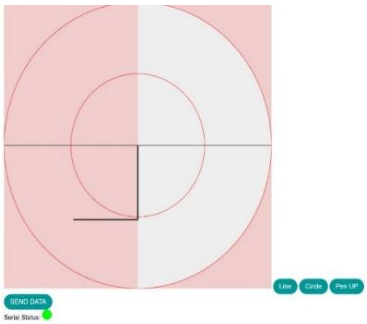
6.2 RIPROGETTAZIONE FIRMWARE:

Proseguendo sul profilo della valutazione all'inizio della tesi, il campo di ricerca non si è concentrato solo ad un aspetto puramente pratico, poichè mediante modellistica matematica, in ambienti Matlab e Simulink, si è riusciti ad ottenere una valutazione teorica del nostro modello e l'equazione dinamica del sistema dello stato iniziale. I tentativi iniziali sono stati di calcolare le matrici di inerzia e Coriolis tramite pura equazione Lagrangiana anche se questa strada ha condotto la tesi ad incongruenze numeriche. L'impiego di *Robotics Toolbox* ha permesso al processo l'estrazione di un modello fedele alla realtà, essenziale per la simulazione di traiettorie critiche polinomiali del settimo grado.

La trasposizione del controllo a dinamica inversa (*Computed Torque*) sul microcontrollore STM32F4 ha tuttavia evidenziato severi colli di bottiglia computazionali legati all'unità in virgola mobile (FPU). Di conseguenza, il progetto ha visto una profonda riscrittura del firmware: si è passati da un'architettura monolitica ad una robusta modulare in linguaggio C, adottando un controllo PID indipendente sui singoli giunti coadiuvato da un termine di *Feedforward* cinematico. Questa scelta, abbinata all'uso dell'accesso diretto in memoria (DMA) per la comunicazione seriale, ha abbattuto il carico CPU, garantendo un *tracking* stabile ad alte accelerazioni.

6.3 PROGETTAZIONE E UTILIZZO DELL'HMI:

Il processo di realizzazione dell'interfaccia è stato implementato successivamente. Lo stadio iniziale dell'interfaccia permetteva il tracciamento esclusivo di linee e curve selezionabili alla destra del canvas e la possibilità di inviare i dati dei punti e **homing** al microcontrollore. Vi era la possibilità di visualizzare le anteprime delle traiettorie dello spostamento del robot.

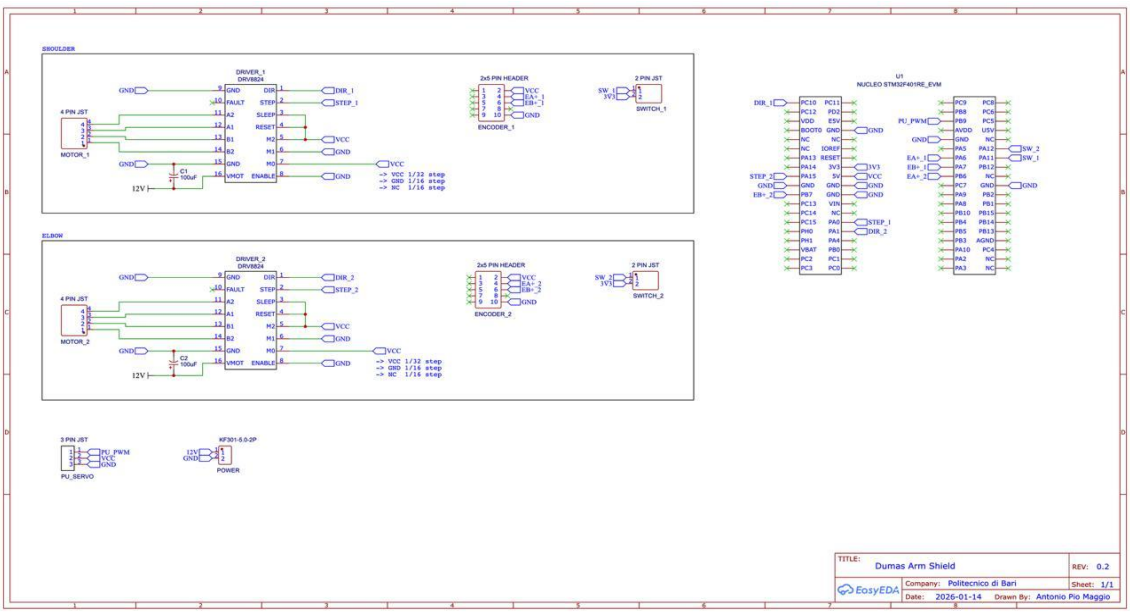


6.4 Interfaccia Grafica Iniziale

Il rimodellamento dell'interfaccia è stato realizzato fin da subito per essere user-friendly.

6.4 SCHEMA ELETTRONICO:

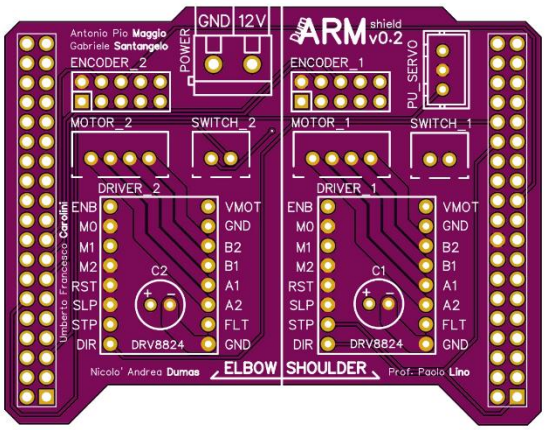
Il seguente schema elettronico riportato è fedele alla realtà del manipolatore e rappresenta i collegamenti dei motori, in alto a sinistra, della STM32, a destra, di corrente in basso a sinistra. La realizzazione della PCB è posteriore alla realizzazione di questo schema.



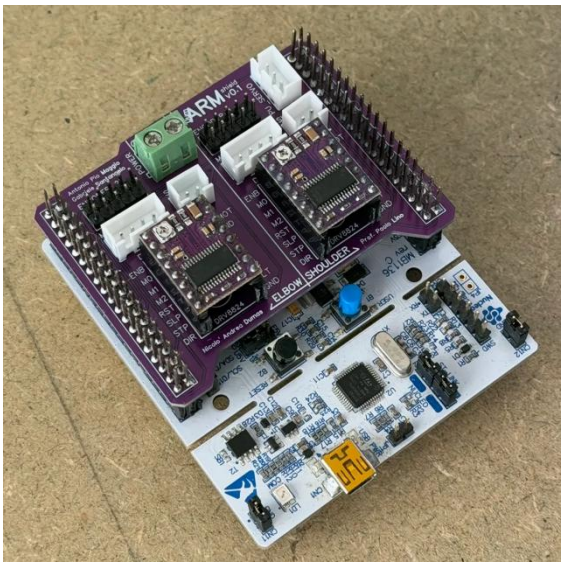
6.5 Schema Elettrico Attuale

6.5 SVILUPPO DELLA PCB:

Per risolvere definitivamente i problemi di diafonia, falsi contatti e disordine visivo ereditati dalla precedente CNC Shield, è stata progettata una *Printed Circuit Board* (PCB) dedicata, denominata DUMARM. Questa scheda custom è stata realizzata interamente sul sito: https://jlcpcb.com/. Quello che la contraddistingue dalla vecchia configurazione è che funge da *hub* centrale di smistamento per l'intero sistema elettrico: essa interconnette il microcontroller (STM32), i driver di potenza per i motori, le linee di segnale degli encoder, i limit switch e l'alimentazione e il segnale digitale per l'end-effector. L'impiego di una singola PCB ha permesso di standardizzare i connettori al modello JST e di isolare le linee di potenza da quelle di segnale, riducendo drasticamente il rumore elettromagnetico e anche l'assemblaggio finale.



6.6 PCB DUMARM



6.7 PCB montata sul controllore

6.6 DESIGN DELLO SCRITTOIO:

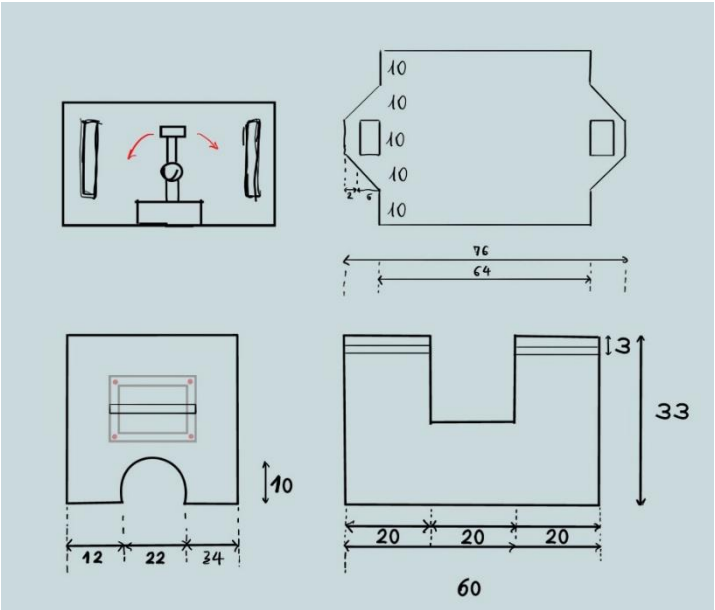
Per garantire i requisiti di trasportabilità, ordine e stabilità meccanica, fondamentali per evitare vibrazioni che comprometterebbero la qualità del tratto grafico, il sistema è stato sottoposto a una completa revisione strutturale. Questo nuova esigenza ha portato all'ideazione e alla realizzazione di uno scrittoio portatile. La struttura non svolge solo il ruolo di base d'appoggio: il piano superiore fornisce una superficie rigida e planare ottimizzata per l'area di lavoro del robot, mentre nella parte inferiore è stato ricavato un vano tecnico nascosto.

6.8 Vano Tecnico

Il piano inoltre è stato rivestito di una pellicola adesiva nera per permettere una pulizia accurata del piano con un semplice panno umidificato. Per garantire la corretta operazione sul foglio sono state integrate nel sistema due morsetti a molla che vanno collocati agli estremi del foglio.

6.9 DESKARM rivestito

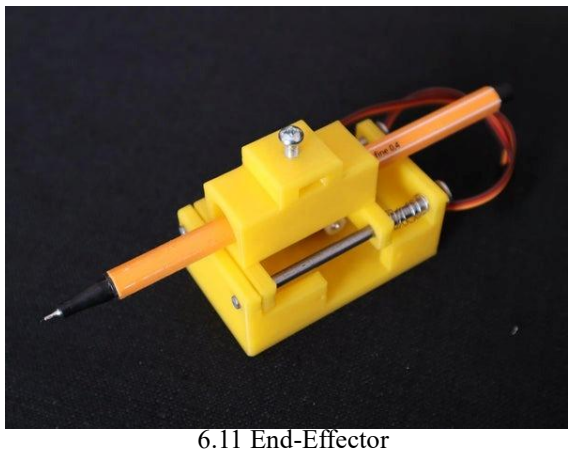
Questo design ha trasformato un progetto concettuale composto da componenti sparsi in una postazione professionale *All-in-One*, esteticamente curata e intrinsecamente sicura, dove l'elettronica è protetta da urti e polvere. Inoltre, per garantire un tracciamento pulito e continuo sul foglio è stata integrata una tovaglia di silicone ancorata al piano che ferma il foglio e non lo fa spostare.



6.10 Scrittoio Progetto iniziale

6.7 END EFFECTOR E MICRO-SERVO:

Affinché il manipolatore SCARA possa eseguire operazioni di disegno o scrittura, il movimento continuo nel piano 2D non era sufficiente, il che ha richiesto l'aggiunta di un grado di libertà ausiliario. L'end-effector è stata sviluppato ricorrendo a tecniche di



6.11 End-Effector

prototipazione rapida, realizzando un supporto customizzato (*pen holder*) capace di ospitare saldamente il pennarello o la penna.
L'intero processo di progettazione è documentato a questo link: https://www.instructables.com/CNC-Pen-Lift-1/ .
In seguito è stato implementato il controllo discreto sull'asse Z (Pen Up / Pen Down) per gestire la traiettoria completa direttamente dal firmware. L'attuazione del meccanismo di sollevamento e abbassamento è stata affidata a un micro-servo MG90D. La scelta di questo componente è dettata dalla necessità di coniugare un peso estremamente ridotto – per minimizzare l'inerzia aggiunta al termine del link L_2 – con una coppia sufficiente a vincere la resistenza meccanica del meccanismo a molla e gravità adottato per il mantenimento della pressione sul foglio.

6.12 Mini Servo MG90S

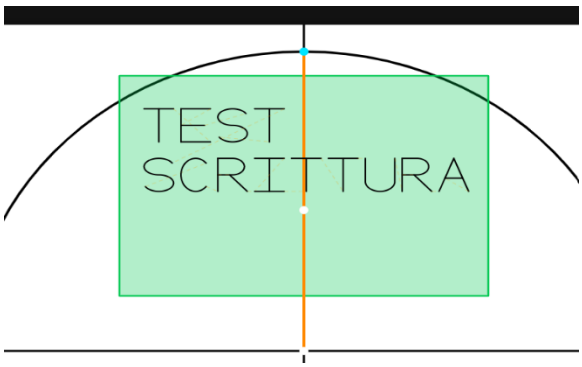
CAPITOLO 7: TEST DI FUNZIONAMENTO

7.1 TEST SUL ROBOT AS-RECEIVED

Sono stati eseguiti test per valutare le modalità di azionamento del manipolatore che sono stati realizzati prima di applicare i miglioramenti software messi a punto in questa tesi.
I test sono stati eseguiti nelle modalità TEXT, DRAWING e IMAGE, nel dettaglio, sono stati presi in considerazione i seguenti criteri di valutazione.
-Precisione del Tracciamento della posizione dei giunti,
-Aderenza al profilo di velocità

-Analisi dello sforzo di controllo:

Il primo test ha riguardato il TEXT MODE per la scrittura digitata e il grafico: (figura 7.1).

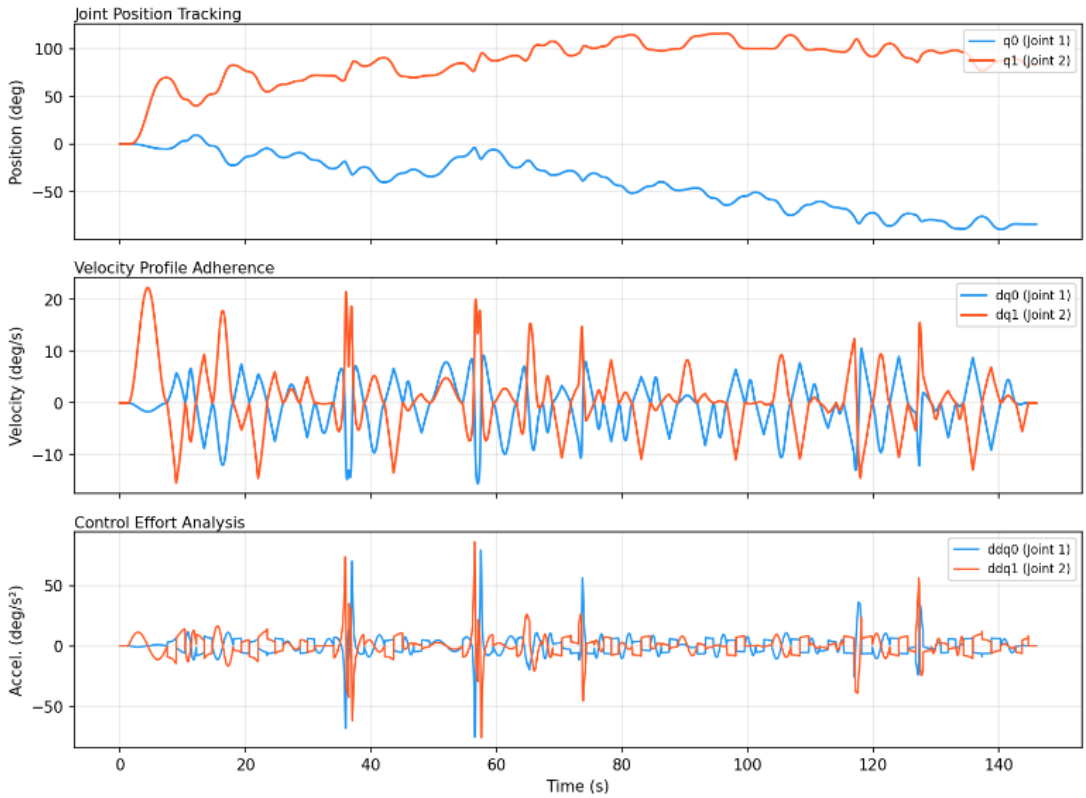


7.1 Test di Scrittura

Qui il robot traccia le parole "TEST SCRITTURA".

È un'operazione che richiede quasi 150 secondi e i grafici (in figura 7.2) ci fanno capire quanto sia impegnativo scrivere per un braccio meccanico.

La scomposizione in modo semplice della posizione dei giunti (Joint Position Tracking): mostra una tendenza generale di tantissime piccole oscillazioni continue. Ogni piccola "ondina" rappresenta i minuscoli movimenti necessari per tracciare le curve e i tratti delle singole lettere (come le pance della 'R' o le curve della 'S').

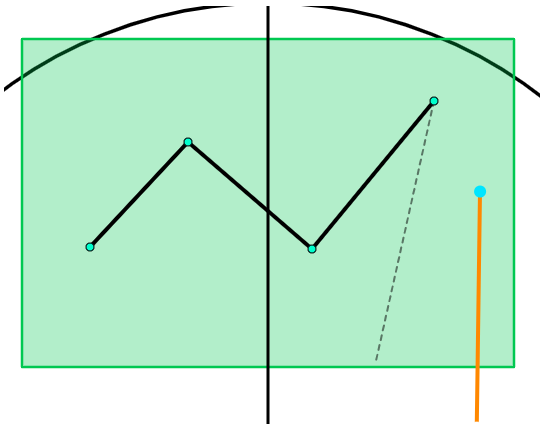


7.2 Grafico senza ottimizzazione del testo

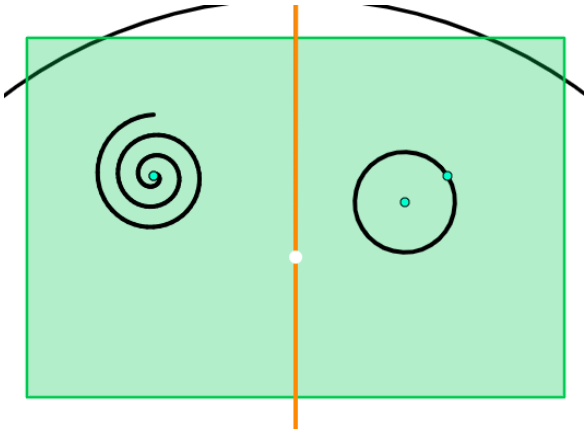
Per il profilo di velocità (Velocity Profile Adherence), il grafico evidenzia un continuo su e giù dei valori. Le linee blu e arancioni attraversano la linea dello zero (velocità nulla) decine di volte. Questo accade perché per scrivere lettere staccate, il robot deve continuamente partire, fare un tratto brevissimo, fermarsi, sollevare la penna se necessario, e ripartire. Non c'è tempo per raggiungere una velocità fluida e costante.

Per l'accelerazione (Control Effort Analysis), il grafico presenta picchi altissimi e improvvisi. Le lettere sono piene di angoli stretti, cambi di direzione repentini e tratti corti. Ogni volta che la penna deve fare uno scatto o una frenata improvvisa per disegnare un dettaglio di una lettera, il controller dei motori elabora una mole notevole di dati.

Il secondo test è diviso in due parti, il primo tracciare una linea spezzettata e il secondo una spirale e una circonferenza:



7.3 Disegno Linea 1



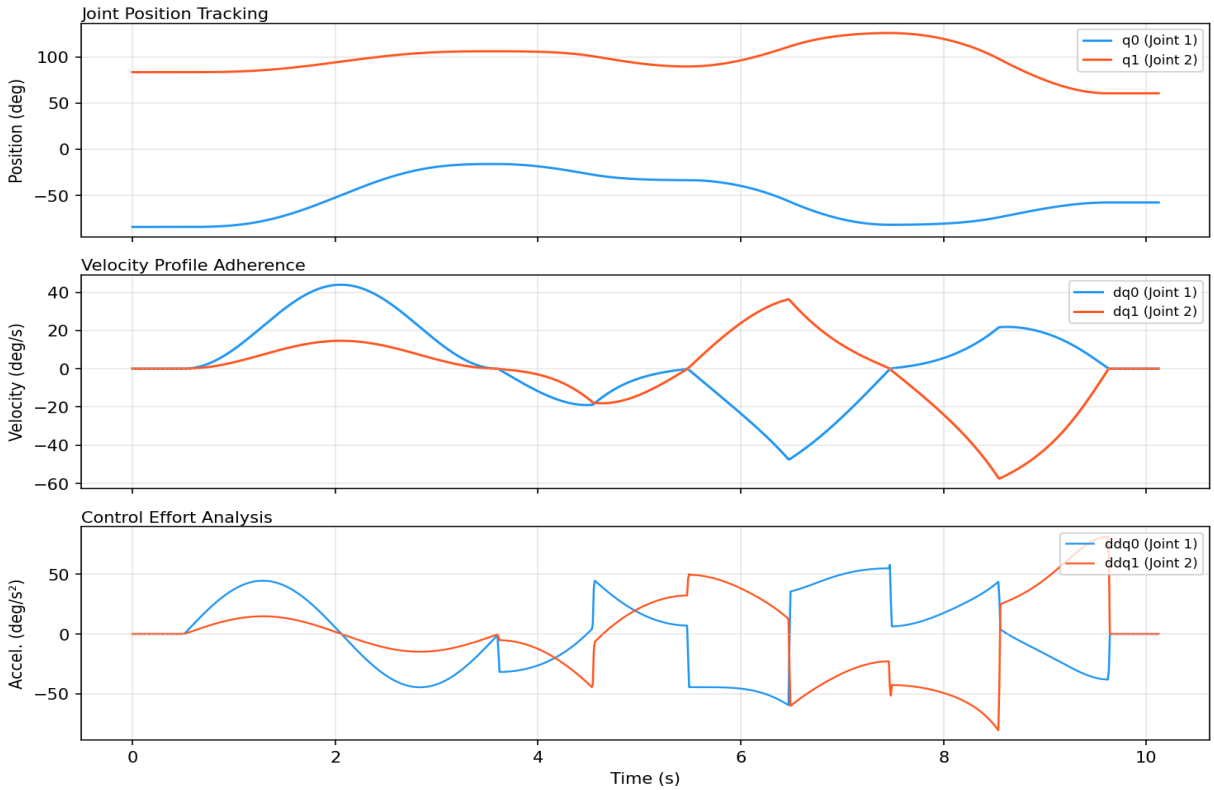
7.4 Disegno Spirale e Cerchio

Per la **linea** il grafico descrive cosa succede nel tempo, per circa 10 secondi, ai due motori del robot: il Giunto 1 (linea blu) e il Giunto 2 (linea arancione).

La posizione (Joint Position Tracking) mostra come cambiano gli angoli dei due giunti per far muovere il braccio. Le curve sono fluide e continue, a dimostrazione che il robot passa dolcemente da un punto all'altro della traiettoria.

La velocità (Velocity Profile Adherence) indica quanto velocemente stanno ruotando i giunti. La cosa più importante da notare qui è che le linee tornano a zero in momenti specifici (ad esempio a 3.5s, 6.5s, 7.5s). Questo significa che il robot si ferma completamente ad ogni vertice del disegno a zig-zag, per poi ripartire verso il punto successivo. Le curve hanno una forma arrotondata all'inizio indicando che il robot accelera e decelera gradualmente su ogni segmento e poi sono spezzate con cambi repentini.

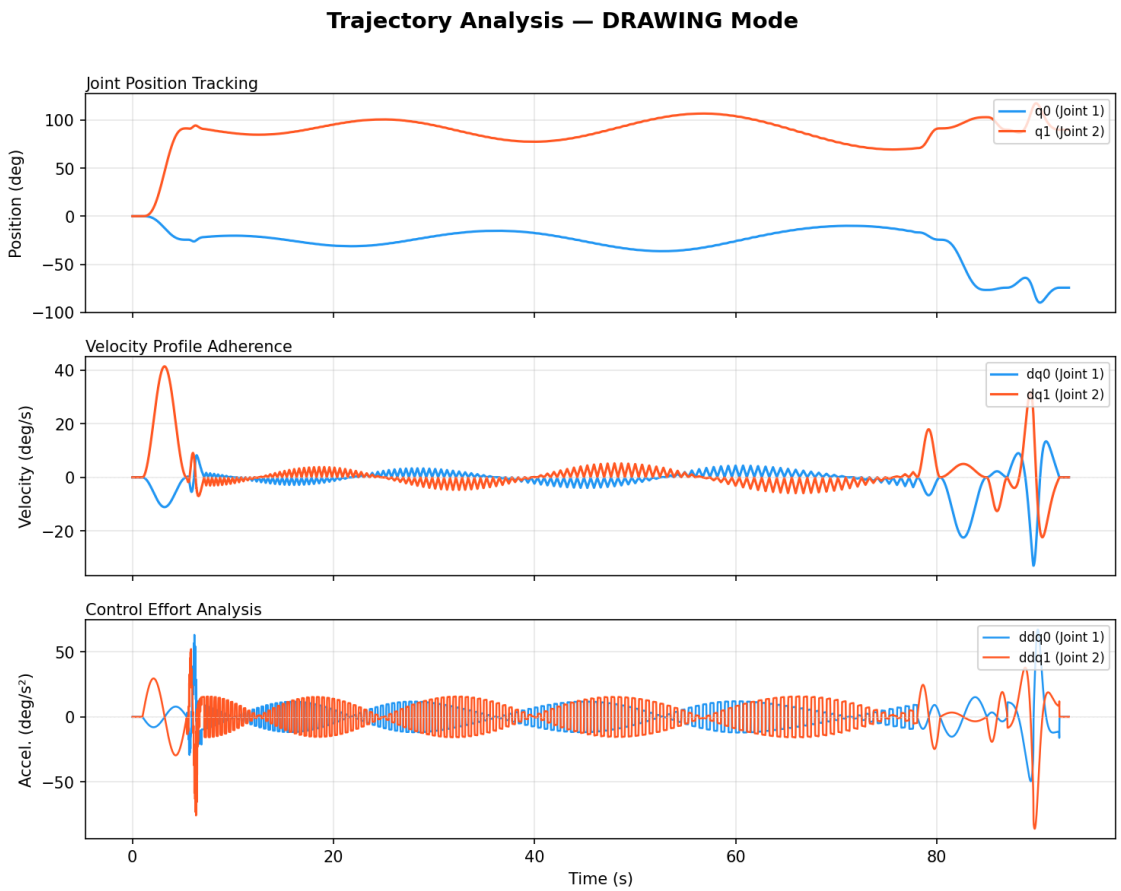
L’accelerazione (Control Effort Analysis) rappresenta lo sforzo richiesto ai motori. Qui si notano dei salti netti e verticali (discontinuità). Questo andamento a gradini indica che il comando di accelerazione cambia in modo istantaneo, molto comuni per gli spostamenti precisi da un punto all'altro.



7.5 Grafico senza ottimizzazione del percorso.

Per il **tracciamento curvo** il primo grafico di figura 7.6 ci svela come funziona il robot quando deve disegnare delle linee non dritte. Se si guarda la parte centrale del grafico, dai 10 agli 80 secondi, le linee dei due giunti, riferite alla posizione (Joint Position Tracking) si muovono creando delle lunghe e morbide curve. Questo andamento oscillatorio continuo è il modo in cui i due motori combinano i loro angoli per generare un movimento circolare perfetto sul foglio. Verso gli 80 secondi questo fenomeno è notevole: il robot solleva la penna e si sposta più in basso per iniziare a disegnare il cerchio isolato.

Per la velocità (Velocity Profile Adherence) secondo grafico di figura 7.6, all'inizio c'è una grande curva, quando il robot va velocemente al punto di partenza, la parte davvero curiosa è durante il disegno: invece di una linea fluida, l'andamento della velocità oscilla continuamente creando una fitta trama. L'ampiezza di queste vibrazioni cambia man mano che la spirale si allarga o si restringe.



7.6 Grafico senza ottimizzazione della spirale e del cerchio

Accelerazione (Control Effort Analysis) terzo grafico di figura 7.6. Questo grafico è la chiave per comprendere cosa sta succedendo. La fitta fascia centrale indica un'oscillazione continua e rapidissima dei comandi, significa che il controllore del robot non sta eseguendo una vera e propria curva continua. Illude l'utente spezzettando la spirale e il cerchio in centinaia di piccolissimi segmenti dritti. Per passare da un micro-segmento all'altro, i motori devono fare una minuscola accelerazione e frenata.

Per il test in IMAGE MODE di un’immagine complessa, al manipolatore non è stato fatto seguire un percorso geometrico o lettere predefinite, ma è stata riprodotta un'immagine

7.7 Fotografia di partenza

7.8 Disegno del Manipolatore

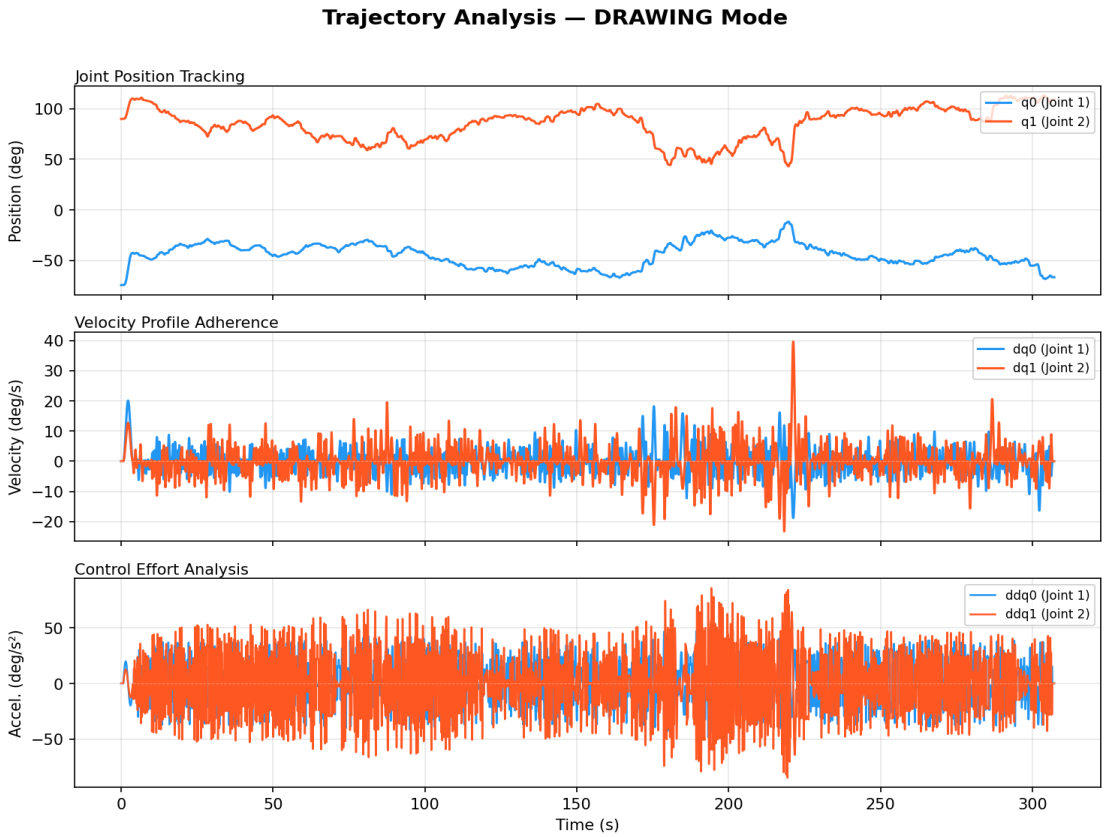
complessa, quella di un cagnolino. Il tracciamento dell’immagine, che dura 310 secondi, è caratterizzato dal seguente comportamento. (vedi figura 7.9 per i grafici).

Posizione (Joint Position Tracking). Anche se c'è un movimento generale le linee di posizione sono incredibilmente frastagliate e ogni minimo gradino su quelle curve rappresenta il robot che disegna un singolo pelo, una macchiolina del muso o i dettagli della scritta "MICKEY".

Velocità (Velocity Profile Adherence): Se nei grafici precedenti vedevamo delle curve riconoscibili, qui vediamo un "rumore" continuo. Il robot non ha quasi mai la possibilità di raggiungere una velocità di crociera fluida. È un continuo scattare in avanti, frenare, cambiare direzione, alzare e abbassare la penna per fare tratteggi brevi ed erratici. Le linee incrociano lo zero (fermo) innumerevoli volte al secondo.

Accelerazione (Control Effort Analysis), il grafico più impressionante di tutti, in cui sembra un blocco di colore solido arancione e blu. Questo "muro" di picchi significa che i comandi di accelerazione e decelerazione vengono inviati a frequenze altissime. In termini fisici, questo si traduce in vibrazioni intense. I motori e i giunti meccanici stanno subendo uno stress fortissimo perché devono vincere l'inerzia del braccio per fare micro-scatti in direzioni sempre diverse.

In sintesi, disegnare a partire dall'estrazione dei contorni di un'immagine fotografica è il lavoro in assoluto più gravoso per il robot. A causa dei tratti corti e scollegati tipici di questo tipo di elaborazione, la macchina lavora costantemente sotto sforzo massimo, con continue vibrazioni e frenate brusche.



7.9 Grafico senza ottimizzazioni dell'immagine

Il test di elaborazione testuale rivela un sistema costantemente soggetto a regimi transitori. L'elevata densità di tratti brevi e angoli acuti, necessaria per la tracciatura dei caratteri tipografici, impedisce agli attuatori di stabilizzarsi su un regime di velocità di crociera. L'analisi dello sforzo di controllo in questa fase mostra picchi impulsivi di elevata ampiezza; si tratta della conseguenza diretta di una continua richiesta di repentine accelerazioni e decelerazioni, che stressa significativamente l'hardware senza mai permettere una fluidità esecutiva.

Passando all'analisi della traiettoria spezzata, emerge in modo palese l'adozione di profili di velocità trapezoidali elementari. Il controllore impone un arresto completo della

dinamica in corrispondenza di ogni nodo geometrico del percorso a zig-zag, rivelando un approccio cinematico strettamente limitato alla logica punto-a-punto. Tale rigidità algoritmica genera discontinuità a gradino nei profili di accelerazione; questi salti si traducono in un jerk teoricamente infinito durante i cambi di direzione, inducendo inevitabili e cicliche sollecitazioni tensionali sui giunti.

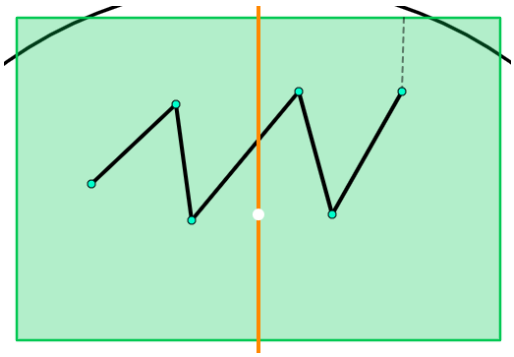
Le medesime lacune algoritmiche divengono ancora più evidenti nell'esecuzione delle primitive curvilinee, dove si palesa la totale assenza di un modulo di interpolazione circolare nativa nel controllore. Strutture come la circonferenza e la spirale vengono processate esclusivamente tramite un'approssimazione lineare a tratti. La drastica discretizzazione delle curve continue in centinaia di micro-segmenti rettilinei innesca un severo fenomeno di chattering nei profili di velocità e accelerazione. Gli attuatori vengono così sottoposti a un regime vibratorio ad alta frequenza nel costante ma inefficiente tentativo di mantenere l'aderenza al percorso nominale.

Questo progressivo degrado cinematico raggiunge infine il suo apice strutturale nella riproduzione dell'immagine raster vettorializzata. L'algoritmo di estrazione dei contorni dalla fotografia genera un database di waypoint disordinati, frammentati e caotici. In assenza di un modulo per l'ottimizzazione globale del percorso o per la minimizzazione dei micro-spostamenti a vuoto, il sistema tenta di eseguire ogni micro-tratto individualmente. Il segnale di controllo osservato risulta visivamente e matematicamente assimilabile a un rumore bianco ad alta densità. Questa condizione dimostra che la meccanica è spinta ai margini estremi delle proprie tolleranze di smorzamento per compensare la totale assenza di algoritmi di raccordo delle traiettorie.

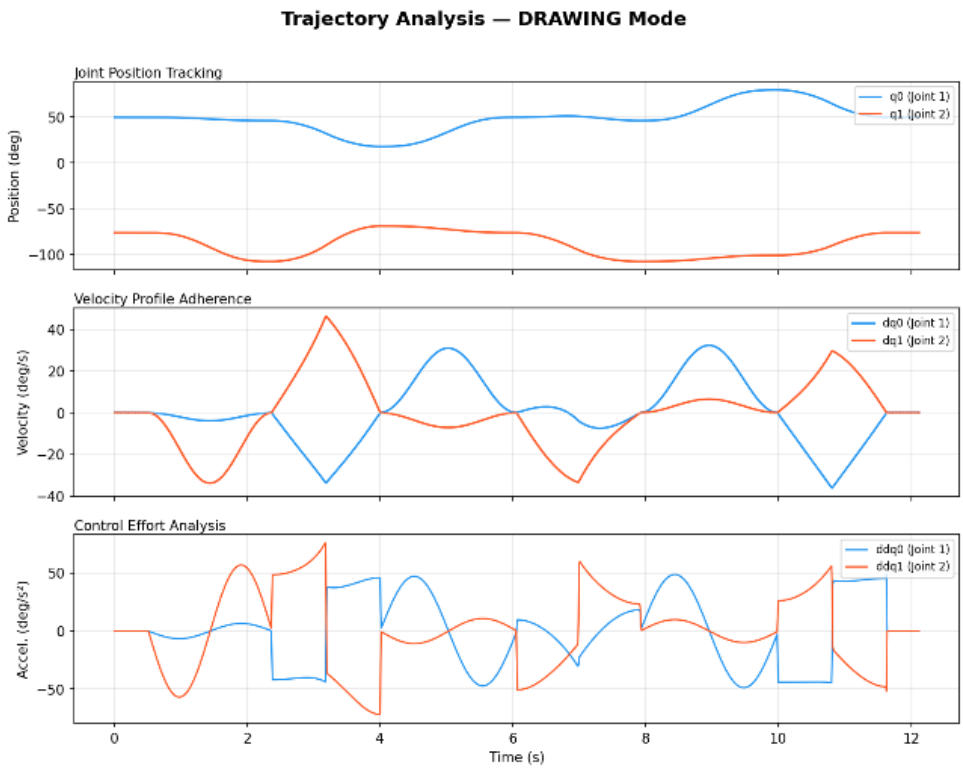
7.2 TEST CON ROBOT MODIFICATO

Una volta implementate le ottimizzazioni, spiegate nel capitolo 4, è stato rieseguito il testo solo in DRAWING MODE, per confrontare lo stato iniziale da quello migliorato per comprendere se fosse utile continuare con questo profilo o utilizzare il profilo del polinomio di quinto grado.

Il test di confronto è stato realizzato per una linea spezzettata e confrontando i grafici si è arrivati ad una conclusione assoluta, a livello matematico c'è un netto miglioramento, ma a livello fisico ci sono ancora problemi.



7.10 Disegno Linea 2



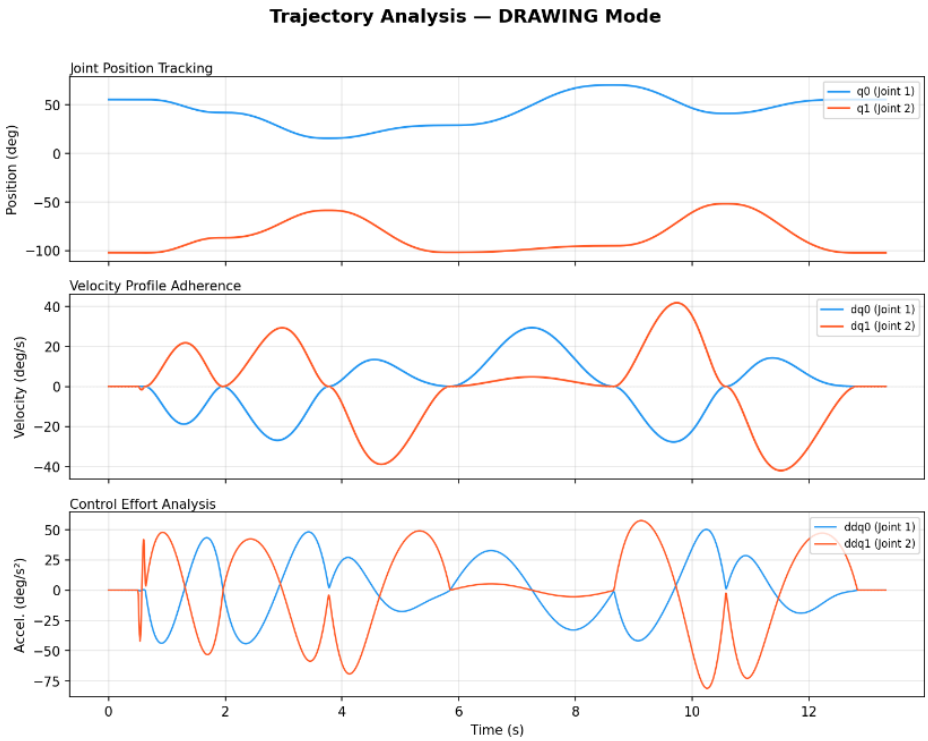
7.11 Grafico Linea con ottimizzazioni

Ecco cosa è cambiato e la situazione dopo aver applicato le modifiche.

Il miglioramento nel secondo grafico è decisamente più pulito nel tracciamento della traiettoria. In Control Effort Analysis e Accelerazione, notiamo dei blocchi orizzontali ben definiti e

squadrati, tra i secondi 2.5 e 4, o tra 10 e 12. Questo dimostra che l'implementazione è avvenuta con successo: ora le fasi di accelerazione e decelerazione costante sono calcolate in modo corretto. Nel primo grafico, invece, l'accelerazione era molto più instabile e irregolare. Il problema non è del tutto risolto, sono presenti salti istantanei in cui il robot passa da un'accelerazione nulla a una massima in zero secondi. Nella realtà fisica, questo genera uno jerk infinito. Il risultato è che il manipolatore vibra e i motori sforzano e i controllori vanno in errore.

La valutazione è chiara il profilo trapezoidale è buono ma non del tutto preciso, per questa condizione si è preferito valutare il profilo del polinomio di quinto grado.



7.12 Grafico Linea con Polinomio di quinto grado

L'analisi cinematica evidenzia l'inadeguatezza del profilo trapezoidale per il tracciamento multi-segmento: garantendo unicamente una continuità di classe C^1, esso genera un'accelerazione a gradino e un jerk tendente all'infinito, causa di severe sollecitazioni meccaniche. L'adozione di una spline polinomiale quintica risolve tale criticità imponendo l'annullamento di velocità e accelerazione nei nodi di giunzione, assicurando così una continuità globale di classe C^2. I dati empirici validano il modello teorico, mostrando come la transizione alla formulazione quintica elimini le singolarità e restituisca un profilo di sforzo armonico e continuo. Si rileva unicamente una residua anomalia computazionale all'istante di innesco $t \cong 0.5s$, ove un transitorio impulsivo

nell'accelerazione denota una locale violazione dei vincoli di derivabilità, riconducibile a una probabile errata inizializzazione delle condizioni al contorno del primo segmento.

Se la derivazione matematica impone rigorosamente che l'accelerazione iniziale sia nulla $c_2 = 0 \rightarrow \ddot{s}(0) = 0$, perché il problema non risiede nella matematica della traiettoria impostata, ma nel passaggio dal pianificatore teorico al loop di controllo reale.

Ecco 2 problemi principali che secondo i test e una valutazione teorica causano il picco a $t \cong 0.5s$, nonostante il polinomio imponga che $s''(0) = 0$:

1) Disallineamento Iniziale, è un errore di Inseguimento a t_0:

Se la posizione fisica reale del giunto al momento dello start differisce anche solo di frazioni di grado dal valore $s(0)$,posto pari a 0 nel codice, il controllore PID esegue un gradino di errore posizionale al primissimo ciclo. Il PID reagisce istantaneamente erogando una coppia massima per colmare questo problema, generando il picco di accelerazione registrato.

2) Gestione del Tempo Scalato:

Il tempo è stato normalizzato per $\tau = \frac{t}{t_f}$.

Per la **regola della catena**, applicata per calcolare la derivata di funzioni composte, l'accelerazione nel dominio del tempo reale scala con il quadrato della durata del segmento: $\ddot{q}(t) = \frac{1}{t_f^2} \times s''(\tau)$

Se nel primissimo segmento di inizializzazione il tempo t_f è settato a un valore infinitesimo qualsiasi microscopico errore numerico nei coefficienti viene amplificato quadraticamente, risultando in un picco a gradino.

CONCLUSIONI E SVILUPPI FUTURI

Il manipolatore DESKARM combina una meccanica di precisione con un controllo digitale all'avanguardia, offrendo movimenti fluidi e ripetibili al millimetro.
Grazie a questo lavoro di tesi, è stato possibile implementare una serie di nuove funzioni quali, scrittura, disegno di figure e conversione da immagine a traiettorie. attraverso la programmazione sia in ambito firmware e dell'interfaccia grafica il manipolatore è più fluido nei movimenti e nelle sue capacità di realizzazione di contenuti grafici. Inoltre, il sistema da essere un semplice braccio privo di un'anima, tramite la realizzazione di componenti hardware quali PCB custom, scrittoio e realizzazione dell'end-effector, tramite stampa 3D, hanno trasformato il braccio in uno strumento creativo.
I risultati ottenuti hanno dimostrato quanto le ottimizzazioni introdotte hanno realmente contribuito ad un miglioramento del sistema.
L'intero progetto sia il firmware che l'interfaccia è disponibile su GitHub [25].
I miglioramenti apportati con questa tesi rendono il manipolatore un sistema intelligente e versatile per la facilità con cui può essere adoperato per scrittura e disegno.
Tuttavia, l'area sensoristica è ancora agli albori, poiché manca un sistema che garantisca il monitoraggio del workspace, elemento chiave per migliorare l'esperienza user-friendly.
Inoltre, il contesto relativo all'End-Effector è ancora in fase di sviluppo poiché al momento è stato realizzato solo il CAD 3D.
Il manipolatore DESKARM è un progetto di tesi applicabile in diversi contesti, in ambito didattico per la realizzazione di schemi elettrici, disegni industriali e architettonici.
Sebbene attualmente il sistema richieda la piena capacità di utilizzo di un computer per poter essere controllato tramite l'interfaccia grafica, l'architettura del progetto si presta perfettamente all'integrazione futura di sistemi di controllo alternativi e accessibili come comandi vocali, sistemi di eye-tracking o interfacce neurali. In questa prospettiva a lungo termine, il manipolatore potrebbe essere introdotto nelle scuole o in centri riabilitativi per permettere a persone con gravi disabilità motorie di scrivere e disegnare. In definitiva, questo lavoro dimostra come una solida progettazione robotica di base sia il primo passo necessario per trasformare una macchina esecutrice in un futuro strumento di inclusione sociale.

Bibliografia e Sitografia

[1] Robot Institute of America, *Definition of a Robot*, Robot Institute of America, 1980.
[2] International Organization for Standardization, ISO 8373:2021 Robotics - *Vocabulary*, ISO, 2021.
[3] International Organization for Standardization, ISO/TR 8373-2.3 Manipulating industrial robots - *Vocabulary*, ISO, 2023.
[4] H. Makino e N. Furuya, "*Selective compliance assembly robot arm*" Proceedings of the 1st International Conference on Assembly Automation (ICAA), pp. 77–86, 1980.
[5] A. Gasparetto e L. Scalera, "*A Brief History of Industrial Robotics in the 20th Century*" Advances in Historical Studies, pp. 24–35, 2019.
[6] B. Siciliano, L. Sciavicco, L. Villani e G. Oriolo, *Robotica: Modellistica, Pianificazione e Controllo*, McGraw-Hill, 2008.
[7] P. I. Corke, "*A Robotics Toolbox for MATLAB*," *IEEE Robotics and Automation Magazine*, pp. 24-32, 1996.
[8] P. I. Corke, Robotics, *Vision and Control: Fundamental Algorithms in MATLAB*, Springer Tracts in Advanced Robotics, 2011.
[9] L. Biagiotti e C. Melchiorri, Trajectory Planning for Automatic Machines and Robots, Springer, 2010.
[10] R. Bussola, G. Incerti, C. Remino e M. Tiboni, "*S-Curve Trajectory Planning for Industrial Robots Based on Curvature Radius*" Robotics, pp. 1-26, 2025.
[11] U. Ramer, "An iterative procedure for the polygonal approximation of plane curves," Computer Graphics and Image Processing, pp. 244–256, 1972.
[12] D. H. Douglas e T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," Cartographica, pp. 112–122, 1973.
[13] G. M. Chaikin, "An algorithm for high-speed curve generation," Computer Graphics and Image Processing, pp. 346–349, 1974.
[14] T. Y. Zhang e C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," Communications of the ACM, pp. 236–239, 1984.
[15] A. V. Hershey, Calligraphy for Computers, U.S. Naval Weapons Laboratory Report NWL-2101, 1967.
[16] G. Bradski, "*The OpenCV Library*" Dr. Dobb's Journal of Software Tools, 2000.
[17] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, pp. 90–95, 2007.
[18] C. R. Harris, K. J. Millman, S. J. van der Walt et al., "Array programming with NumPy," Nature, pp. 357–362, 2020.

[19] STMicroelectronics, STM32F411RE: ARM Cortex-M4 32b MCU+FPU Datasheet, 2014.
[20] Texas Instruments, DRV8825 Stepper Motor Controller IC Datasheet, 2014.
[21] E. Gamma, R. Helm, R. Johnson e J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.
[22] International Organization for Standardization, ISO 6983-1:2009 Automation systems and integration - Numerical control of machines, 2009.
[23] D. R. Finley, HSP Color Model - Alternative to HSV (HSB) and HSL, 2006.
[24] Protoneer, Arduino CNC Shield - 100% GRBL Compatible, 2013.
[25] https://github.com/NicolADumas/DESKARM

RINGRAZIAMENTI